

Sheet 2/87 · LeetMastery Study-Order · 36-up Landscape · 6x6 Round 1 · High · Easy p.70

Sheet 3/87 · LeetMastery Study-Order · 36-up Landscape · 6x6 Round 1 · High · Easy p.106

Sheet 4/87 · LostMastery Study-Order 2/41 · 36-up Landscape · 6x6 Round 1 · High · Easy p.142

```
# The guess API is already defined for you.
# @param num, your guess
# @return -1 if num is higher than the picked number
#          1 if num is lower than the picked number
#          0 if num is equal to the picked number (you are right)
def guess(num):
    pass

class Solution:
    def guessNumber(self, n):
        """
        :type n: int
        :rtype: int
        """
        # Python solution using binary search
        class Solution:
            def guessNumber(self, n):
                # Initialize low and high pointers
                low, high = 1, n
                # Continue binary search until low equals
                # high
                while low < high:
                    # Guaranteed midpoint safely to prevent
                    # overflow
                    mid = low + (high - low) // 2
                    # Call the guess API with mid
```

```

        result = guessAndId()
        # If the guess is correct, return and
        # if result == 0:
        return true
    high pointer
    elif result == -1:
        high = result - 1
    # If guess was too low, adjust the low
    pointer
    else:
        low = mid - 1
    # Return low (initially should never reach
    here if pick exists in [1, N])
    return low
}
//C++
// # The guess API is already defined for you.
//
// Returns int, your guess.
// Throws -1 if you are higher than the picked
// number
// # If you are lower than the picked number
// the return value is 0
// # If guess is correct return 1
//
class Solution {
public:
    int guessNumber(int n) {
        int low = 1, high = n;
        while (low <= high) {
            int mid = (low + high) / 2;
            int result = guess(mid);
            if (result == 0)
                return mid;
            else if (result == -1)
                high = mid - 1;
            else
                low = mid;
        }
    }
};

```

```
return hlist.hlist_of(range(1, n + 1), 0,
key=lambda x: -qsum(x))
```

Binary Search

704 Binary Search

Easy

Intuition: Simplified: WalkCC - LastCase

Array: Binary Search

LeetCode 359

Time Complexity: O(log n) Space Complexity: O(1)

Problem

Given an array of integers nums which is sorted in ascending order and an integer target, write a function to search target in nums. If target exists, return its index. Otherwise, return -1.

You must write an algorithm with O(log n) runtime complexity.

Example 1:

Input: nums = [-1,0,1,2,3,4,5], target = 0

Output: 1

Explanation: 0 exists in nums and its index is 1

Example 2:

Input: nums = [-1,0,1,2,3,4,5], target = 2

Output: 3

Explanation: 2 does not exist in nums so return -1

Constraints:

1 <= nums.length <= 10⁴

Round 1 - High - Easy

p.148

```

• 104 < mem0, target < 104
• All the integers in nums are unique
• nums is sorted in ascending order.

```

Brute Force (Python)

```

class Solution:
    def search(self, nums: List[int], target: int) -> int:
        for i, val in enumerate(nums):
            if val == target:
                return i
        return -1

```

Community Solutions (Python)

```

class Solution:
    def search(self, nums: List[int], target: int) -> int:
        i = bisect.bisect_left(nums, target)
        return -1 if i == len(nums) or nums[i] != target else i

```

LeetCode

Round 1 - High Easy p.140

```
class Solution:
    def search(self, nums: List[int], target: int)
    - int:
        l, r = 0, len(nums) - 1
        while l < r:
            mid = (l + r) // 2
            if nums[mid] == target:
                return mid
            elif nums[mid] < target:
                l = mid + 1
            else:
                r = mid
        return l if nums[l] == target else -1

+ Comments
# Python code for binary search

def search(nums, target):
    # Set initial boundaries for the search
    left, right = 0, len(nums) - 1

    # Loop until the search space is empty
    while left < right:
        # Calculate middle index to avoid overflow
        mid = left + (right - left) // 2
```

```
# Check if the mid element matches the
target
if nums[mid] == target:
    return mid
# If target is smaller than mid element,
narrow search to left half
elif target < nums[mid]:
    right = mid - 1
# Else, narrow search to right half
else:
    left = mid + 1

# Target not found, return -1
return -1

# Example usage
print(nums.index(11, 1, 5, 9, 12, 9)) # Expected
output: 4
```

<VLCm

```
class Solution:
    def search(self, nums: List[int], target: int)
    => int

    left, right = 0, len(nums) - 1
    while left < right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid
    return left if nums[left] == target else -1
```

Matrix

Round 1 - High - Easy - 5 questions

Round 1 - High - Easy

p.113

Matrix	
#422 Valid Word Square	54%
LeetCode - SingleTest - WA/CC - LeetCode	
Array - Matrix	
Easy Premium 58 Your Company's QN "1" where 1 is the number of words and 1 is the average length of the words. Space Complexity: O(1) additional space is used for the checking procedure. Problem Given an array of strings words, return true if it forms a valid word square. A sequence of strings forms a valid word square if the <i>i</i>th row and columns read the same string, where 0 <= i < min(words.length, words[0].length). Example 1:	

a	b	c	d
b	n	r	t
c	r	m	y
d	t	y	e

Shant's words are ["abdr","bnrt","cmry","Wpqr"]
 Output: true
 Explanation:
 The 1st row and 1st column both read "abdr".
 The 2nd row and 2nd column both read "bnrt".
 The 3rd row and 3rd column both read "cmry".
 The 4th row and 4th column both read "Wpqr".
 Therefore, it is a word read square.

Example 2:

a	b	c	d
b	n	r	t
c	r	m	
d	t		

```

Input words = ["abcd","bcd","cde","bcd"]
Output: true
Explanation:
The 1st row and 1st column both read "abcd".
The 2nd row and 2nd column both read "bcd".
The 3rd row and 3rd column both read "cde".
The 4th row and 4th column both read "bcd".
Therefore, it is a valid word square.

```

Example 3:

b	a	l	l
a	r	e	a
r	e	a	d
l	a	d	y

```
Input: words = ["ball","area","read","day"]  
Output: false  
Explanation:  
The 1st row reads "read" while the 3rd column reads  
"lead".  
Therefore, it is NOT a valid word square.  
Constraints:  
n == words.length <= 500  
1 <= words[i].length <= 500  
words[i] consists of only lowercase English letters.
```

Round 1 - High - Easy

3.157

```

# Brute Force (Python)

def brute_force_approach(
    class Solution:
    def validSquare(self, words):
        # Brute Force O(N^2 * n^2) - Flood fill from
        every cell = isNeighbor(), isNeighbor()
        result = 0
        for i in range(0, len(words)):
            visited = set()
            def flood_fill(i, j):
                if (i, j) in visited or not
                    return
                visited.add((i, j))
                for dx, dy in [(0, 1), (0, -1), (1, 0), (-1, 0)]:
                    flood_fill(i + dx, j + dy)
                result += len(result), len(visited)
            return result

```

```

class Solution:
    def validWordSquare(self, words: List[str]) -> bool:
        for i, w in enumerate(words):
            for j, c in enumerate(word):
                if len(words) <= j or len(words[j]) <= i:
                    return False
            if c != words[j][i]:
                return False
        return True

+ LeetCode

+ LeetCode
def validWordSquare(self, words: List[str]) -> bool:
    n = len(words)
    for i, w in enumerate(words):
        for j, c in enumerate(word):
            if w <= n or j <= len(words[j]) or
            c != words[j][i]:
                return False
    return True

+ Completed

```

```
# Function to check if the provided words form a
valid word square
def validWordSquare(words):
    # Iterate over each row with its index
    for i in range(len(words)):
        # Iterate over each character in the
        current row
        for j in range(len(words[i])):
            # Check if the corresponding column
            exists
            if j == len(words) or i ==
len(words[i]) or words[i][j] != words[j][i]:
                return False
            return True

# Example test case
words = ["abcd", "bcde", "cdef", "efgh"]
print(validWordSquare(words)) # Expected output:
True

# TC: O(n^2)
```

```
class Solution:
    def validateSquare(self, words: List[str]) ->
bool:
    n = len(words)
    for i, w in enumerate(words):
        for j, v in enumerate(words):
            if j == n or i == len(words)-1] or
            i < j: words[j][i]:
                return False
    return True
```

Matrix

#566 Reshape the Matrix

54%

leetcode.com > SimpleArray > Matrix > LeetCode

Array - Matrix - Simulation

Link > Discuss

Time Complexity: $O(n \times m)$ as we iterate through all elements of the matrix once. Space Complexity: $O(1)$ as we do additional list used to store flattened elements for the new matrix.

Problem

In MATLAB, there is a handy function called reshape which can reshape an $m \times n$ matrix into a new one with a different size $r \times c$ keeping its original data.

You are given an $m \times n$ matrix mat and two integers r and c representing the number of rows and the number of columns of the wanted reshaped matrix.

The reshaped matrix should be filled with all the elements of the original matrix in the same row-traversing order as they were.

If the reshape operation with given parameters is possible and legal, output the new reshaped matrix; Otherwise, output the original matrix.

Example 1:

Round 1

High > Easy

3/362

```

Input: mat = [[1,2],[3,4]], r = 2, c = 4
Output: [[1,2,3,4]]

Example 2:

Input: mat = [[1,2],[3,4]], r = 2, c = 4
Output: [[1,2],[3,4]]

Comments:
n = mat.length
m = mat[0].length
if (m <= n) return mat
for (int i = 0; i < m; i++)
    mat[i] = merge(mat[i], c - 1);
return mat;

```

```

Brute Force (Python)

# Brute Force Approach
class Solution:
    def matrixBlockSum(self, mat: List[List[int]], k: int) -> List[List[int]]:
        # Brute Force O(R^2 * C^2) - Flood Fill from
        # every cell:
        m, n = len(mat), len(mat[0])
        result = [[0] * n for _ in range(m)]
        for r in range(m):
            for c in range(n):
                visited = set()
                def flood(r, c):
                    if (r, c) in visited or not
                        return
                    visited.add((r, c))
                    for dr, dc in [(0, 1), (0, -1), (1, 0), (-1, 0)]:
                        flood(r+dr, c+dc)
                result[r][c] = sum(mat[r1][c1] for (r1, c1) in visited)
                return result
        return result

```

• Complexity: $O(R^2 \cdot C^2)$

• Summary: Simple but inefficient. Not recommended for large inputs.

```

class Solution:
    def matrixKthSmallest(self, matrix: List[List[int]]):
        n = len(matrix)
        if n == 1:
            return matrix[0][0]
        return min
        ans = [0] * n
        for i in range(n):
            for j in range(n):
                ans[i] += matrix[i][j]
        return ans

```

```

1 #Multiply
2
3 # Function to reshape the matrix and
4 # multiply them together. r, c1
5 def calculate_dimensions_of_the_original
6     matrix:
7     n = len(matrix)
8     m = len(matrix[0])
9
10 # Check if reshape is possible
11 if m * n == r * c:
12     return True
13
14 # Flatten the original matrix into a list
15 flat_list = []
16 for row in mat:
17     for col in row:
18         flat_list.append(row)
19
20 # Create the new matrix by slicing the flat
21 list
22 new_matrix = []
23 for i in range(len(flat_list), c):
24     new_matrix.append(flat_list[i::c])
25
26
27 return new_matrix
28
29 # Example call:
30 # print(matrix reshape([1,2,3],[3,4], 3, 4))
31
32 # LCa

```

```

class Solution:
    def matrixKthSmallest(self, mat: List[List[int]],
                           k: int) -> List[List[int]]:
        n, m = len(mat), len(mat[0])
        if m == n == k:
            return mat
        ans = [0] * k
        for i in range(n):
            for j in range(m - k + 1):
                ans[j // k * m + j % k] = mat[i][j + k - 1]
        return ans

```

Matrix

#766 Toeplitz Matrix

SAC

LeetCode

SimplexLeet

Leet

LeetCode

Array

Matrix

Link

CodeSignal

Time Complexity

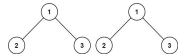
$O(n^2)$

Space Complexity

$O(1)$

Problem

Given an $m \times n$ matrix, return true if the matrix is Toeplitz. Otherwise, return false.
A matrix is Toeplitz if every diagonal from top-left to bottom-right has the same elements.
Example 1:



Round 1

High

Easy

p.168

```

Input: matrix = [[1,2,3,4],[1,3,3,1],[3,3,3,1]]
Output: true
Explanation:
The matrix above is symmetric, because the diagonals are:
"1331", "13 31", "1 1", "1 1", "1 1", "1 1", "1 1", "1 1", "1 1", "1 1".
In each diagonal all elements are the same, so the answer is True.
Example 2:

```

```

Input: matrix = [[1,2,3,4],[1,3,3,1],[3,3,3,1],[4,1,1,3]]
Output: false
Explanation:
The diagonal "13 31" has different elements.

```

```

Constraints:
n == matrix.length
m == matrix[i].length
1 <= m, n <= 20

```

Round 1 - High - Easy

q.169

```

• 0 <= matrix[i][c] <= 99
Follow-up:
• What if the matrix is stored on disk, and the memory is limited such that you can only load at most one row of the matrix into the memory at once?
• What if the matrix is so large that you can only load a partial row into the memory at once?
Brute Force (Python)

# Brute Force Approach
class Solution:
    def isToeplitzMatrix(self, matrix):
        for i in range(0, len(matrix)-1):
            # Read full row from
            # memory cell
            row = matrix[i], len(matrix[i])
            result = 0
            for c in range(1, len(row)):
                visited = set()
                def flood(r, c):
                    if (r,c) in visited or not
                    return
                    visited.add((r,c))
                    for dr,dc in [(0,1), (0,-1), (-1,0), (1,0)]:
                        flood(r+dr, c+dc)
                result = not(result, len(visited))
            return result

```

```

Community Solutions (Python)
• NumPy
class Solution:
    def isIsomorphic(self, s1: str, s2: str) -> bool:
        len1, len2 = len(s1), len(s2)
        for i in range(len1):
            if s1[i] == s2[i] or (s1[i] != s2[i] and i % 2 == 1):
                return False
        return True
• itertools
class Solution:
    def isIsomorphic(self, s1: str, s2: str) -> bool:
        len1, len2 = len(s1), len(s2)
        for i in range(1, len1):
            if s1[i] != s2[i] and i % 2 == 1:
                return False
        return True
• SimpleList

```

```
# Function to determine if the matrix is Toeplitz
def isToeplitz(matrix):
    # Get the number of rows and columns in the
    # matrix
    rows = len(matrix)
    cols = len(matrix[0])

    # Iterate starting from row 1 and column 1
    # because first row and column are trivially Toeplitz
    for i in range(1, rows):
        for j in range(1, cols):
            # Compare current element with its
            # top-left neighbor
            if matrix[i][j] != matrix[i-1][j-1]:
                return False

    return True

# Example usage
matrix = [[1,2,3,4], [5,1,2,3], [6,5,1,2]]
print(isToeplitz(matrix)) # Expected
# Output: True

v1@cs
```

```
class Solution:
    def isPowerOfTwo(self, n: int) -> bool:
        if n <= 0: return False
        n = abs(n)
        return all(
            n % (1 << i) == 0
            for i in range(1, n)
        )
```

Matrix

#667 Transpose Matrix

54%

LeetCode - SimpleHash - WalCec - LeetCode

Array - Matrix - Simulation

Link CodeSignal

Time Complexity: $O(n \times m)$ in Space Complexity: $O(n)$ for the new matrix holding the transposed values

Problem:
 Given a 2D integer array matrix, return the transpose of matrix.

The transpose of a matrix is the matrix flipped over its main diagonal, switching the matrix's row and column indices.

1	2	3	4
5	6	7	8
9	10	11	12

→

1	5	9
2	6	10
3	7	11
4	8	12

Example 1:

Input: matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]

Output: [[1,5,9],[2,6,10],[3,7,11],[4,8,12]]

Round 1 - High - Easy

g.274

```

Example 2:
Input: matrix = [[1,2,3],[4,5,6]]
Output: [[1,4],[2,5],[3,6]]

Constraints:
m == matrix.length
n == matrix[0].length
1 <= m,n <= 1000
1 <= m * n <= 100
1 <= matrix[i][j] <= 100

Brute Force (Python)

# Brute Force Approach
class Solution:
    def transpose(matrix):
        # Brute Force (O(n^2)) - Flound fill from
        every cell
        n = len(matrix), len(matrix[0])
        result = 0
        for i in range(n):
            for j in range(m):
                visited = set()
                def dfs(matrix, i)

```

```

    if (r.c) is visited or not
    (Success and Success): return
    visited.add(r.c)
    for dr,dc
    in [(0,1),(1,0),(-1,0),(0,-1)]:
        if dr,dc not in
        result: result.add(r.c+dr,c+dc)
    return result

```

Community Solutions (Python)

Naive

```

class Solution:
    def trap(self, height: List[int]) -> int:
        left, right = 0, len(height)-1
        ans = 0
        while left < right:
            for i in range(left+1, right):
                if height[i] < min(height[left], height[right]):
                    ans += height[i]-min(height[left], height[right])
            left, right = max(left, height[left]), min(right, height[right])
        return ans

```

Advanced

```

class Solution:
    def trap(self, height: List[int]) -> int:
        left, right = 0, len(height)-1
        return trap(height, height[left], height[right], left, right)

```

• Simplest

Round 1 - High - Easy p.176

```
# Python solution with clear comments

def transposed(matrix):
    # Number of rows in the original matrix
    num_rows = len(matrix)
    # Number of columns in the original matrix
    num_cols = len(matrix[0])

    # Create a new matrix with dimension swapped:
    num_cols = num_rows
    transposed = [[]] * num_rows

    for i in range(num_rows):
        # Place the element at the swapped
        # index in the transposed matrix
        transposed[i][:] = matrix[i][:]

    return transposed

# Example usage:
matrix = [[2, 3, 3], [4, 5, 6]]
print(transposed(matrix))

+-----
```

```
class Solution:
    def transpose(self, matrix: List[List[int]])
    -> List[List[int]]:
        return list(map(list, zip(*matrix)))
```

Matrix	
#2373 Largest Local Values in a Matrix	Easy
LeetCode - SimpleArray - WalkC - LeetCode	
Array - Matrix	
Using GridTraverse	
Time Complexity: $O(n^2)$, We iterate over $(n-2) \times (n-2)$ positions, and for each position, we look at 8 neighbors.	
Space Complexity: $O(n^2)$, The additional space is required for the output matrix.	
Problem	
You are given an $n \times n$ integer matrix grid.	
Generate an integer matrix result of size $(n-2) \times (n-2)$ such that:	
<code>result[i][j]</code> is equal to the largest value of the 3×3 matrix in grid centered around row $i + 1$ and column $j + 1$.	
In other words, we want to find the largest value in every contiguous 3×3 matrix in grid.	
Return the generated matrix.	
Example 1:	

9	9	8	1
5	6	2	6
8	2	6	4
6	2	2	2

9	9
8	6

Input: grid = [[9,9,8,1],[5,6,2,6],[8,2,6,4],[6,2,2,2]]

Output: [[9,8,1],[6,2,6]]

Explanation: The diagram above shows the original matrix and the generated matrix. The cells that are highlighted in the original matrix correspond to the largest values of a contiguous 3 x 3 matrix in grid.

Example 2:

Round 1 : High - Easy

p.180

#13 Missing Ranges [Easy] - Array & Hashing Key Insights - Use a pointer (or iterate index) to traverse the nums array while keeping track of the previous range, and consider. - Check for missing numbers before the first element, between consecutive elements, and after the last element. - When a gap is found, convert it to a range string that is either a single number if the gap is exactly one number or an interval. - Handle edge cases where the nums array is empty. Space & Time Time Complexity: $O(n)$, where n is the length of the nums array, since we iterate through the entire array. Space Complexity: $O(1)$ additional space (ignoring the output list), as we only use a few extra variables. Solution The solution involves adding a single pass through the input array and calculating the missing ranges by comparing the current number with the previous one. First, initialize a variable "prev" to one less than the lowest bound so that missing numbers from the very	#146 Moving Average from Data Stream [Easy] - Array & Hashing Key Insights - Use a queue (or circular buffer) to keep track of the current window elements. - Maintain a running sum to quickly update the window's total when a new element comes in. - When the window is full, subtract the value of the element that falls out of the window as it is added. - Each call to peek() or next() is executed in constant time, $O(1)$. Space & Time Time Complexity: $O(n)$ per call to next(). Space Complexity: $O(windowSize)$ space. Solution The solution involves a queue data structure to store the elements of the sliding window. A running sum	#160 Invalid Anagram Mappings [Easy] - Array & Hashing Key Insights - Since nums1 is an anagram of nums2, every element in nums1 has a corresponding position in nums2. - A dictionary (or hash table) can be used to map each number in nums1 to its corresponding value in nums2. - When iterating through nums1, we can assign the most available index from the dictionary for that number. Space & Time Time Complexity: $O(n)$ where n is the number of elements in the arrays. Iterate once to build the dictionary and another pass to form the mapping.	#161 Missing Number [Easy] - Array & Hashing Key Insights - Use a hash table or set to store unique numbers from the constant time lookup. - Iterate through each element in arr and check if $arr[i] + 1$ exists in the set. - In this approach, including duplicates. Space & Time Time Complexity: $O(n)$ where n is the number of elements in the array. Iterate once to build the set and another pass to check for missing elements.	#126 Counting Elements [Easy] - Array & Hashing Key Insights - Use a hash table or set to store unique numbers from the constant time lookup. - Iterate through each element in arr and check if $arr[i] + 1$ exists in the set. - In this approach, including duplicates. Space & Time Time Complexity: $O(n)$ where n is the number of elements in the array. Iterate once to build the set and another pass to check for missing elements.	#126 Counting Elements [Easy] - Array & Hashing Key Insights - Use a hash table or set to store unique numbers from the constant time lookup. - Iterate through each element in arr and check if $arr[i] + 1$ exists in the set. - In this approach, including duplicates. Space & Time Time Complexity: $O(n)$ where n is the number of elements in the array. Iterate once to build the set and another pass to check for missing elements.	#134 Count Good Triples [Easy] - Array & Hashing Key Insights - Use a hash table (or dictionary) to store the frequency of each character present in the string. - Check all possible triplets and count only those meeting the criteria. - Absolute difference comparison is the key condition. Space & Time Time Complexity: $O(n^3)$ where n is the length of the string. Space Complexity: $O(1)$ extra space (ignoring input array).	#131 Random Node [Easy] - String Key Insights - Use a hash table (or dictionary) to store the frequency of each character present in the string. - Check all possible triplets and count only those meeting the criteria. - Absolute difference comparison is the key condition. Space & Time Time Complexity: $O(n^3)$ where n is the length of the string. Space Complexity: $O(1)$ extra space (ignoring input array).	#131 Random Node [Easy] - String Key Insights - Use a hash table (or dictionary) to store the frequency of each character present in the string. - Check all possible triplets and count only those meeting the criteria. - Absolute difference comparison is the key condition. Space & Time Time Complexity: $O(n^3)$ where n is the length of the string. Space Complexity: $O(1)$ extra space (ignoring input array).	#131 Random Node [Easy] - String Key Insights - Use a hash table (or dictionary) to store the frequency of each character present in the string. - Check all possible triplets and count only those meeting the criteria. - Absolute difference comparison is the key condition. Space & Time Time Complexity: $O(n^3)$ where n is the length of the string. Space Complexity: $O(1)$ extra space (ignoring input array).
#160 Invalid Anagram Mappings [Easy] - Array & Hashing Key Insights - Use a hash table or set to store unique numbers from the constant time lookup. - Iterate through each element in arr and check if $arr[i] + 1$ exists in the set. - In this approach, including duplicates. Space & Time Time Complexity: $O(n)$ where n is the number of elements in the array. Iterate once to build the set and another pass to check for missing elements.	#160 Invalid Anagram Mappings [Easy] - Array & Hashing Key Insights - Use a hash table or set to store unique numbers from the constant time lookup. - Iterate through each element in arr and check if $arr[i] + 1$ exists in the set. - In this approach, including duplicates. Space & Time Time Complexity: $O(n)$ where n is the number of elements in the array. Iterate once to build the set and another pass to check for missing elements.	#160 Invalid Anagram Mappings [Easy] - Array & Hashing Key Insights - Use a hash table or set to store unique numbers from the constant time lookup. - Iterate through each element in arr and check if $arr[i] + 1$ exists in the set. - In this approach, including duplicates. Space & Time Time Complexity: $O(n)$ where n is the number of elements in the array. Iterate once to build the set and another pass to check for missing elements.	#160 Invalid Anagram Mappings [Easy] - Array & Hashing Key Insights - Use a hash table or set to store unique numbers from the constant time lookup. - Iterate through each element in arr and check if $arr[i] + 1$ exists in the set. - In this approach, including duplicates. Space & Time Time Complexity: $O(n)$ where n is the number of elements in the array. Iterate once to build the set and another pass to check for missing elements.	#160 Invalid Anagram Mappings [Easy] - Array & Hashing Key Insights - Use a hash table or set to store unique numbers from the constant time lookup. - Iterate through each element in arr and check if $arr[i] + 1$ exists in the set. - In this approach, including duplicates. Space & Time Time Complexity: $O(n)$ where n is the number of elements in the array. Iterate once to build the set and another pass to check for missing elements.	#160 Invalid Anagram Mappings [Easy] - Array & Hashing Key Insights - Use a hash table or set to store unique numbers from the constant time lookup. - Iterate through each element in arr and check if $arr[i] + 1$ exists in the set. - In this approach, including duplicates. Space & Time Time Complexity: $O(n)$ where n is the number of elements in the array. Iterate once to build the set and another pass to check for missing elements.	#160 Invalid Anagram Mappings [Easy] - Array & Hashing Key Insights - Use a hash table or set to store unique numbers from the constant time lookup. - Iterate through each element in arr and check if $arr[i] + 1$ exists in the set. - In this approach, including duplicates. Space & Time Time Complexity: $O(n)$ where n is the number of elements in the array. Iterate once to build the set and another pass to check for missing elements.	#160 Invalid Anagram Mappings [Easy] - Array & Hashing Key Insights - Use a hash table or set to store unique numbers from the constant time lookup. - Iterate through each element in arr and check if $arr[i] + 1$ exists in the set. - In this approach, including duplicates. Space & Time Time Complexity: $O(n)$ where n is the number of elements in the array. Iterate once to build the set and another pass to check for missing elements.	#160 Invalid Anagram Mappings [Easy] - Array & Hashing Key Insights - Use a hash table or set to store unique numbers from the constant time lookup. - Iterate through each element in arr and check if $arr[i] + 1$ exists in the set. - In this approach, including duplicates. Space & Time Time Complexity: $O(n)$ where n is the number of elements in the array. Iterate once to build the set and another pass to check for missing elements.	#160 Invalid Anagram Mappings [Easy] - Array & Hashing Key Insights - Use a hash table or set to store unique numbers from the constant time lookup. - Iterate through each element in arr and check if $arr[i] + 1$ exists in the set. - In this approach, including duplicates. Space & Time Time Complexity: $O(n)$

Sheet 11/87 · LandMastery Study-Order · 36-up Landscape · 6×6 · Round 2 · High · Medium · p.294

# Brute Force Approach class Solution: def minDifference(self, heights: List[int]) -> int: # Brute Force O(N^2) - try every pair of lines res = 0 for i in range(len(heights)): for j in range(i + 1, len(heights)): water = min(height[i], height[j]) res = max(res, water) return res Complexity Solutions (Python) class Solution: def minDifference(self, heights: List[int]) -> int: # Sort heights heights.sort() l = 0 r = len(heights) - 1 while l < r: height = min(height[l], height[r]) ans = max(ans, height) l += 1 r -= 1 if height[l] < height[r]: l += 1 else: r -= 1 return ans	l = 1 else: r -= 1 return ans class Solution: def minDifference(self, heights: List[int]) -> int: # Sort heights heights.sort() l = 0 r = len(heights) - 1 while l < r: height = min(height[l], height[r]) ans = max(ans, height) l += 1 r -= 1 if height[l] < height[r]: l += 1 else: r -= 1 return ans	# Function to calculate the maximum water container def minDifference(self, heights: List[int]) -> int: # Sort heights heights.sort() l = 0 r = len(heights) - 1 while l < r: height = min(height[l], height[r]) ans = max(ans, height) l += 1 r -= 1 if height[l] < height[r]: l += 1 else: r -= 1 return ans	Input: heights = [1,6,2,5,4,3,7] Output: 6 class Solution: def minDifference(self, heights: List[int]) -> int: # Sort heights heights.sort() l = 0 r = len(heights) - 1 while l < r: height = min(height[l], height[r]) ans = max(ans, height) l += 1 r -= 1 if height[l] < height[r]: l += 1 else: r -= 1 return ans	Two Pointers #153 Sum class Solution: def minDifference(self, heights: List[int]) -> int: # Sort heights heights.sort() l = 0 r = len(heights) - 1 while l < r: height = min(height[l], height[r]) ans = max(ans, height) l += 1 r -= 1 if height[l] < height[r]: l += 1 else: r -= 1 return ans	Input: heights = [1,6,2,5,4,3,7] Output: 6 class Solution: def minDifference(self, heights: List[int]) -> int: # Sort heights heights.sort() l = 0 r = len(heights) - 1 while l < r: height = min(height[l], height[r]) ans = max(ans, height) l += 1 r -= 1 if height[l] < height[r]: l += 1 else: r -= 1 return ans	Input: heights = [1,6,2,5,4,3,7] Output: 6 class Solution: def minDifference(self, heights: List[int]) -> int: # Sort heights heights.sort() l = 0 r = len(heights) - 1 while l < r: height = min(height[l], height[r]) ans = max(ans, height) l += 1 r -= 1 if height[l] < height[r]: l += 1 else: r -= 1 return ans
res.add(nums[i], nums[j]) return list(it) for i in res Complexity Solutions (Python) class Solution: def minDifference(self, heights: List[int]) -> int: # Sort heights heights.sort() l = 0 r = len(heights) - 1 while l < r: height = min(height[l], height[r]) ans = max(ans, height) l += 1 r -= 1 if height[l] < height[r]: l += 1 else: r -= 1 return ans	l = 1 else: r -= 1 return ans class Solution: def minDifference(self, heights: List[int]) -> int: # Sort heights heights.sort() l = 0 r = len(heights) - 1 while l < r: height = min(height[l], height[r]) ans = max(ans, height) l += 1 r -= 1 if height[l] < height[r]: l += 1 else: r -= 1 return ans	l = 1 else: r -= 1 return ans class Solution: def minDifference(self, heights: List[int]) -> int: # Sort heights heights.sort() l = 0 r = len(heights) - 1 while l < r: height = min(height[l], height[r]) ans = max(ans, height) l += 1 r -= 1 if height[l] < height[r]: l += 1 else: r -= 1 return ans	l = 1 else: r -= 1 return ans class Solution: def minDifference(self, heights: List[int]) -> int: # Sort heights heights.sort() l = 0 r = len(heights) - 1 while l < r: height = min(height[l], height[r]) ans = max(ans, height) l += 1 r -= 1 if height[l] < height[r]: l += 1 else: r -= 1 return ans	l = 1 else: r -= 1 return ans class Solution: def minDifference(self, heights: List[int]) -> int: # Sort heights heights.sort() l = 0 r = len(heights) - 1 while l < r: height = min(height[l], height[r]) ans = max(ans, height) l += 1 r -= 1 if height[l] < height[r]: l += 1 else: r -= 1 return ans	l = 1 else: r -= 1 return ans class Solution: def minDifference(self, heights: List[int]) -> int: # Sort heights heights.sort() l = 0 r = len(heights) - 1 while l < r: height = min(height[l], height[r]) ans = max(ans, height) l += 1 r -= 1 if height[l] < height[r]: l += 1 else: r -= 1 return ans	l = 1 else: r -= 1 return ans class Solution: def minDifference(self, heights: List[int]) -> int: # Sort heights heights.sort() l = 0 r = len(heights) - 1 while l < r: height = min(height[l], height[r]) ans = max(ans, height) l += 1 r -= 1 if height[l] < height[r]: l += 1 else: r -= 1 return ans
while j < k and nums[j] == nums[j + 1]: nums[j] = nums[j + 1] nums[j + 1] = nums[j] return ans Complexity Solutions (Python) class Solution: def minDifference(self, heights: List[int]) -> int: # Sort heights heights.sort() l = 0 r = len(heights) - 1 while l < r: height = min(height[l], height[r]) ans = max(ans, height) l += 1 r -= 1 if height[l] < height[r]: l += 1 else: r -= 1 return ans	l = 1 else: r -= 1 return ans class Solution: def minDifference(self, heights: List[int]) -> int: # Sort heights heights.sort() l = 0 r = len(heights) - 1 while l < r: height = min(height[l], height[r])					

Sheet 20/87 · Lesson Mastery Study-Order · 36-up Landscape · 6x6 · Round 2 · High · Medium · 8-718

# Even length: split into two equal halves <pre> n = len(s) mid = left + length // 2 # Compare left half (left, mid-1) with right half (mid, right) comp = reader.compareDict(left, mid - 1, mid, right) if comp == 1: # Large integer is in the left return left else: # Large integer is in the right return right </pre> when: Values of equal size, ignoring middle element. half = length // 2 Compare left half (left, left+half-1) with right half (right-half, right) comp = reader.compareDict(left, left + half - 1, right - half + 1, right) If comp == 1: Large integer in left half right = left + half - 1 elif comp == -1: Large integer in right half	also: left = right - half + 1 if halves are equal, so the middle element is the large one return left if self: # This is ArrayReader's API interface. # You should not implement it, or speculate about its implementation # class ArrayReader(object): # Compare the sum of arr[left..right] with the sum of arr[left..right] # return 1 if sum(arr[left..right]) > sum(arr[left..right]) # return 0 if sum(arr[left..right]) == sum(arr[left..right]) # return -1 if sum(arr[left..right]) < sum(arr[left..right]) # def compareDict(self, left, left + half - 1, right - half + 1, right): if comp == 1: Large integer in left half right = left + half - 1 elif comp == -1: Large integer in right half	# Return the length of the array # def length(self) -> int: return len(self.array) class Solution: def getKth(self, k): if k < 1: return -1 if k > len(self.array): return -1 if k == 1: return self.array[0] if k == len(self.array): return self.array[-1] if k < len(self.array) / 2: return self.getKth(k) if k > len(self.array) / 2: return self.getKth(k - len(self.array) / 2 + 1) if k == len(self.array) / 2 + 1: return self.array[k - 1]	Matrix Round 2 - High - Medium - 14 questions LeetCode - SimplifyTest - WAKCC - LeetCode Array - Math - Matrix Time Complexity: O(N) - Since the board size is fixed at 100, Space Complexity: O(1) - The extra space used by the sets is bounded by the fixed board size. Problem: Given a 100 x 100 matrix, find the maximum value of the sum of the elements in the matrix, such that the sum of the elements in the matrix is equal to the sum of the elements in the matrix. Example 1: Input: [[1, 2, 3, 4, 5, 6, 7, 8, 9, 10], [11, 12, 13, 14, 15, 16, 17, 18, 19, 20], [21, 22, 23, 24, 25, 26, 27, 28, 29, 30], [31, 32, 33, 34, 35, 36, 37, 38, 39, 40], [41, 42, 43, 44, 45, 46, 47, 48, 49, 50], [51, 52, 53, 54, 55, 56, 57, 58, 59, 60], [61, 62, 63, 64, 65, 66, 67, 68, 69, 70], [71, 72, 73, 74, 75, 76, 77, 78, 79, 80], [81, 82, 83, 84, 85, 86, 87, 88, 89, 90], [91, 92, 93, 94, 95, 96, 97, 98, 99, 100]] Output: 100	#36 Valid Sudoku LeetCode - SimplifyTest - WAKCC - LeetCode Array - Math Table - Matrix Time Complexity: O(1) - Since the board size is fixed at 100, Space Complexity: O(1) - The extra space used by the sets is bounded by the fixed board size. Problem: Given a 9 x 9 Sudoku board, return true if it is a valid Sudoku, otherwise return false. A valid Sudoku is one that has the following properties: 1. Each row must contain the digits 1-9 without repetition. 2. Each column must contain the digits 1-9 without repetition. 3. Each of the nine 3 x 3 sub-boxes of the grid must contain the digits 1-9 without repetition. Note: a. A Sudoku board (partially filled) could be valid but is not necessarily solvable. b. Only the filled cells need to be validated according to the mentioned rules. Example 1: Input: board = <pre> [["5","3",".",".","7",".",".",".","."], [["2",".",".","6",".",".",".","9","."], [["9","8",".",".","1",".","5","6","."], [["4",".","8",".","3",".",".","1","."], [["7",".","2",".","9",".",".","4","."], [["6",".",".","8","6","3",".",".","."], [[".","4","1","9","5",".",".","."], [["8","7",".",".","2",".","9",".","."], [["3",".","9",".","4","5",".",".","."]] </pre> Output: true	Round 2 - High - Medium - 14 questions LeetCode - SimplifyTest - WAKCC - LeetCode Array - Math Table - Matrix Time Complexity: O(1) - Since the board size is fixed at 100, Space Complexity: O(1) - The extra space used by the sets is bounded by the fixed board size. Problem: Given a 9 x 9 Sudoku board, return true if it is a valid Sudoku, otherwise return false. A valid Sudoku is one that has the following properties: 1. Each row must contain the digits 1-9 without repetition. 2. Each column must contain the digits 1-9 without repetition. 3. Each of the nine 3 x 3 sub-boxes of the grid must contain the digits 1-9 without repetition. Note: a. A Sudoku board (partially filled) could be valid but is not necessarily solvable. b. Only the filled cells need to be validated according to the mentioned rules. Example 1: Input: board = <pre> [["5","3",".",".","7",".",".",".","."], [["2",".",".","6",".",".",".","9","."], [["9","8",".",".","1",".","5","6","."], [["4",".","8",".","3",".",".","1","."], [["7",".","2",".","9",".",".","4","."], [["6",".",".","8","6","3",".",".","."], [[".","4","1","9","5",".",".","."], [["8","7",".",".","2",".","9",".","."], [["3",".","9",".","4","5",".",".","."]] </pre> Output: true
# Even length: split into two equal halves <pre> n = len(s) mid = left + length // 2 # Compare left half (left, mid-1) with right half (mid, right) comp = reader.compareDict(left, mid - 1, mid, right) if comp == 1: # Large integer is in the left return left else: # Large integer is in the right return right </pre> when: Values of equal size, ignoring middle element. half = length // 2 Compare left half (left, left+half-1) with right half (right-half, right) comp = reader.compareDict(left, left + half - 1, right - half + 1, right) If comp == 1: Large integer in left half right = left + half - 1 elif comp == -1: Large integer in right half	also: left = right - half + 1 if halves are equal, so the middle element is the large one return left if self: # This is ArrayReader's API interface. # You should not implement it, or speculate about its implementation # class ArrayReader(object): # Compare the sum of arr[left..right] with the sum of arr[left..right] # return 1 if sum(arr[left..right]) > sum(arr[left..right]) # return 0 if sum(arr[left..right]) == sum(arr[left..right]) # return -1 if sum(arr[left..right]) < sum(arr[left..right]) # def compareDict(self, left, left + half - 1, right - half + 1, right): if comp == 1: Large integer in left half right = left + half - 1 elif comp == -1: Large integer in right half	# Return the length of the array # def length(self) -> int: return len(self.array) class Solution: def getKth(self, k): if k < 1: return -1 if k > len(self.array): return -1 if k == 1: return self.array[0] if k == len(self.array): return self.array[-1] if k < len(self.array) / 2: return self.getKth(k) if k > len(self.array) / 2: return self.getKth(k - len(self.array) / 2 + 1) if k == len(self.array) / 2 + 1: return self.array[k - 1]	Matrix Round 2 - High - Medium - 14 questions LeetCode - SimplifyTest - WAKCC - LeetCode Array - Math - Matrix Time Complexity: O(N) - Since the board size is fixed at 100, Space Complexity: O(1) - The extra space used by the sets is bounded by the fixed board size. Problem: Given a 100 x 100 matrix, find the maximum value of the sum of the elements in the matrix, such that the sum of the elements in the matrix is equal to the sum of the elements in the matrix. Example 1: Input: [[1, 2, 3, 4, 5, 6, 7, 8, 9, 10], [11, 12, 13, 14, 15, 16, 17, 18, 19, 20], [21, 22, 23, 24, 25, 26, 27, 28, 29, 30], [31, 32, 33, 34, 35, 36, 37, 38, 39, 40], [41, 42, 43, 44, 45, 46, 47, 48, 49, 50], [51, 52, 53, 54, 55, 56, 57, 58, 59, 60], [61, 62, 63, 64, 65, 66, 67, 68, 69, 70], [71, 72, 73, 74, 75, 76, 77, 78, 79, 80], [81, 82, 83, 84, 85, 86, 87, 88, 89, 90], [91, 92, 93, 94, 95, 96, 97, 98, 99, 100]] Output: 100	#36 Valid Sudoku LeetCode - SimplifyTest - WAKCC - LeetCode Array - Math Table - Matrix Time Complexity: O(1) - Since the board size is fixed at 100, Space Complexity: O(1) - The extra space used by the sets is bounded by the fixed board size. Problem: Given a 9 x 9 Sudoku board, return true if it is a valid Sudoku, otherwise return false. A valid Sudoku is one that has the following properties: 1. Each row must contain the digits 1-9 without repetition.	

```

    dir(x[i][j])
    i, j = x, y
    return ans

```

Matrix

640

160

145 Minimum Path Sum

Hard

Intuition: Simplex!n, Walk!n

Array: Dynamic Programming, Matrix

Little Denny Zhang

Time Complexity ($O(n \times m)$), where m and n are the number of rows and columns. Space Complexity ($O(n \times m)$) for the DP table (this can be optimized to $O(m)$ with in-place computations).

Problem

Given a $m \times n$ grid filled with non-negative numbers, find a path from top left to bottom right, which minimizes the sum of all numbers along its path.

Note: You can only move either down or right at any point in time.

Example 1:

Found 2 · High · Medium

p.758

1	3	1
1	5	1
1	2	1

Input: grid = [[1,3,1],[5,5,1],[1,2,1]]

Output: 7

Explanation: Because this path 1 -> 3 -> 1 -> 1 -> 1 utilizes the use.

Example 2:

Input: grid = [[1,1,1],[1,5,1]]

Output: 12

Constraints:

- m == grid.length
- n == grid[0].length

Round 2 - High - Medium
g.759

```

• 1 <= m, n <= 200
• 0 <= grid[i][j] <= 200

Brute Force (Python)

# Brute Force Approach
class Solution:
    def canReach(self, grid):
        # Brute Force Size O(N^2) - flood fill from every cell
        n, m = len(grid), len(grid[0])
        result = 0
        for i in range(n):
            for j in range(m):
                visited = set()
                if floodFill(i, j, visited):
                    result += 1
        return result

    def floodFill(self, i, j, visited):
        # (i, j) is visited or not
        if (i, j) in visited:
            return False
        # (i, j) is not visited
        visited.add((i, j))
        # for dr, dc in
        # [(0,1), (0,-1), (1,0), (-1,0)]:
        #     if floodFill(self, i+dr, j+dc, visited):
        #         result += 1
        result = 1
        return result

```

Community Solutions (Python)

- WalkCC

Round 2 : High Medium

q_360

```
class Solution:
    def minGridSum(self, grid: List[List[int]]) -> int:
        n = len(grid)
        m = len(grid[0])

        for i in range(1):
            for j in range(1):
                if i < m and j < n:
                    grid[i][j] = min(grid[i - 1][j],
                                      grid[i][j - 1])
                    elif i < n:
                        grid[i][m] = grid[i - 1][m]
                    elif j < m:
                        grid[n][j] = grid[n][j - 1]
        return grid[n - 1][m - 1]
```

```

class Solution:
    def minPathSum(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        f = [[0] * n for _ in range(m)]
        f[0][0] = grid[0][0]
        for i in range(1, m):
            f[i][0] = f[i-1][0] + grid[i][0]
        for j in range(1, n):
            f[0][j] = f[0][j-1] + grid[0][j]
        for i in range(1, m):
            for j in range(1, n):
                f[i][j] = min(f[i-1][j], f[i][j-1]) + grid[i][j]
        return f[-1][-1]

```

Complexity:

- # Python solution using dynamic programming
- # Runtime complexity:
 - Number of rows and columns is the grid
 - rows = len(grid)
 - cols = len(grid[0])
- # Initialize the DP table directly in grid to save space
- # Fill the first row by accumulating the sum from the left
- for j in range(1, cols):
 grid[0][j] += grid[0][j-1]

```

// Fill the first column by accumulating the sum
dim0 =
    for i in range(1, row0):
        grid[i][0] = grid[i-1][0]

// Complete the minimum path sum for the rest of
// the cells
for i in range(1, row0):
    for j in range(1, col0):
        // The cell value is added to the
        // minimum of the top or left cell
        grid[i][j] = min(grid[i-1][j],
                           grid[i][j-1]) + grid[i][j]

// The bottom-right cell contains the answer
return grid[row0-1][col0-1]

// Example usage
example_grid = [[1,2,3], [1,5,1], [4,2,1]]
print(minPathSum(example_grid)) // Output should be 7

```

```

class Solution:
    def minPathSum(self, grid: List[List[int]]) -> int:
        n, m = len(grid), len(grid[0])
        f = [0] * m
        for i in range(n):
            f[0] = grid[i][0]
            for j in range(1, m):
                f[j] = min(f[j-1] + grid[i][j], f[j] + grid[i][j])
            for j in range(m-1, 0, -1):
                f[j] = f[j-1] + grid[i][j]
        return f[-1]

```

Matrix

#73 Set Matrix Zeroes

NEED

LeetCode - SimplifyAnk - WaackCC - LeetCode
 Matrix - Matrix
 Link: Grid 383
 Time Complexity: $O(m \times n)$ Space Complexity: $O(1)$
 (in-place, using only constant extra space)
Problem:
 Given an $m \times n$ integer matrix, if an element is 0, set its entire row and column to 0's.
 You must do it in place.

Example 1:

1	2	3
4	5	6
7	8	9

→

7	4	1
8	5	2
9	6	3

Solution: matrix = [[1,1,1],[1,0,1],[1,1,1]]
 indices = [(1,0),(0,1),(2,1)]

Example 2:

Swap: swap1 ← [1, 3, 2, 1], [3, 4, 5, 2], [2, 1, 5, 1]
 Output: [[A, R, A], [A, S, A], [A, T, A]]

Conclusion:

```

n = m = matrix.length
m = m * 0.5 // length
m = m < n ? m : 200
-211 <= matrix[i][j] <= 211 - 1
  
```

Follow up:

- A straightforward solution using O(m) space is probably a bad idea.
- A simple improvement uses O(n) = rows, but still not the best solution.
- Could you devise a constant space solution?

Brute Force (Python)

```
# Brute Force Approach
class Solution:
    def arithmeticSlices(self, nums):
        # Brute Force O(n^3) time - record zero
        # positions, then sum up results
        ans = 0
        for i, v in enumerate(nums):
            ans += (i-1)*(i-2)
            for j, k in enumerate(nums[i+1:]):
                for m, n in enumerate(nums[i+2+j]):
                    if nums[i]-nums[j]==nums[j]-nums[k]:
                        ans+=1
        return ans//2

# Community Solution (Python)

class Solution:
    def arithmeticSlices(self, nums):
        n = len(nums)
        if n < 3:
            return 0
        ans = 0
        for i in range(1, n-2):
            if nums[i]-nums[i-1] == nums[i+1]-nums[i]:
                ans += 1
        return ans
```

```

class Solution:
    def setZeroes(self, matrix: List[List[int]]) -> None:
        """
        Do not return anything, modify matrix in-place instead.
        """
        n = len(matrix)
        m = len(matrix[0])
        shouldSetIfRowZero = 0
        shouldSetIfColZero = 0
        for i in range(n):
            if matrix[i][0] == 0:
                shouldSetIfColZero = 1
            for j in range(1, m):
                if matrix[i][j] == 0:
                    matrix[i][0] = 0
                    matrix[i][j] = 0

        for i in range(1, n):
            if matrix[i][0] == 0:
                for j in range(1, m):
                    matrix[i][j] = 0

        if shouldSetIfRowZero:
            for i in range(n):
                matrix[i][0] = 0

        if shouldSetIfColZero:
            for j in range(1, m):
                matrix[0][j] = 0

```

```
# Fill B for the first row if needed.
if shouldFillRow(rownum):
    matrix[i][j] = 0

# Fill B for the first column if needed.
if shouldFillCol(colnum):
    for row in matrix:
        row[i] = 0

class Solution:
    def setZeroes(self, matrix: List[List[int]])
    -> None:
        m = len(matrix), len(matrix[0])
        row = False, col = False
        for i in range(m):
            if matrix[i][0] == 0:
                row = True
        for j in range(n):
            if matrix[0][j] == 0:
                col = True
        for i in range(m):
            for j in range(n):
                if row[i] or col[j]:
                    matrix[i][j] = 0
```

• Simplest

```

# Python solution for the Set Matrix Zeros problem

def setZeroes(matrix):
    # Step 1: Determine the dimensions of matrix
    m = len(matrix)
    n = len(matrix[0])

    # Determine if first row or first column should
    # be zeroed
    first_row_zero = any(matrix[i][0] == 0 for i in
        range(m))
    first_col_zero = any(matrix[i][0] == 0 for i in
        range(n))

    # Step 2: Set the first row and column as markers
    for i in range(m):
        if matrix[i][0] == 0:
            matrix[i][0] = -1
    for j in range(n):
        if matrix[0][j] == 0:
            matrix[0][j] = -1

    # Step 3: Zero out cells based on the markers
    for i in range(m):
        for j in range(n):
            if matrix[i][0] == -1 or matrix[0][j] == -1:
                matrix[i][j] = 0

    # Step 4: Restore the first row and column
    if first_row_zero:
        for i in range(m):
            matrix[i][0] = 0
    if first_col_zero:
        for j in range(n):
            matrix[0][j] = 0

```

```

for j in range(1, n)
  if matrix[i][0] == 0 or matrix[i][j] ==
0:
    matrix[i][j] = 0

# Step 4: Zero out the first row if needed
if first_row_zero:
    for j in range(n):
        matrix[i][j] = 0

# Step 5: Zero out the first column if needed
if first_col_zero:
    for i in range(n):
        matrix[i][0] = 0

# Example usage:
# matrix = [[1,1,1],[1,0,1],[1,1,1]]
# setZeroes(matrix)
# print(matrix)

+LEde

```

[illegible]

Matrix	
#74 Search a 2D Matrix	NEED
LeetCode - Simple/Leet - Multi/CC - LeetCode	
Array - Binary Search - Matrix	
LeetCode 74	
Time Complexity: $O(\log(m * n))$ Space Complexity: $O(1)$	
Problem	
You are given an $m \times n$ integer matrix with the following properties:	
- Each row is sorted in non-decreasing order. - The first integer of each row is greater than the last integer of the previous row.	
Given an integer target, return true if target is in matrix or false otherwise.	
You must write a solution in $O(\log(m * n))$ time complexity.	
Example 1:	

1	3	5	7
10	11	16	20
23	30	34	60

Input: matrix =
 [[1, 3, 5, 7], [10, 11, 16, 20], [23, 30, 34, 60]], target = 3
 Output: false

Example 2:

[illegible]

```

class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        if not matrix:
            return False

        m = len(matrix)
        n = len(matrix[0])
        l = 0
        r = n - 1

        while l <= r:
            mid = (l + r) // 2
            i = mid // n
            j = mid % n

            if matrix[i][j] == target:
                return True
            elif matrix[i][j] < target:
                l = mid + 1
            else:
                r = mid - 1

        return False

```

• [LeetCode](#)

```
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        n = len(matrix)
        left = 0
        right = 0
        m = n - 1
        while left < right:
            mid = (left + right) // 1
            if matrix[mid][0] == target:
                return True
            elif matrix[mid][0] > target:
                right = mid
            else:
                left = mid + 1
        return matrix[left][0] == target

class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        # Get number of rows and columns
        if not matrix or not matrix[0]:
            return False
        row, cols = len(matrix), len(matrix[0])
        left = 0
        right = 0
        row = cols - 1
        while left < right:
            # Perform binary search over the virtual flattened array
            while left < right:
```

```

# Find the middle index
mid = left + right // 2

# Map the middle index to row and column in
# the matrix
row = mid // cols
col = mid % cols
mid_value = matrix[row][col]

# Check if the mid_value equals the target
if mid_value == target:
    return True

elif mid_value < target:
    subarray = left + 1 # Move to the right
    else:
        right = mid - 1 # Move to the left

# Target not found
return False

# Example usage:
matrix = [[1, 5, 7], [10, 11, 16], [22, 28, 34], [40]]
target = 9
print(searchMatrix(matrix, target))

```

```

Round 2 - High-Medium
p.773

class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])
        left, right = 0, m - 1
        while left < right:
            mid = (left + right) // 2
            r, y = dim(matrix, mid) # # rows: divide
            if matrix[mid][y] == target:
                return True
            elif matrix[mid][y] > target:
                right = mid - 1
            else:
                left = mid + 1
        return False

# testcases

class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])

```

```

bound2 - High - Medium
p.754

left, right = 0, n - 1
while left < right:
    mid = (left + right) // 2
    y = diminished(x)
    if matrix[mid] == target:
        right = mid
    else:
        left = mid + 1
return matrix[left // n][left % n] ==
target

```

Ranked 2 - High - Medium

p7571

Matrix

#221 Maximal Square

Hard

LeetCode - SimpleEasy - Medium - Hard - Expert - Interview

Array - Dynamic Programming - Math - String - Sorting - Trie

Like: 3693

Time Complexity: $O(n^2)$, where n is the number of rows and n is the number of columns. Space Complexity: $O(1)$ for the DP table (this can be optimized to $O(1)$ with space reduction techniques).

Problem

Given an $m \times n$ binary matrix filled with 0's and 1's, find the largest square containing only 1's and return its area.

Example 1:

Round 2 - High - Medium p.776

1	0	1	0	0
1	0	1	1	1
1	1	1	1	1
1	0	0	1	0

Input: matrix = [[1,0,1,0,0],[1,0,1,1,1],[1,1,1,1,1],[1,0,0,1,0]]

Output: 4

Example 2:

Round 2 - High - Medium p.777

0	1
1	0

Input: matrix = [[1,"1"],["0","0"]]
Output: 1

Example 3:

Input: matrix = [[1,"1"]]
Output: 0

Constraints:

- 0 <= matrix.length
- 0 <= matrix[0].length
- 1 <= m[i].length <= 10
- matrix[i][j] is '0' or '1'.

```

Brute Force (Python)

# Brute Force Approach
class Solution:
    def isAnagram(self, s1: str, s2: str) -> bool:
        # Brute Force Approach - O(n^2) - flawed find from
        every cell
        n, m = len(s1), len(s2)
        result = 0
        for i in range(n):
            for j in range(m):
                if s1[i] == s2[j]:
                    result += 1
                    break
            if (i+1) % 10 == 0 or i == n-1:
                print(i, result)
                result = 0
        return result == m

# Brute Force Approach - O(n^2) - flawed find from
every cell
n, m = len(s1), len(s2)
result = 0
for i in range(n):
    for j in range(m):
        if s1[i] == s2[j]:
            result += 1
            break
    if (i+1) % 10 == 0 or i == n-1:
        print(i, result)
        result = 0
return result == m

```

```

class Solution:
    def matrixSquare(self, matrix: List[List[int]]) -> List:
        n = len(matrix)
        s = len(matrix[0])
        dp = [[0] * s for _ in range(n)]
        dp[0][0] = 0

        for i in range(1, n):
            for j in range(1, s):
                if i == 0 or j == 0 or matrix[i][j] == 'X':
                    dp[i][j] = 0
                else:
                    dp[i][j] = min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1

        return dp[-1][-1]

```

```

class Solution:
    def minCostSquare(self, matrix: List[List[int]]) -> int:
        m, n = len(matrix), len(matrix[0])
        dp = [[0] * n for i in range(m)]
        m -= 1
        for i in range(m):
            for j in range(n):
                if matrix[i][j] == 7:
                    dp[i + 1][j + 1] = min(dp[i][j + 1], dp[i][j + 1] + 1)
                    m = max(m, dp[i + 1][j + 1])
        return m + m

```

Summary

- Python implementation of the Minimal Square problem

```

def minCostSquare(matrix):
    # Check if matrix is empty
    if not matrix or not matrix[0]:
        return 0
    rows = len(matrix)
    cols = len(matrix[0])
    # DP table with extra row and col for ease of
    # boundary conditions
    dp = [[0] * (cols + 1) for i in range(rows + 1)]

```

```

max_xcld = 0
# Traverse through each cell in the matrix
for i in range(1, row - 1):
    for j in range(1, cols - 1):
        # If of the current cell is '1'
        if matrix[i][j] - 1 != '1':
            # Compute the minimum value among
            # the top, left, and right neighbors and add 1
            dp[i][j] = min(dp[i][j-1], dp[i][j+1],
                           dp[i-1][j], dp[i+1][j]) + 1
            # The area of the square is the square of the
            # side length
            return max_xcld = max_xcld, i, j
# Example usage:
matrix = [
    ["1", "0", "1", "0", "0", "1"],
    ["0", "1", "1", "0", "1", "0"],
    ["1", "1", "0", "1", "1", "0"],
    ["1", "0", "0", "0", "1", "1"],
    ["0", "1", "1", "1", "0", "0"],
    ["1", "0", "0", "0", "0", "1"]
]
print(minimalSquare(matrix))

# O(n^2)

```

```

//
// eq - 10x10 square with full of 1s
// this square more 1 line add
// this square more 1 line right
// so there needs an extra single 2 at bottom
// right, so make it a larger full square
//
class Solution {
public:
    def mainSquare(int n):
        list1=list(range(1))
        n = len(matrix1), len(matrix2[0])
        m = 0
        for i in range(1) for j in range(1)
            m = 1
            for j in range(1):
                if matrix1[i][j] == '1':
                    def i = 1, j = 1
                    while(i <= n, j <= m):
                        m = matrix1[i][j] == '1'
                        return m

```

#311 Sparse Matrix Multiplication

LeetCode - Simplify - Walk-CC - LeetCode
 Array-Dims - Matrix - Linear-Program 56
 Time Complexity: $O(n^3)$ as in the worst-case, but significantly lower when many elements are zero. Space Complexity: $O(1)$ as the resulting matrix.

Problem:
 Given two sparse matrices mat1 of size n x k and mat2 of size k x m, return the result of mat1 x mat2. You may assume that multiplication is always possible.

Example 1:

1	0	0	$\times$	7	0	0	$=$	7	0	0
0	3	0		0	0	1		0	0	1
-1	0	3		0	0	1		-1	0	3

Input: mat1 = [[1,0,0],[0,3,0],[-1,0,3]], mat2 = [[7,0,0],[0,0,1],[0,0,1]]
Output: [[7,0,0],[0,0,3],[-1,0,3]]

Example 2:

```

Round 2: High Medium
a:784

Input: mat1 = [[0]], mat2 = [[0]]
Output: [[0]]

const mat1 =
    m = mat1.length
    n = mat1[0].length
    mat2 = mat2.length
    l = mat2[0].length
    i = 0; m <= l; i++
    i = 0; n <= l; i++
    for (let i = 0; i < m; i++)
        for (let j = 0; j < n; j++)
            mat1[i][j] = 0
}

# Brute Force Approach
def bruteForce(mat1, mat2):
    m1 = len(mat1)
    n1 = len(mat1[0])
    m2 = len(mat2)
    n2 = len(mat2[0])
    if m1 < n2 or m2 < n1:
        return 0
    result = 0
    for i in range(m1):
        for j in range(n2):
            val = mat1[i][j] * mat2[j][0]
            result += val
    return result

```

```

2 and 2 High Medium p.785

if (x, y) is visited or not
(Bounds and Bounds) return
    visited.add((x,y))
    for dx in dxs:
        for dy in dys:
            (nx, ny) = (x+dx, y+dy)
            if (nx, ny) not in visited:
                result += max(result, listVisited(nx, ny))
    return result

Community Solutions (Python)

2 Python
class Solution:
    def multiply(self, mat1: List[List[int]], mat2: List[List[int]]) -> List[List[int]]:
        n = len(mat1)
        m = len(mat2)
        l = len(mat1[0])
        ans = [[0] * l for _ in range(n)]
        for i in range(n):
            for j in range(l):
                for k in range(m):
                    ans[i][j] = ans[i][j] + mat1[i][k] * mat2[k][j]
        return ans

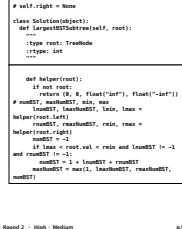
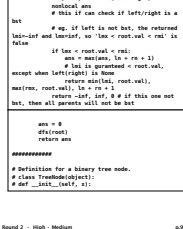
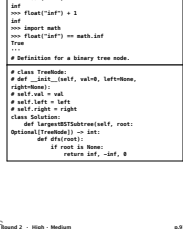
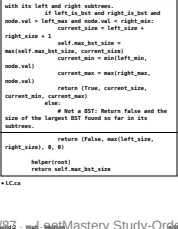
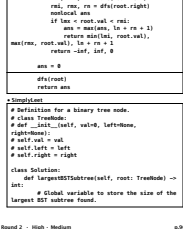
• LeetCode

Source: 7 and 2 High Medium p.783
```

```
class Solution:
    def multiply(self, mat1: List[List[int]],
                 mat2: List[List[int]]) -> List[List[int]]:
        m1, n1 = len(mat1), len(mat1[0])
        m2, n2 = len(mat2), len(mat2[0])
        ans = [[0] * n2 for _ in range(m1)]
        for i in range(m1):
            for k in range(len(mat2)):
                ans[i][k] = mat1[i][k] * mat2[k][0]
        return ans

#Complexity
# Time complexity: O(m1 * n1 * n2)
# For dimensions of mat1 and mat2
m1, n1 = len(mat1), len(mat1[0])
m2, n2 = len(mat2), len(mat2[0])
# Initialize result matrix with zeros
result = [[0] * n2 for _ in range(m1)]
# Iterate through each row in mat1
for i in range(m1):
    # For each element in row i of mat1
```


Sheet 25/87 · Level 2 · High · Medium Mastery Study-Order · 36-up Landscape · 6x6 · Sound 2 · High · Medium



Sheet 28/87 · Lesson Mastery Study-Order · 36-up Landscape · 6×6 · Round 2 · High · Medium · a1006

Sheet 29/87 · Leaf Mastery Study-Order · 36-up Landscape · 6×6 · Sound 2 · High · Med

	Trees & BST	
#1490 Clone N-ary Tree		NEED
LeetCode - Simplified - WkUdC - LeetCode		
Hash Table - Tree - DFS - BFS		
Problem Statement		
Time Complexity (O(N)), where N is the number of nodes in the tree. Since each node is visited exactly once. Space Complexity (O(N)), accounting for the recursive call stack or an explicit queue on BFS.		
Problem		
Given a root of an N-ary tree, return a deep copy (clone) of the tree.		
Each node in the n-ary tree contains a val (int) and a list (List<Node>) of its children.		
Example 1:		
<pre>graph TD A["root = {val: 1, children: [2, 3]}"] --> B["{val: 2, children: [4, 5]}"] A --> C["{val: 3, children: [6]}"] B --> D["{val: 4, children: []}"] B --> E["{val: 5, children: []}"] C --> F["{val: 6, children: []}"]</pre>		
Tree 1: First input serialization is represented by this level-order traversal, each group of children is separated by the null value (see Example).		
Example 1:		

```

graph TD
    1((1)) --- 2((2))
    1 --- 4((4))
    2 --- 5((5))
    2 --- 6((6))
  
```

Diagram illustrating a binary tree structure with root node 1. The root node 1 has children 2 and 4. Node 2 has children 5 and 6.

Input: root = (1,null,2,2,4,null,5,4)
 Output: [1,null,2,2,4,null,5,4]

Example 2 : High Medium

```

graph TD
    1((1)) --- 2((2))
    1 --- 3((3))
    2 --- 4((4))
    2 --- 5((5))
    3 --- 6((6))
    3 --- 7((7))
    4 --- 8((8))
    4 --- 9((9))
    5 --- 10((10))
    5 --- 11((11))
    6 --- 12((12))
    6 --- 13((13))
    7 --- 14((14))
    7 --- 15((15))
  
```

Input: root = {1,0,1,2,3,4,5,0,0,1,6,7,8,0,0,1},
 isBst = {0,0,0,0,0,1,0,0,0,1,1,1,1,1,1,1},
 Output: 12

Input: root = {2,1,6,0,0,0,0,0,0,0,0,0,0,0,0,0},
 isBst = {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0},
 Output: 0

Constraints:

- The depth of the n-ary tree is less than or equal to 1000.
- The total number of nodes is between [0, 10⁴].

Follow up: Can you solve this problem for the graph problem?

```

Brain Force (Python)

# Brain Force Approach
class Solution:
    def climbStairs(self, n: int) -> int:
        # Brain Force - collect order, then
        process series list
        memo = {}
        def collect(n):
            if not memo: return
            collect(n-1)
            val1, appendOrder = val
            collect(n-2, right)
        collect(n-1)
        return val[n-1] if n else 0
# Community Solution (Python)

• LeetCode

```

```

# Validation for a Node.
class Node:
    def __init__(self, val=None, children=None):
        self.val = val
        self.children = children if children is
        None else []

class Solution:

    def dfs(self, root: 'Node') -> 'Node':
        if root is None:
            return None
        children = [self.dfs(child)
                    for child in root.children]
        return Node(root.val, children)

    def simplifyTree(self) -> 'Node':

```

```
# Definition for a Node.
class Node:
    def __init__(self, val=None, children=None):
        self.val = val
        self.children = children if children is None else []

def dfs(node):
    if not node:
        return None
    if not node.children:
        return node
    # Create a list for the current node.
    cloned_node = Node(node.val)
    # Recursively clone each child and add it to the
    # cloned_node's children list.
    for child in node.children:
        cloned_child = dfs(child)
        cloned_node.children.append(cloned_child)
    # Return the cloned node.
    return cloned_node

# Example usage:
# original_tree = Node(1, [Node(2), Node(3), Node(4), Node(5), Node(6)])
# cloned_tree = cloneTree(original_tree)
```

```

class Node
{
public:
    //Definition for a Node.
    class Node
    {
    public:
        int val;
        Node* next;
        Node* prev;
        Node* child;
        Node(int val): val(val), next(nullptr), prev(nullptr), child(nullptr) {}
        Node(int val, Node* next, Node* prev, Node* child): val(val), next(next), prev(prev), child(child) {}
    };
};

```

Times 857	
1522 Diameter of N-ary Tree	Hard
LeetCode - https://leetcode.com/problems/diameter-of-n-ary-tree/">https://leetcode.com/problems/diameter-of-n-ary-tree/	
Tree - DFS	
Time Complexity - $O(N)$	
Space Complexity - $O(N)$	
Given a root of an N-ary tree, you need to compute the length of the diameter of the tree. The diameter of an N-ary tree is the length of the longest path between any two nodes in the tree. This path may or may not pass through the root. (N-ary tree serialization is represented in this level order traversal, each group of children is separated by the null value.) Example 1:	

```

graph TD
    1((1)) ---|red| 3((3))
    1 ---|black| 2((2))
    3 ---|black| 5((5))
    3 ---|black| 6((6))
    2 ---|black| 4((4))
  
```

Input: root = {1, null, 3, 2, 4, null, 5, 6}

Output: 3

Explanation: Diameter is shown in red color.

Example 2:

Example 1

Input: root = [1,null,2,null,3,4,null,5,null,6]
Output: 4
Example 1

Round 2 - High - Medium

[illegible]

```
class Solution:
    def diameter(self, root: 'Node') -> int:
        ans = 0

        def dfs(node: 'Node') -> int:
            "Returns the maxium depth of the subtree rooted at root"
            nonlocal ans
            maxDepth1 = 0
            maxDepth2 = 0
            for child in root.children:
                maxDepth1 = maxDepth1 + 1 + dfs(child)
                maxDepth2 = max(maxDepth1, maxDepth2)
            ans = max(ans, maxDepth1 + maxDepth2)
            return 1 + maxDepth1

        dfs(root)

        return ans

# LeetCode
```

```

--
-- Definition for a Node.
class Node:
    def __init__(self, val=None, children=None):
        self.val = val
        self.children = children if children is not
None else []

--
-- class Solution:
--
-- def diameter(self, root: Node) -> int:
--
--     type root: Node
--     :type root: Node
--     :rtype: int
--
--     def dfs(node):
--         if not node:
--             return 0
--         return 1 +
--             max([dfs(child) for child in node.children])
--
--     return dfs(root)

```

```

    #! = *2 = 8
    for child in root.children:
        t = df(child)
        if t >= 1:
            #! = *2 + 1
            elif t >= 0:
                #! = max(maxes, #! + #C2)
            return t + 1

    ans = 0
    df(root)
    return ans

#Complexity:
# Time Complexity : O(N).
# Space Complexity : O(1).
# Class Name :
# Method Name : self.val,self.left,self.right as children[]
# Merged code :
self.children = children if children is not None else []

# class Solution:
#     def diameter(self, root: Node) -> int:
#         # Global variable to keep track of the maximum diameter found so far.
#         self.max_diameter = 0
```

```
def dfcircle():
    if not done:
        return 0 # Base case: if the node
        in Name, return depth+0

    # Variables to hold the top two max depths
    from children
    max_depth1, max_depth2 = 0, 0

    # Traverse over each child to compute
    their maxime depth
    for child in node.children:
        child_max = dfcircle(child) # Recursive
        on the child

        # Update the two largest depths
        if child_max > max_depth1:
            max_depth1, max_depth2 =
            max_depth1 + child_max
            max_depth2 = child_max

        # Update the maxime diameter.
        The candidate is the sum of the two deepest paths.
        self.max_diameter =
        max(self.max_diameter, max_depth1 + max_depth2)
```

```
# Return the maximum depth from this node.
return max_depth3 - 1

dfs(node)
return self_max_diameter

class Solution:
    # Definition for a Node.
    class Node:
        def __init__(self, val: int, left: Node, right: Node):
            self.val = val
            self.left = left
            self.right = right

        def __str__(self):
            return f'Node({self.val}, {self.left}, {self.right})'

    def dfs(self, node):
        if not node:
            return 0, 0
        left_depth, left_diameter = self.dfs(node.left)
        right_depth, right_diameter = self.dfs(node.right)
        max_depth = max(left_depth, right_depth) + 1
        max_diameter = max(left_diameter, right_diameter, max_depth - 1)
        return max_depth, max_diameter

    def maxDiameter(self, root: Node) -> int:
        _, max_diameter = self.dfs(root)
        return max_diameter
```

```
def diameter(self, root: Node) -> int:
    """
    :type root: Node
    :rtype: int
    """
    def dfs(node):
        if not node:
            return 0
        left = dfs(node.left)
        right = dfs(node.right)
        self.ans = max(self.ans, 1 + left + right)
        return max(left, right) + 1

    dfs(root)
    return self.ans - 1
```

Times & BST

#1650 Lowest Common Ancestor of a Binary Tree III

LeeCode - SimpleLeet - WalrCe - LeeCode

Hash Table - Tree - Two Pointers

Time Complexity: 50

Time Complexity: O(N)

Problem

Given two nodes of a binary tree *t* and *s*, return their lowest common ancestor (LCA).

Each node has a reference to its parent node; definitions for Node is below.

```
class Node {
    public Node left;
    public Node right;
    public Node parent;
}
```

According to the definition of LCA on Wikipedia: "The lowest common ancestor of two nodes *p* and *q* is a node *x* such that *x* is the lowest node that has both *p* and *q* as a descendants (nodes are often said to be a descendant of itself)."

Example 1:

```
graph TD; 3((3)) --- 5((5)); 3 --- 1((1)); 5 --- 6((6)); 5 --- 2((2)); 1 --- 0((0)); 1 --- 8((8)); 2 --- 7((7)); 2 --- 4((4));
```

Input: root = [3,5,1,6,2,8,0,7,4], p = 5, q = 2

Output: 2

Explanation: The LCA of nodes 5 and 1 is 3.

Example 2:

```

graph TD
    3((3)) --- 5((5))
    3 --- 1((1))
    5 --- 6((6))
    5 --- 2((2))
    1 --- 0((0))
    1 --- 8((8))
    2 --- 7((7))
    2 --- 4((4))
  
```

Input: root = [3,5,1,2,6,8,4], p = 5, q = 4
 Output: 5
 Description: The LCA of nodes 5 and 4 starts at node 3 and is a descendant of LCA according to the LCA verification.

Example 2:
 Input: root = [1,2], p = 1, q = 2
 Output: 1
 Constraints:

- Number of nodes in the tree is in the range [2, 10⁵].
- 1 ≤ *N* ≤ 1000 & *N*total < 100
- All *N*total are unique.
- *n* = 0
- *n* and *n* exist in the tree.

Brute Force (Python)

```

1 # Brute Force Approach
2 class Solution:
3     def leastCommonAncestor(self, n1, n2):
4         # Brute Force - collect inorder, then
5         # process sorted list
6         vals = []
7         def collectInorder(n):
8             if not n.data:
9                 return
10            collectInorder(n.left)
11            vals.append(n.data)
12            collectInorder(n.right)
13
14        collectInorder(n1)
15        return vals[n1 < n2] if vals else #

```

• *naiveC++*

```

class Solution:
    # Same as 108. Intersection of Two Linked Lists
    def intersect(self, headA: Optional[ListNode], headB: Optional[ListNode]) -> Optional[ListNode]:
        'headA' -> 'headB'

        n = 0
        m = 0

        while n < m:
            n = n.next if n else 0
            m = m.next if m else 0

        return n

# Time Complexity: O(n)
# Space Complexity: O(1)

```

```

class Solution:
    def lowestCommonAncestor(self, p: 'Node', q: 'Node') -> 'Node':
        if p is null:
            return p
        while node:
            node = node.parent
        node = node.parent
        while node not in vis:
            node = node.parent
        return node

# Simplified
# Definition for a Node.
# class Node:
#     def __init__(self, val: int, left: Node, right: Node):
#         self.val = val
#         self.left = left
#         self.right = right
#         self.parent = parent
# def lowestCommonAncestor(self, p: Node, q: Node) -> Node:
#     # Compute the depth for p, 1

```

```
def getDepth(node):
    depth = 0
    while node:
        depth += 1
        node = node.parent
    return depth

depth = getDepth(p)
depth = getDepth(s)
```

Doing both nodes to the same depth level

```

p = p.parent
while depth >= depth(s):
    depth -= 1
    p = p.parent
    depth(s) += 1
```

Traverse up until we find the common ancestor

```

while p != q:
    p = p.parent
    q = q.parent

    return p
```

• iCafe

```

====
# Definition for a Node.
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
        self.parent = None

====

class Solution:
    def lowestCommonAncestor(self, p: 'Node', q: 'Node') -> 'Node':
        n, b, p, q =
        while n:
            n = n.parent if n.parent else q
            n = n.parent if n.parent else p
        return n

```

```
# ref: https://jamesgroomson.com/leetcode-3024-1-
# unit-consumption-of-a-bakery-train-silo-simplified
3076

def limitConcurrence(self, a: 'Node', b:
    'Node') -> 'Node':
    concurrence = self.l

    while a is not None:
        concurrence.add(a)
        a = a.parent

    while b is not None:
        if b in concurrence:
            return b
        b = b.parent

    # or another one
    def limitConcurrence(self, a: 'Node', b:
        'Node') -> 'Node':
        pointer1, pointer2 = a, b

        while pointer1 != pointer2:
```

```

        pointertok = pointertok.parent if pointertok
    else:
        pointertok = pointertok.parent if pointertok
    else:
        return pointertok

```

DFS

Round 2 - High - Medium - 16 questions

#110 Surrounded Regions

LeetCode - Simplified - WACC - LeetCode
Area: 2D, DFS, Union-Find, Matrix
Time: 20min
Yuan Zhang Zheng

Time Complexity: $O(n^2)$ Space Complexity: $O(n^2)$
is the worst case due to recursion stack or queue storing $O(n^2)$

Problem

You are given an $m \times n$ matrix board containing letters 'X' and 'O', capture regions that are surrounded.

- Connect a cell to a connected to adjacent cells horizontally or vertically.
- Region: to form a region connect every 'O' cell.
- Surrounded: A region is surrounded if it is formed by the 'X' cells in that region are on the edge of the board.

Regions that are completely surrounded by 'X' cells, capture them, and replace the 'O' with 'X'.

Example 1:

Input: board = `[["X","X","X","O"],`
`[["X","X","O","O"],`
`[["X","X","O","X"],`
`[["O","X","X","X"]]`

Output: `[["X","X","X","X","X"],`
`["X","O","X","X","X"],`
`["X","O","X","X","X"],`
`["X","X","X","X","X"],`
`["X","X","X","X","X"]]`

Explanation:

In the above diagram, the bottom region is not captured because it is on the edge of the board and cannot be surrounded.

Example 2: `Input: board = ["X"]`

Output: `["X"]`

Constraints:

- `m == board.length`
- `n == board[0].length`
- `1 <= m, n <= 100`
- `board[i][j] is 'X' or 'O'.`

Brute Force (Python)

Board 3: `["X","X","X","X","X"],`
`["X","O","X","X","X"],`
`["X","O","X","X","X"],`
`["X","X","X","X","X"],`
`["X","X","X","X","X"]]`

```

# Merge Sort Approach
class Solution:
    def sortArray(self, nums):
        # Merge Sort (Divide) - DFS from border
        # 1. Sort each array, flip the sort
        from collections import deque
        if len(nums) <= 1:
            return nums
        m = len(nums) // 2
        left = self.sortArray(nums[:m])
        right = self.sortArray(nums[m:])
        # Merge
        res = []
        i = j = 0
        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                res.append(left[i])
                i += 1
            else:
                res.append(right[j])
                j += 1
        # Add the remaining elements
        res += left[i:]
        res += right[j:]
        return res

```

```

for i in range(n):
    for j in range(m):
        if board[i][j] == "O" and (i,j) not
in safe:
            board[i][j] = "X"

Community Solutions (Python)

# wordSearch
class Solution:
    def solve(self, board: List[List[str]]) -> None:
        if not board:
            return

        DRS = [(0, 1), (2, 0), (0, -1), (-2, 0)]
        n = len(board)
        m = len(board[0])
        o = collections.deque()

        for i in range(n):
            for j in range(m):
                if board[i][j] == "O" and (i,j) not in o:
                    o.append((i, j))
                    board[i][j] = "X"

# Mark the grids that stretch from the four
sides with 0's
while q:
    i, j = o.popleft()

```

```

for dy, dy in DDG:
    x = 1, 0
    y = 0, 1
    if x < 0 or y < 0 or x < y or y == 0:
        continue
    if board[x][y] != 'O':
        continue
    q.append(x, y)
    board[x][y] = '*'

for row in rows:
    for i, c in enumerate(row):
        result[i] = 'O' if c == 'O' else '*'

class Solution:
    def solve(self, board: List[List[str]]) -> None:
        """
        Do not return anything, modify board in-place instead.
        """
        if not board or not board[0]:
            return
        n, m = len(board), len(board[0])
        for i in range(1, n - 1):
            for j in range(1, m - 1):
                if board[i][j] == 'O':
                    q = [(i, j)]
                    board[i][j] = '*'
                    while q:
                        x, y = q.pop()
                        for dx, dy in DDG:
                            nx, ny = x + dx, y + dy
                            if 0 < nx < n and 0 < ny < m and board[nx][ny] == 'O':
                                q.append((nx, ny))
                                board[nx][ny] = '*'
                    board[i][j] = 'O'

```

```

for i in range(1):
    offset, n = 1
    for j in range(1):
        offset, n = 2, 2
    for k in range(1):
        if board[i][j] == "x":
            board[i][j] = "o"
        elif board[i][j] == "o":
            board[i][j] = "x"

def Simulate():
    def solve(board):
        if not board or not board[0]:
            return
        row, col = len(board), len(board[0])
        def dfs(i, j):
            # Return if out of bounds or current cell is not "o"
            if i < 0 or i > R or j < 0 or j > C or board[i][j] != "o":
                return

```

```

# Mark the cell as safe
board[i][j] = '*'

# Explore four possible directions
df = 1
dfx = i + 1
dfy = j
while dfx < 2 and dfy < 2:
    if board[dfx][dfy] == '*':
        dfx = i + 1
        dfy = j + 1
    else:
        dfx = i
        dfy = j

# Traverse the board borders
for i in range(rows):
    if board[i][0] == '*':
        dfx = i
        dfy = 0
    for j in range(cols):
        if board[i][j] == '*':
            dfx = i
            dfy = j
        if board[rows - 1][j] == '*':
            dfx = rows - 1
            dfy = j

# Flip unmarked '*'s and restore safe cells
for i in range(rows):
    for j in range(cols):
        if board[i][j] == '*':
            board[i][j] = '.'
        elif board[i][j] == '.':
            board[i][j] = '*'

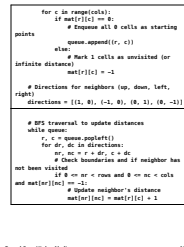
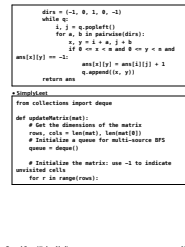
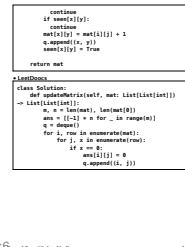
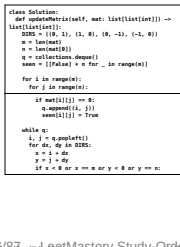
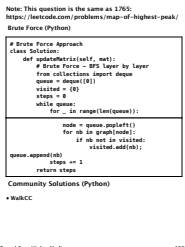
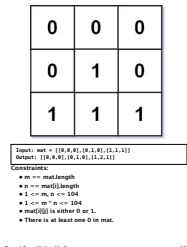
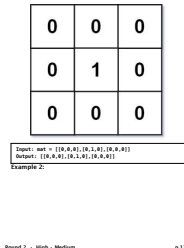
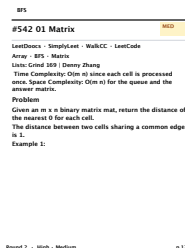
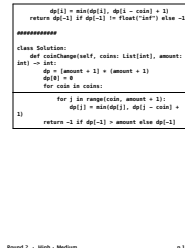
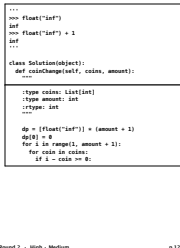
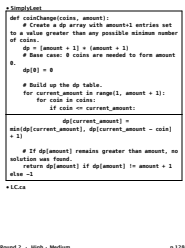
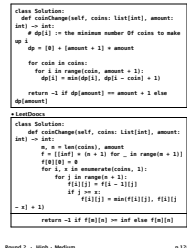
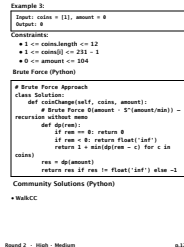
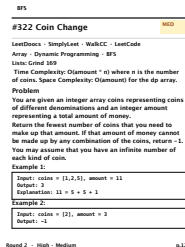
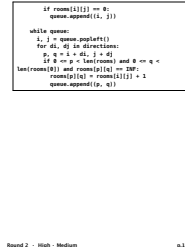
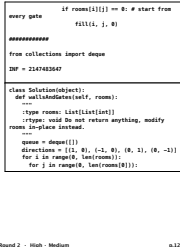
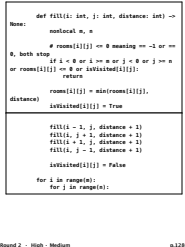
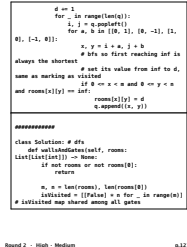
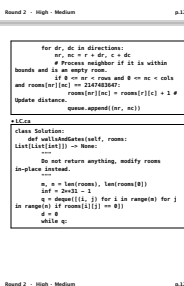
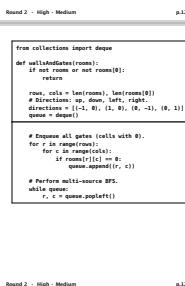
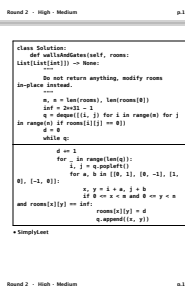
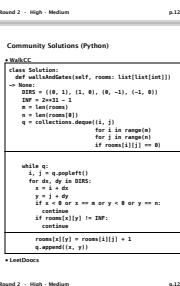
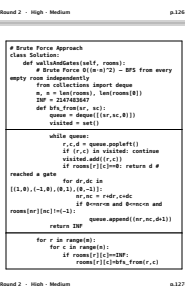
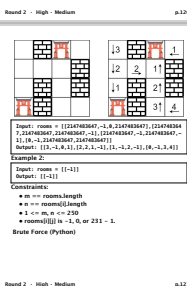
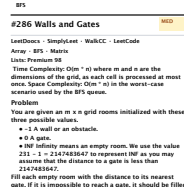
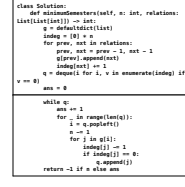
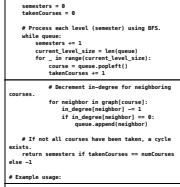
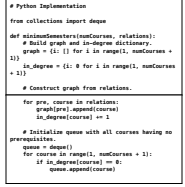
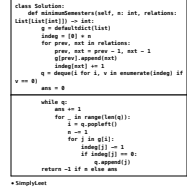
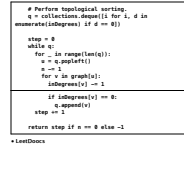
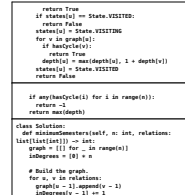
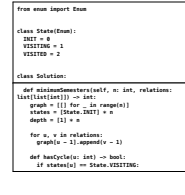
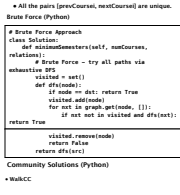
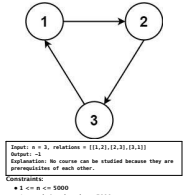
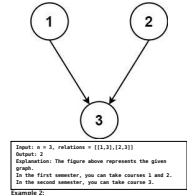
+100x

```

```

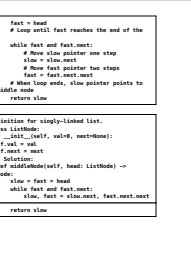
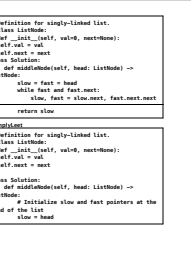
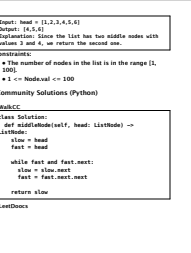
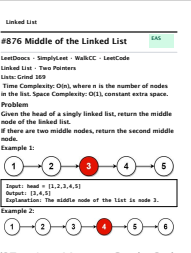
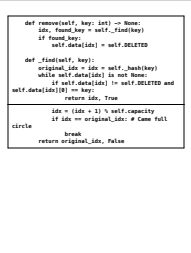
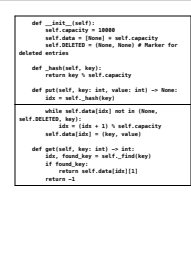
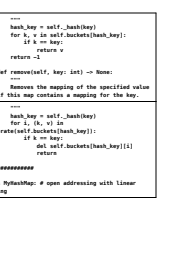
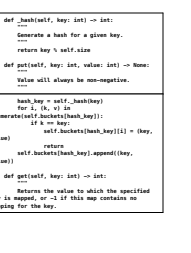
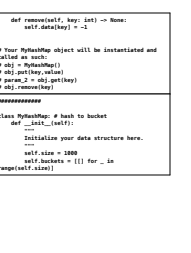
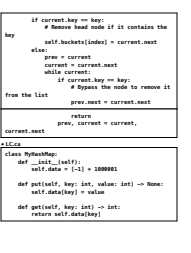
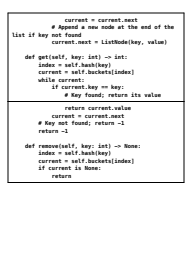
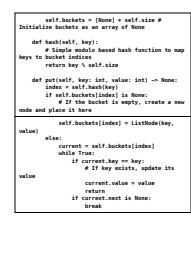
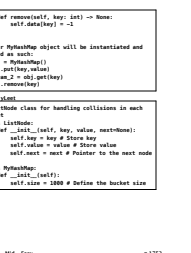
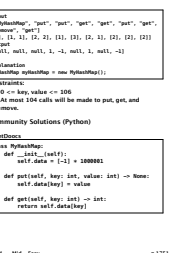
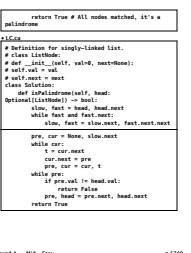
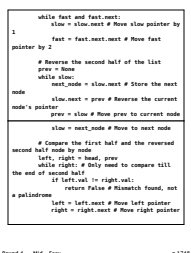
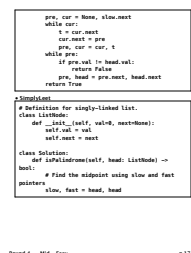
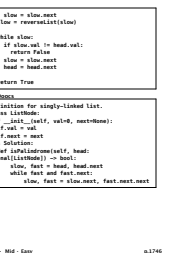
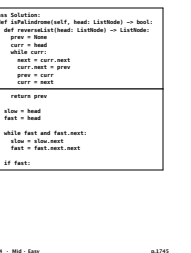
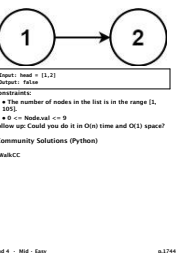
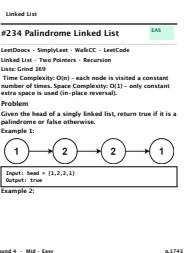
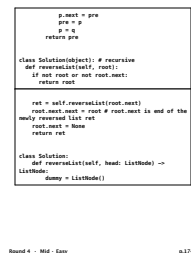
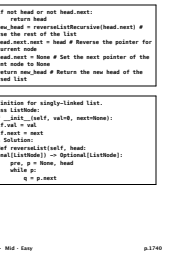
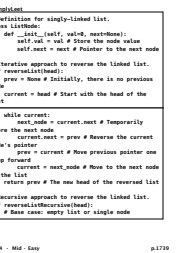
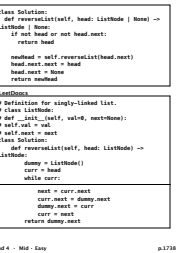
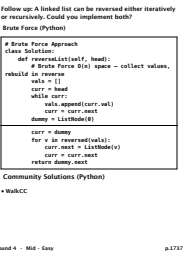
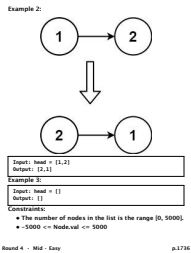
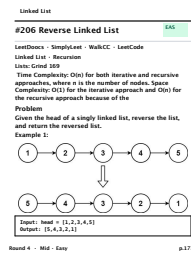
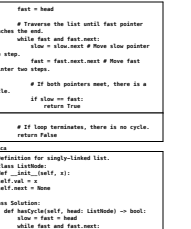
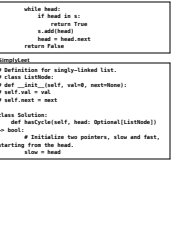
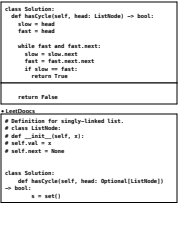
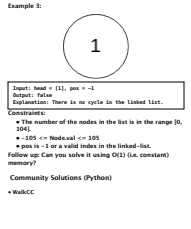
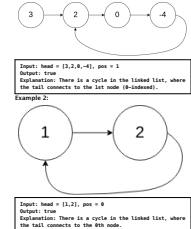
class Solution:
    def solve(self, board: List[List[str]]) ->
    None:
        """
        Do not return anything, modify board
        in-place instead.
        """
        def dfs(x, y):
            if R <= x <= R2 and C <= y <= C2:
                board[x][y] = "0"
            for i in range(x-1, x+1):
                for j in range(y-1, y+1):
                    if board[i][j] == "0" and (
                        i != R or j != C2 or j != C-1
                    ):
                        dfs(i, j)
        m = len(board)
        n = len(board[0])
        for i in range(m):
            for j in range(n):
                if board[i][j] == "0" and (
                    i != 0 or j != 0 or j != n-1 or j != n-2
                ):
                    dfs(i, j)

```

Sheet 45/87 · LostMastery Study-Order · 36-up Landscape · 6×6 · Sound 3 · High · Hard

Graphs Round 3 - High - Hard - 2 questions	#305 Number of Islands II LeetCode - Simplify: WalkCC - LeetCode Array: Hash Table - Union Find - Matrix Union-Find Problem 89 Time Complexity: $O(N \times M \times \log N)$ where N is the number of operations and M is the number of islands. Space Complexity: $O(N \times M)$ for storing the union-find set. Problem: You are given an empty 2D binary grid <code>grid</code> of size <code>n x m</code>. The grid represents a map where <code>0's</code> represent water and <code>1's</code> represent land. Initially, all the cells of <code>grid</code> are water (<code>0</code>), and the cells are <code>0's</code>. You may perform an add land operation which turns the water at position <code>(i, j)</code> into a land. You are given an array <code>positions</code> where <code>positions[i] = [i, j]</code> is the position <code>(i, j)</code> at which we should operate the <code>i</code>th operation. Return an array of integers <code>answer</code> where <code>answer[i]</code> is the number of islands after turning the cell <code>(i, j)</code> into a land. An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water. Example 1:	Input: n = 3, m = 3, positions = [[0, 0], [0, 1], [0, 2], [1, 0], [2, 2]] Output: [1, 2, 3] Explanation: Initially, the 3x3 grid is filled with water. Operation 0: addLand(0, 0) turns the water at (0, 0) into a land. We return 1. Operation 1: addLand(0, 1) turns the water at (0, 1) into a land. We return 2. Operation 2: addLand(0, 2) turns the water at (0, 2) into a land. We return 3. Operation 3: addLand(1, 0) turns the water at (1, 0) into a land. We return 3. Operation 4: addLand(2, 2) turns the water at (2, 2) into a land. We return 3. Example 2: Input: n = 1, m = 1, positions = [[0, 0]] Output: [1] Constraints: $1 \leq n, m \leq 10^5$, positions.length $\leq 10^4$ $0 \leq i < n$, $0 \leq j < m$ $0 \leq i < n$, $0 \leq j < m$ $0 \leq i < n$, $0 \leq j < m$ Follow up: Could you solve it in time complexity $O(N \log N)$, where N is positions.length? Brute Force (Python)
---	---	--



Sheet 51/87 - LoofMastery Study-Order - 36-up Landscape - 6x6

Sheet 52/87 · Mid · LowMastery Study-Order · 36-up Landscape · 6×6 · Round S · Mid · Medium p.1870


```
Input: maze = [[0,0,0,0,0],[1,1,0,0,1],[0,0,0,0,0],[0,1,1,1,1],[0,1,0,0,1]], start = (4,1), destination = (0,1)
Output: -1
```

Constraints

- $m = \text{maze.length}$
- $n = \text{maze[0].length}$
- $1 \leq m, n \leq 100$
- maze[i][j] is 0 or 1
- $\text{start.length} = 2$
- $\text{destination.length} = 2$
- $0 \leq \text{start[0], destination[0]} < m$
- $0 \leq \text{start[1], destination[1]} < n$

• Both the start and the destination exist in an empty square, and they will not be in the same position initially.

• The maze contains at least 2 empty spaces.

Bruce Force (Python)

```
# Brute Force Approach
class Solution:
    def shortestDistance(self, nums, start, destination):
        # Brute Force (Br Log n) - sort repeatedly instead of heap
        data = list(nums)
        result = []
        for i in range(1, len(data)):
            data.sort()
            result.append(data.pop(0))
        return result
```

Community Solutions (Python)

```
class Solution:
    def shortestDistance(self, nums, start: List[int], destination: List[int]):
        i = int(-1)
        j = int(-1)
        for i in range(1, len(nums)):
            nums[i-1], nums[i] = min(nums[i-1], nums[i]), max(nums[i-1], nums[i])
        for i in range(1, len(nums)):
            if i == start:
                j = i
            elif i == destination:
                break
        dist = [i for i in range(1, len(nums))]
        return dist[j]
```

[illegible]

```

directions = [-1, 0, 1, 0, 0, -1], (0, -1), (0, 1)
while heap:
    d, row, col = heappop(heap)
    # If we have reached the destination,
    return the corresponding distance
    if (row, col) == destination:
        return d
    # If it is not already found a better path to
    (row, col)
    if d < dist[row][col]:
        continue
    for dir, d2 in directions:
        next_row, next_col = row + dir[0], row + dir[1]
        # We'll bail the unit out if it's a wall,
        # or if it's already been found
        while d2 <= d and next_row[0] <= d2[next_row[0]
        + 1] <= d2[next_row[0] + 1]:
            next_row, next_col = row + dir[0], row + dir[1]
            count += 1
        # Check if the new distance is shorter
        if d < count:
            dist[next_row][next_col] = d
            dist[next_row][next_col] = d +
            count

```

```

heapq.heapqpush(heap,
[dist[next_row][next_col], next_row, next_col])
return -1

def bfs():
    class Solution:
        def shortestPath(self, grid: List[List[int]]):
            n = len(grid)
            m = len(grid[0])
            dist = [[-1] * m for _ in range(n)]
            q = [(0, 0, 1)]
            while q:
                d, i, j = heapq.heappop(q)
                if dist[i][j] < d:
                    continue
                for di, dj in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
                    ni, nj = i + di, j + dj
                    if 0 <= ni < n and 0 <= nj < m and grid[ni][nj] < 2 and dist[ni][nj] > d + 1:
                        dist[ni][nj] = d + 1
                        heapq.heappush(q, (d + 1, ni, nj))
            return dist[0][m - 1] if dist[0][m - 1] > -1 else -1
    sol = Solution()
    return sol.shortestPath(grid)

```

```
return -1 if dist[d1][d2] == inf else
dist[d1][d2]
```

Round 1 - Mid - Medium

2:30

#621 Task Scheduler

Medium

LeetCode, SimpleHash, Stack, List, LintCode

Array, Hash Table, Greedy, Binary Search, Counting

Code: 188

Task Complexity: O(N) where N is the number of tasks (inserting frequency), then iterating over constant 20 intervals. Since Complexity: O(N) since the frequency used was an array of hash map with five

Problem

You are given an array of CPU tasks, each labeled with a letter from A to Z, and a number n. Each CPU interval has to finish at least the completion of one task. Tasks can be completed in any order, but there is a constraint: there must be a gap of at least n intervals between two tasks with the same label.

Return the minimum number of CPU intervals required to complete all tasks.

Example 1:

Input: tasks = ["A","A","A","B","B","B"], n = 2

Output: 8

Explanation: A possible sequence is: A -> B -> A -> B -> A -> B -> idle -> B.

Round 1 - Mid - Medium

2:30

Round 5 – Mid – Medium 2/30

After completing task A, you must wait 2 intervals before doing again. The same applies to task B. In the 3rd interval, neither A nor B can be done, so you idle. By the 4th interval, you can do A again & 2 intervals have passed.

Example 2:
 Input: tasks = ["A","C","C","B","B","B"], n = 1
 Output: 6
 Explanation: A possible sequence is A -> C -> O -> C -> O -> B.
 With a cooling interval of 1, you can repeat a task after just one other task.

Example 3:
 Input: tasks = ["A","A","A","B","B","B"], n = 3
 Output: 10
 Explanation: A possible sequence is A -> B -> idle -> A -> B -> idle -> A -> B -> idle -> A -> B.
 There are only two types of tasks, A and B, which need to be separated by 3 intervals. This leads to idling twice between repetitions of these tasks.

Constraints:

- tasks.length <= 100
- taskid in tasks is an English letter (a-z).
- n <= 100

Brute Force (Python)

```
Round 2 - Mid - Medium
```

```
# Single Pass Approach
class Solution {
    def longestInterval(self, tasks: List[int]) -> int:
        # Use CPU schedule list by tick
        from collections import Counter
        counts = Counter(tasks)
        time = 0
        while counts:
            # Pick up task; pick up to (n-1)
            count = counts.most_common(1)[0][1]
            heap = [time + count, counts.popitem()]
            heap.sort()
            for i in range(count):
                counts[i] -= 1
            time += 1
            if counts[i] == 0: del counts[i]
            if counts: # idle the rest of the cycle
                time += heap[0] - 1 - len(heap[0])
            return time
```

Community Solutions (Python)

```
==>MUSIC
```

```
Round 2 - Mid - Medium
```

```
#>2
```

```

class Solution {
    def leastInterval(char[] tasks, int n): {
        int[] count = new int[26];
        for (char task : tasks) {
            count[task - 'A']++;
        }
        // Put the most frequent task in the slot first
        Arrays.sort(count);
        // Get the number of tasks with same frequency as maxFreq, we'll append them after the maxFreq
        int maxFreq = count[25];
        // Get the number of tasks with less frequency as maxFreq
        int next = count[24];
        // If the next frequent task is frequent enough to
        // force some idle slots,
        // then the next frequent task is not frequent enough
        // to force some idle slots
        // }
        return maxFreq + (next * (n - maxFreq)) + next;
    }
}

```

• LeetCode

```

Round 5 - Mid - Medium

class Solution {
public:
    def leastInterval(self, tasks: List[str], n) -> int:
        cnt = {}
        for t in tasks:
            cnt[t] = cnt.get(t, 0) + 1
        tasks = sorted(cnt.items(), key=lambda x: x[1], reverse=True)
        res = 0
        while tasks:
            res += 1
            task, count = tasks.pop()
            if count > 1:
                tasks.append((task, count - 1))
            # If there are n tasks, then the next task can be scheduled after n units of time.
            # If there are less than n tasks, then the next task can be scheduled after the last task.
            if len(tasks) < n:
                res += n - len(tasks)
            else:
                res += n
        return res
}

Python solution for Task Scheduler
from collections import Counter

def leastInterval(tasks, n):
    # Count the frequency of each task
    task_counts = Counter(tasks)
    # Find the maximum frequency among tasks
    max_freq = max(task_counts.values())
    # Count how many tasks have the maximum frequency
    max_count = sum(1 for count in task_counts.values() if count == max_freq)
}

```

[illegible]

```

class Solution:
    def longestInterval(self, tasks: List[int], n: int) -> int:
        task = Counter(tasks)
        s = max(task.values())
        n = n - s + 1
        for i in range(1, n + 1):
            return max(task, (i - 1) * n + i) + 1

```

#658 Find K Closest Elements

LeetCode · Simplify · Watch · LeetCode

Array · Binary Search · Sorting · Hash · Two Pointers

Like · 100 · 100 · 100 · 100 · 100 · 100 · 100 · 100 · 100

Time Complexity: $O(\log n + k)$, where $\log n$ for binary search and $O(k)$ for collecting results. Space Complexity: $O(k)$ for storing the output.

Problem

Given a sorted integer array arr , two integers k and x , return the k closest integers to x in the array. The result should also be sorted in ascending order.

An integer a is closer to x than an integer b if:

- $a < b$ and $|a - x| < |b - x|$
- $a > b$ and $|a - x| < |b - x|$

Example 1:

```
Input: arr = [1,2,3,4,5], k = 4, x = 3
Output: [2,3,4,5]
```

Example 2:

```
Input: arr = [1,2,3,4,5], k = 4, x = -1
Output: [1,2,3,4]
```

Constraints:

- $1 \leq k \leq \text{arr.length}$

Round 5 - Mid - Medium

2/21

```

• 1 <= arr.length <= 104
• arr is sorted in ascending order.
• 104 <= arr[i], x <= 104

Brute Force (Python)

• Brute Force Approach
class Solution:
    def findClosestElements(self, arr: List[int], k: int, x: int) -> List[int]:
        # Brute Force (Log n) - sort repeatedly
        for i in range(k):
            result = []
            for _ in range(k):
                data = arr
                result.append(data.pop(0))
            return result

Community Solutions (Python)

• WalkC

```

```

class Solution {
public:
    def findIntersectionPoints(self, arr: list[int], k: int) -> list[int]:
        set, x, ans = list(set()), 0, []
        while True:
            r = len(arr) - k
            while l < r:
                m = (l + r) // 2
                if m - arr[m] == arr[m] - x:
                    x = m
                    ans.append(m)
                else:
                    l = m + 1
            return ans if l < k else []
}

// Time Complexity: O(N)
// Space Complexity: O(1)

```

```

# Function to find the k elements closest to x in a
# sorted array
def findKClosestElements(A, x, k):
    # Initialize the binary search bounds for the
    # starting index of the window
    left, right = 0, n
    # Binary search for the best starting index
    while left < right: // 2
        mid = (left + right) // 2
        # Compare the distance from x to arr[mid]
        # and arr[mid-1]
        # If arr[mid] is further from x than
        # arr[mid-1], move window to the left
        if x - arr[mid] > arr[mid] - x:
            left = mid + 1
        else:
            right = mid
    # Towards the left
    # Return the k elements starting from index
    # left
    return arr[left:left + k]

# Example usage
# arr = [1,2,3,4,5,6,7,8,9]
# k = 3, x = 4.5
# Output: [3,4,5]

```

```

class Solution:
    def findInsertElements(self, arr: List[int])
        k: int = len(arr) - List(arr)
        left, right = k, len(arr) - k
        while left < right:
            mid = (left + right) // 2
            if arr[mid] > arr[mid + k] == 1:
                right = mid
            else:
                left = mid + 1
        return arr[left : left + k]

```

[illegible]

```

graph TD
    0((0)) ---|100| 1((1))
    0((0)) ---|100| 2((2))
    1((1)) ---|100| 2((2))
    1((1)) ---|600| 3((3))
    2((2)) ---|200| 3((3))
    0((0)) ---|500| 3((3))
  
```

Input: $n = 4$, $\text{flights} =$
 $[(0, 1, 100), (1, 2, 100), (2, 3, 200), (1, 3, 600), (0, 2, 100), (0, 3, 500)]$
 src = 0, dest = 2, k = 1
 Output: 700

The graph is shown above.

The optimal path is to take a stop from city 0 to 2 is
 0 is marked as red and take $100 + 600 = 700$.
 Note that the path through 1, 2, 3 is cheaper
 but is invalid because it uses 2 stops.

Example 2:

```

graph TD
    0((0)) -- 100 --> 1((1))
    0((0)) -- 500 --> 2((2))
    1((1)) -- 100 --> 2((2))

```

Input: $n = 3$, flights =
 $[[0, 1, 100], [0, 2, 500], [1, 2, 100]]$, src = 0, dst = 2, k =
 1
 Output:
 100
 Explanation:
 The graph is shown above.
 The optimal path with at most 1 stop from city 0 to 2
 is reached by red and has cost: $100 + 0 = 100$.

Example 2:

```

graph TD
    0((0)) ---|100| 1((1))
    0 ---|500| 2((2))
    1 ---|100| 2
  
```

Input: $n = 3$, $Right = [0, 1, 500]$, $Left = [0, 2, 100]$, $src = 0$, $dst = 2$

Return type: `int`

Explanation:
The graph is shown above.
The shortest path with no slope from city 0 to 2 is marked in red and has cost 500.

Constraints:

- $0 < n < 100$
- $0 <= Right.Length <= (n - 1) / 2$
- $0 <= Left.Length <= 3$
- $0 <= Right[i], Left[i] <= 100$

Round 2 - Mid - Medium

```

• Sum = tol
• n = c * (m - 1) / 2
• There will not be any more flights between two cities.
• 0 <= w_i, d_i, b_i < n
• w = tol - tol

Route Force (Python)

• Route Force Approach
class Solution:
    def findCheapestPrice(self, n, flights, src, dst, k):
        # Route Force O(n^2 * k) O(n^2) trying all paths with at most k stops
        from collections import defaultdict
        graph = defaultdict(list)
        for u, v, w in flights:
            graph[u].append((v, w))
        self.res = float('inf')
        def dfs(src, stops, cost):

```

```

if node == del:
    self.remove(self.root, cont):
return
if steps < 0: # return
    for root, price in graph(nodes):
        if cont < price * self.root # go
only on cost
        df(self, steps+1, cont+price
df(self, 0, 0)
return self.root if self.root < float('inf')
else -1

```

Community Solutions (Python)

```

#BFS
class Solution:
    def findCheapestPrice(
        self,
        flights: List[List[int]],
        src: int,
        dst: int,
        k: int) -> List:
        graph = {} for _ in range(n):

```

Round 5 - Mid - Medium

```

for u, v, w in flights:
    graph[u].append((v, w))

return self._dijkstra(graph, src, dest, k)

def _dijkstra(
    self,
    graph: List[List[Tuple[int, int]]],
    src: int,
    dest: int,
    k: int
) -> List:
    dist = [math.inf] * (k + 2) for _ in
range(len(graph))

    dist[src][k] = 0

    heap = [(dist[src][k], src, k, 2)]

    (d, u, steps) = heappop(heap)

    while u != dest:
        d, u, steps = heappop(heap)

```

```

if s == 0:
    return d
if steps == 8 and d: distal[steps]:
    continue
for v in graph[1]:
    if s + v == distal[steps - 1]:
        h = distal[steps - 1] + v
        hmap[hmap.keys()+hmap, distal[steps -
2], v, steps - 1]
return -1

#main
class Solution:
    def findShortestPath(self,
        hmap):
        n = len(hmap); k = len
        -1
        INF = 2**32-1
        dist = [INF]*n
        dist[0] = 0
        for _ in range(n):
            h = dist.copy()
            for v, s, p in hmap:
                dist[s] = min(dist[s], hmap[p][v] +
                h[s])
        #
        return -1 if dist[n] == INF else
        dist[n]

```

```

3 simulation
4 import numpy
5
6 def findNeighboursFrom(c, flights, src, dst, k):
7     # Build graph, each city maps to a list of
8     # neighbouring cities (flight origin)
9     g = {}
10    for i, city in enumerate(c):
11        for src, dst, city, price in flights:
12            g.setdefault(src, []).append((dst, city, price))
13
14    # Priority queue: (total cost, current city,
15    # steps so far)
16    heap = [(0, src, 0)]
17
18    # Dictionary to store the best cost to reach a
19    # city with a given step count
20    best = {}
21
22    while heap:
23        cost, city, steps = heapq.heappop(heap)
24        # Return cost if destination is reached
25        # if city == dst:
26            return cost

```

```

% If the current number of steps exceeds
skip further processing
if steps > k
    continue

% Expand the neighborhood flights
for neighbor, price in graph[city]
    new_cost = cost + price
    % Only proceed if this route is cheaper
    not yet explored for this step
    if neighbor < best[neighbor, step + 1]
        best[neighbor, step + 1] =
            new_cost

        keep_neighborhoods = keep_new_cost =
            new_cost < 2 * best_cost
        return new_cost
    end

% Example usage:
prices = graph([1,3,4,5,6], [1,3,4,5,6], [1,2,3,4,5],
    1.5, 100, [1,3,4,5,6], [2,3,4,5,6], 3, 1)

% VCAs

```

```

class Solution {
    def findClosestPrime(int left, int right):
        self.is_prime = is_prime
        return self.findClosestPrime(left, right)

    def is_prime(self, n):
        if n < 2:
            return False
        for i in range(2, int(n**0.5) + 1):
            if n % i == 0:
                return False
        return True

    def findClosestPrime(self, left, right):
        for i in range(left, right + 1):
            if self.is_prime(i):
                return i
        return -1
}

```

#733 K Closest Points to Origin

LeetCode - SimpleHash & QuickSort - LeetCode

Author: Nihal - Divide and Conquer - Sorting - Heap

Link: <https://leetcode.com/problems/k-closest-points-to-origin/>

Time Complexity: $O(n \log n)$ with a sorting algorithm, $O(n \log k)$ with a heap algorithm, or $O(n \log k)$ with QuickSelect algorithm. Space Complexity: $O(1)$ if extra space is not used for sorting, or $O(k)$ for Heap

Problem

Given an array of points where $points[i] = [x_i, y_i]$ represents a point on the X-Y plane and an integer k , return the k closest points to the origin $(0, 0)$.

The distance between two points on the X-Y plane is the Euclidean distance ($\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$).

You may return the answer in any order. The answer is guaranteed to be unique (for the order that it is returned).

Example 1:

Input: $points = [[1, 2], [3, 4], [-1, -2], [1, -1], [-1, 2]]$, $k = 2$

Output: $[[1, -1], [-1, 2]]$

Explanation: The distance between the points in the input is $\sqrt{5}$, $\sqrt{20}$, $\sqrt{5}$, $\sqrt{2}$, and $\sqrt{13}$ respectively. The two closest points to the origin are $(1, -1)$ and $(-1, 2)$.

Input: $points = \{(-1,1),(-2,2)\}$, $k = 1$
Output: $\{(-2,1)\}$
Explanation:
The distance between $(-1, 1)$ and the origin is $\sqrt{2}$.
The distance between $(-2, 2)$ and the origin is $\sqrt{8}$.
Since $\sqrt{2} < \sqrt{8}$, $(-1, 1)$ is closer to the origin.
We only need the closest $k = 1$ points from the origin, so the answer is just $\{(-1,1)\}$.
Example 2:

```
Input: points = [[2,3],[5,-1],[-2,4]], k = 2
Output: [[2,3],[5,-1]]
Explanation: The answer [[2,3],[5,-1]] would also be
valid.
```

Constraints:

- 1 ≤ k ≤ points.length ≤ 104
- 100 ≤ xi, yi ≤ 100

Brute Force (Python)

```
# Brute Force Approach
class Solution:
    def KClosest(self, points):
        # Brute Force: For each p in points, sort all points
        # by Euclidean distance
        points.sort(key=lambda p: p[0]**2 + p[1]**2)
        return points[:k]
```

Community Solution (Python)

- `WahlCC`

```

class Solution {
    def findPairs(self, nums: List[int], k: int) -> List[List[int]] {
        mapNums = {}
        for num, y in nums.items():
            if (k < 0) continue
            if (k > 0) {
                mapNums[num - k] = mapNums.get(num - k, 0) + 1
                mapNums[num + k] = mapNums.get(num + k, 0) + 1
            }
        return mapNums.items()
    }
}

```

Complexity:
 Time: $O(N)$
 Space: $O(N)$

LeetCode: <https://leetcode.com/problems/k-diff-pairs-in-an-array/>

Notes:

Sheet 59 of 60

Mastery Study-Oz

```
# Python Solution
def findClosest(points, N):
    # Sort the points based on their squared
    # Euclidean distance from the origin
    points.sort(key=lambda point: point[0]**2 +
    point[1]**2)
    # Return the first k points from the sorted
    # list
    return points[:N]

# Example usage
# points = [(1,1), (-2,2)]
# k = 1
# print(findClosest(points, N))
```

#1057 Campus Bikes

```
lastEnds = SimpleList <WallClock> LastEndCode
Array <Ready> = ReadyList
Array <Previous End> = PreviousEndList
Time Complexity (O = n * log(n + m)) where n is the
number of workers and m is the number of bikes.
Time Complexity (O = m * log(n + m)) where n is the
number of workers and m is the number of bikes.
```

Problem

The campus represented on the X, Y plane, there are n workers and m bikes, with $n \leq m$.

We are given an array $workers$ of length n where $workers[i]$ = (x_i, y_i) is the position of the i th worker. We are also given an array $bikes$ of length m where $bikes[i]$ = (x_i, y_i) is the position of the i th bike. All the given positions are unique.

Assign a bike to each worker. Among the available bikes and workers, we choose the shortest pair, link that pair with the shortest Manhattan distance between them, and repeat the process until no more pairs can be made. This process results in a matching between the given workers and bikes.

If there are multiple shortest pairs between workers and bikes, we choose the pair with the smallest worker index. If there are multiple pairs that share the same shortest distance from the worker, we choose the pair with the smallest bike index.

Return s - the sum of all $Manhattan$ distances between pairs.

Runtime = 864 - Medium

Report this process until there are no available work
Returns an array `ans` of length `n`, where `ans[i]` is the index (`i`-indexed) of the link that the `i`th worker assigned to.

The Manhattan distance between two points `p1` and `p2` is `Manhattan(p1, p2) = |p1.x - p2.x| + |p1.y - p2.y|`.

Example 1:

Worker 0

Worker 1

Worker 2

Worker 3

Manhattan

Bound : N^2 - N - Medium

Sheet 61/87 · LostMastery Study-Order · 36-up Landscape · 6x6 · Bound 5 · Mid · Medium · p.2194

Sheet 62/87 · Mid-Mastery Study-Order · 36-up Landscape · 6×6 · Sound 5 · Mid-Medium · p.2230

Sheet 63/87 · LostMastery Study-Order · 36-up Landscape · 6x6 · Round 5 · Mid · Medium · p.2266


```

if self.capacity <= 0:
    return
if key in self.key_to_val_freq:
    # Update value and frequency if key
    exists.
    self.key_to_val_freq[key] = (value,
self.key_to_val_freq[key][1])
    self.update_freq(key)
else:
    # If cache is full, evict the least
    frequently used (and least recently used)
    if len(self.key_to_val_freq) ==
self.capacity:
        # Remove the first key from the
        lowest frequency list.

```

```

        key_to_remove, _ = self.freq_to_key[
            self.min_freq, position(last_val)]
        del
        self.key_to_val[key_to_remove]
        if not
            self.freq_to_keys[self.min_freq]:
            del
            self.freq_to_keys[self.min_freq]
        # Insert the new key with frequency 1.
        self.key_to_val[key] = (value, 1)
        if 1 not in self.freq_to_keys:
            self.freq_to_keys[1] =
OrderedDict({
            self.freq_to_keys[1] = None
            self.min_freq, 1 # Reset min_freq.
}

+LCss
from collections import defaultdict

class LFUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.key_to_value = {}
        self.key_to_freq = defaultdict(int)
        self.freq_to_keys =
        defaultdict(OrderedDict)
        self.min_freq = 0

```

```
def get(self, key: int) -> int:
    if key not in self.key_to_value:
        return -1

    # Update the frequency
    self._update_frequency(key)
    return self.key_to_value[key]

def put(self, key: int, value: int):
    if self.capacity == 0:
        return

    if key in self.key_to_value:
        # Update the value and frequency
        self.key_to_value[key] = value
        self._update_frequency(key)
    else:
        if len(self.key_to_value) == self.capacity:
            # Evict the least frequent key
            self._evict()
```

```
# Add the new key-value pair
self.key_to_value[key] = value
self.key_to_freq[key] += 1
self.freq_to_keys[freq] = None
self.min_freq = freq + 1

def update_frequency(self, key: int) -> None:
    freq = self.key_to_freq[key]
    del self.freq_to_keys[freq][key]

    if not self.freq_to_keys[freq]:
        # If there are no keys with the
        # previous frequency, update min_freq
        if self.min_freq == freq:
            self.min_freq += 1
        del self.freq_to_keys[freq]

    freq += 1
    self.key_to_freq[key] = freq
    self.freq_to_keys[freq][key] = None
```

```
def _write(self) -> None:
    key = self.freq_to_key(self.min_freq)
    obj, key = _key_pos(LastValue)
    del self.key_to_value[prev_key]
    del self.key_to_freq[prev_key]

# Your LRUCache object will be instantiated and
called as such:
obj = LRUCache(capacity)
# obj._l = obj.get()
# obj._put(key, value)

#####

class Node:
    def __init__(self, key: int, value: int) ->
    None:
        self.key = key
        self.value = value
        self.freq = 1
```

```

self.prev = None
self.next = None

class DoublyLinkedList:
    def __init__(self) -> None:
        self.head = Node(-1, -1)
        self.tail = Node(-1, -1)
        self.head.next = self.tail
        self.tail.prev = self.head

    def add_first(self, node: Node) -> None:
        node.prev = self.head
        node.next = self.head.next
        self.head.next.prev = node
        self.head.next = node

    def remove(self, node: Node) -> None:
        node.next.prev = node.prev
        node.prev.next = node.next

```

```

node.next, node.prev = None, None
return None

def remove_last(self) -> Node:
    return self.remove(self.tail.prev)

def is_empty(self) -> bool:
    return self.head.next == self.tail

class LRUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.min_freq = 0
        self._nodes = defaultdict(Node)
        self.freq_map = defaultdict(list)
        self._head = self._tail = None

    def get(self, key: int) -> int:
        if self.capacity == 0 or key not in self._nodes:
            return -1

```

```

node = self.map[key]
self.incr_freq(node)
return node.value

def put(self, key: int, value: int) -> None:
    if self.capacity == 0:
        return
    if key in self.map:
        node = self.map[key]
        node.value = value
    else:
        self.incr_freq(node)
        return
    if len(self.map) == self.capacity:
        ls = self.freq_map[self.min_freq]
        node = ls.remove_last()
        self.map.pop(node.key)
    node = Node(key, value)
    self.add_node(node)
    self.map[key] = node
    self.min_freq = 1

```

```
def incr_freq(self, node: Node) -> None:
    freq = node.freq
    ls = self.freq_map[freq]
    ls.remove(node)
    if ls.is_empty():
        self.freq_map.pop(freq)
    if freq == self.min_freq:
        self.min_freq += 1
    node.freq += 1
    self.add_node(node)

def add_node(self, node: Node) -> None:
    freq = node.freq
    ls = self.freq_map[freq]
    ls.add_first(node)
    self.freq_map[freq] = ls

# Your LFUCache object will be instantiated
called as such:
```

Round 6 - Mid - Hard

Stack

Round 6 - Mid - Hard - 8 questions

Round 6 - Mid - Hard

Stack

#32 Longest Valid Parentheses

100%

LeetCode - SimplyEasT - WalkCC - LeetCode

String - Dynamic Programming - Stack

Like: 604 569

Time Complexity: $O(n)$ where n is the length of the string. Space Complexity: $O(n)$ for the stack data structure.

Problem

Given a string containing just the characters '(' and ')', return the length of the longest valid (well-formed) parentheses substring.

Example 1:

```
Input: s = "(())"
Output: 4
Explanation: The longest valid parentheses substring is "()".
```

```

Round 6 - Mid - Hard
#

Input: s = ""
Output: 0

Constraints:
0 <= s.length <= 3 * 104
s[i] is '(', or ')'.

Community Solutions (Python)

# WalkCC

class Solution:
    def longestValidParentheses(self, s: str) -> int:
        s2 = ')' + s
        dp[i] := the length of the longest valid
        parentheses in the substring
        s2[i-1] = i
        dp = [0] * len(s2)
        for i in range(1, len(s2)):
            if s2[i] == ')' and s2[i-1] == '(':
                dp[i] = dp[i-2] + 2
            elif s2[i] == ')' and s2[i-1] == ')':
                dp[i] = dp[i-2] + 2
            elif s2[i] == '(' and s2[i-1] == '(':
                dp[i] = dp[i-2] + 2
            else:
                dp[i] = 0
        return max(dp)

```

```

class Solution:
    def isValidParentheses(self, s: str) -> bool:
        n = len(s)
        if n % 2 != 0:
            return False
        if s == "":
            return True
        stack = []
        for i in range(n):
            if s[i] in "({[" and not stack:
                stack.append(s[i])
            elif s[i] in ")}]" and stack:
                if s[i] == "(" and stack[-1] == "[":
                    stack.pop()
                elif s[i] == "(" and stack[-1] == "(":
                    stack.pop()
                elif s[i] == "(" and stack[-1] == "{":
                    stack.pop()
                elif s[i] == ")" and stack[-1] == "(":
                    stack.pop()
                elif s[i] == ")" and stack[-1] == "[":
                    stack.pop()
                elif s[i] == ")" and stack[-1] == "{":
                    stack.pop()
                else:
                    return False
            else:
                stack.append(s[i])
        return len(stack) == 0

```

```

Round 6 - Mid - Hard
    stack.append(i)
    else:
        ans = max(ans, i - stack[-1])
    return ans

# Sample Input
# Python solution using a stack
def longestValidParentheses(s: str) -> int:
    # Initialize the stack with the base index -1
    stack = [-1]
    max_length = 0

    # Iterate over the string with index
    for i, char in enumerate(s):
        if char == '(':
            # Push the index of '(' onto the stack
            stack.append(i)
        else:
            # Pop the last index for ')'
            stack.pop()

    # The length of the longest valid parentheses is the
    # difference between the current index and the index
    # of the last element in the stack
    return max_length

```

```

Round 6 - Mid - Hard

current_length      max_length = max(max_len,
                                     current_length)

return max_length

# Example usage:
print(longestValidParentheses("(){}"))
4

1C6a

class Solution:
    def longestValidParentheses(self, s:
int:
        left = right = 0
        res = 0
        for c in s: # from left to right
            will never hit left=right
                if c == '(':
                    left += 1
                else:
                    right += 1

```

```
Round 6 - Mid - Hand
```

```
res = max(res, 2 * left)
if left > right:
    left = right = 0

left = right = 0 # don't forget to reset
for c in reversed(s): # from right to left,
    '()' == will never hit left=right
    if c == '(':
        left += 1
    else:
        right += 1

if left == right:
    res = max(res, 2 * left)
if left > right: # reverses '(' to ')'
    left = right = 0

return res
```

```

Round 6 - Mid - Hard
p:

def longestValidParentheses(self, s: str) ->
int:
    n = len(s)
    if n < 2:
        return 0
    dp = [0] * n
    for i in range(1, n):
        if s[i] == ')':
            if i - 1 == '(':
                dp[i] = 2 * (dp[i - 2] if i >
                else 0)
            else:
                j = i - dp[i - 1] - 1
                if j >= 0 and s[j] == '(':
                    dp[i] = 2 * dp[i - 1] +
                    dp[j - 1]
    return max(dp)
#####

```

```
Round G - Mid - Hard

1. #
2. #
3. #
4. #
5. #
6. #
7. #
[None, None, None, None, None, None, None]
>>>

class Solution:
    def LongestValidParentheses(self, s: str) ->
int:
    n = len(s)
    # dp[i] = (n + 1)
    for i, c in enumerate(s, 1): # starting i
        from 1, not 0
            if c == ')':
                if i > 1 and s[i - 2] == "(":
                    f[i] = f[i - 2] + 2
```

Round 6 - Mid - Hard

p=2

Stack

#42 Trapping Rain Water

Solve

LeetCode · SimplyLeet · WalkCC · LeetCode

Array · Two Pointers · Dynamic Programming · Stack · Monotonic Stack

Like: Grind 569

Time Complexity: O(n) Space Complexity: O(1) when using the two-pointer approach

Problem

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Example 1:

```
Round C - Mid - Hard
p.

Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]
Output: 6
Explanation: The above elevation map (black section) is
represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this
case, 6 units of rain water (blue section) are being
trapped.

Example 2:
Input: height = [4,2,0,3,2,5]
Output: 9

Constraints:
• n == height.length
• 1 <= n <= 2 * 104
• 0 <= height[i] <= 105

Brute Force (Python)

# Brute Force Approach
class Solution:
    def trap(self, height):
```

```

return res

Community Solutions (Python)
• WalkCC
class Solution:
    def trap(self, height: List[int]) -> int:
        n = len(height)
        l = [0] * n
        r = [0] * n
        for i, h in enumerate(height):
            l[i] = h if i > 0 else max(h, l[i-1])
            r[i] = h if i < n-1 else max(h, r[i+1])
        return sum(min(l[i], r[i]) - h for i, h in enumerate(height))

```

```

class Solution:
    def trap(self, height: list[int]) -> int:
        if not height:
            return 0

        ans = 0
        l = 0
        r = len(height) - 1
        maxi = height[l]
        maxR = height[r]

        while l < r:
            if maxi < maxR:
                ans += maxi - height[l]
                l += 1
                maxi = max(maxi, height[l])
            else:
                ans += maxR - height[r]
                r -= 1
                maxR = max(maxR, height[r])

```

```

class Solution:
    def trap(self, height: List[int]) -> int:
        n = len(height)
        left = [height[0]] * n
        right = [height[-1]] * n
        for i in range(1, n):
            left[i] = max(left[i - 1], height[i])
            right[n - i - 1] = max(right[n - i],
                                     height[n - i - 1])
        return sum(max(left, right) - h for l, r, h in
                    zip(left, right, height))

```

• **Simple Python**

• Python solution using the two pointers approach

```

def trap(height):
    # Initialize left and right pointers and water
    # accumulator
    left, right = 0, len(height) - 1
    water = 0

```

```

Round 6 - Mid - Hard
P

if height[left] < height[right]:
    # left side is the determining side
    if height[left] == left_max:
        left_max = height[left] # update
    left_max since current height is higher:
    else:
        water = left_max - height[left]
water trapped at left position
    left -= 1 # move left pointer to the
    right
    else:
        # right side is the determining side
        if height[right] == right_max:
            right_max = height[right] # update
        right_max since current height is higher
        else:
            water = right_max - height[right]
# water trapped at right position
    right -= 1 # move right pointer to the
    left

```

```
class Solution:
    def trap(self, height: List[int]) -> int:
        n = len(height)
        if n == 3:
            return 0

        l = height[0]
        r = height[n - 1]
        for i in range(1, n):
            l = max(l, height[i - 1])
            height[i] = min(l, height[i])

        self.compute(height, 0, n - 1)

        return sum(height)

    def compute(self, height, l, r):
        if l > r:
            return 0
        if l == r:
            return height[l]
        if l < r:
            m = (l + r) // 2
            height[m] = min(height[l], height[r])
            self.compute(height, l, m)
            self.compute(height, m, r)
```

```

height[i]

Round 6 - Mid - Hand

n:

left = height[0] = n
right = height[-1] = n
for i in range(1, n):
    left[i] = max(left[i - 1], height[i])
    right[i - 1] = max(right[n - i],
        height[n - i - 1])
    return sum(min(l, r) - h for l, r, h in
        zip(left, right, height))

#####

class Solution:
    def trap(self, height: List[int]) -> int:
        if not height:
            return 0

        total_water = 0
        max_index = 0

        for i in range(1, len(height)):
            if height[i] > height[max_index]:
                max_index = i

        for i in range(len(height) - 1, 0, -1):
            if height[i] > height[max_index]:
                max_index = i

        for i in range(1, len(height)):
            left_max = height[0]
            right_max = height[-1]

            if left_max < height[i]:
                left_max = height[i]

            if right_max < height[i]:
                right_max = height[i]

            total_water += min(left_max, right_max) - height[i]

        return total_water

```

```

        if height[i] > max_height:
            max_height = height[i]
            max_index = i

    # Calculate water trapped on the
    # of the maximum height
    left_max = height[0]
    for i in range(1, max_index):
        if left_max < height[i]:
            left_max = height[i]
        total_water += left_max - height[i]

    # Calculate water trapped on the
    # of the maximum height
    right_max = height[-1]
    for i in range(len(height) - 2,
-2):
        if right_max < height[i]:
            right_max = height[i]
        total_water += right_max - height[i]

```

Round 6 - Mid - Hard 2/21

Stack

#84 Largest Rectangle in Histogram Hard

leetDouce - SimplyLeeT - WalCC - LeeCode

Array - Stack - Monotonic Stack

Lists: Grid 169

Time Complexity: $O(n)$ Space Complexity: $O(n)$

Problem

Given an array of integers heights representing the histogram's bar height where the width of each bar is 1, return the area of the largest rectangle in the histogram.

Example 1:

Round 6 - Mid - Hard

Input: heights = [2,1,5,6,2,3]
Output: 10

Explanation: The above is a histogram where width of each bar is 1.
The largest rectangle is shown in the red area, which has an area = 10 units.

Example 2:



The diagram shows a histogram with six bars of width 1. The heights of the bars are 2, 1, 5, 6, 2, and 3. A red rectangle is drawn with a width of 2 units and a height of 5 units, spanning the third and fourth bars. The number '4' is written inside the red rectangle. Another red rectangle is shown below the fourth bar with a width of 1 unit and a height of 3 units, with the number '3' written inside it.

```

Round 6 - Mid - Hard

# 1 <= heights.length <= 105
# 0 <= heights[i] <= 104

Brute Force (Python)

# Brute Force Approach
class Solution:
    def largestRectangleArea(self, heights):
        # Brute Force (O(n^2)) - fix left bar, expand
        # right, track min height
        n = len(heights)
        res = 0
        for i in range(n):
            min_h = heights[i]
            for j in range(i, n):
                min_h = min(min_h, heights[j])
                res = max(res, min_h * (j - i + 1))
        return res

```

```
class Solution:
    def longestIncreasingSubsequence(self, heights: List[int]) -> int:
        ans = 0
        stack = []

        for i in range(len(heights) + 1):
            while stack and i == len(heights) or heights[stack[-1]] > heights[i]:
                stack.pop()
            w = i - stack[-1] - 1 if stack else i
            ans = max(ans, b + w)

            stack.append(i)

        return ans

# Test Cases
class Solution:
    def longestIncreasingSubsequence(self, heights: List[int]) -> int:
        ans = 0
        stack = []

        for i in range(len(heights) + 1):
            while stack and i == len(heights) or heights[stack[-1]] > heights[i]:
                stack.pop()
            w = i - stack[-1] - 1 if stack else i
            ans = max(ans, b + w)

            stack.append(i)

        return ans
```

```

if stk:
    left[i] = stk[-1]
    stk.append(i)
return max(h = (right[i] - left[i] - 1) *
i, h in enumerate(heights))

```

• **Complexity**

Python solution

```

def largestRectangleArea(heights):
    # Append a zero to flush out the remaining bars
    in the stack
    heights.append(0)
    stack = [] # Stack to store indices of bars
    max_area = 0

    # Iterate over each index and bar height
    for i, h in enumerate(heights):

```

```

# Process bars that are taller than
current bar
while stack and heights[stack[-1]]
    height = heights[stack.pop()]
of the bar at the index popped from the stack
# Calculate width: if stack
width is the current index; otherwise, its
difference
width = i if not stack else:
stack[-1] - 1
max_area = max(max_area, height
width)
stack.append(i)

return max_area

# Example usage:
if __name__ == '__main__':
    histograms = [2, 1, 5, 6, 2, 3]

```

```

...
##### h #####
right_i

##### n = (right_i - left_i - 1) #####

class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        n = len(heights)
        sth = []
        left = [-1] + n

        for i in range(n):
            while sth and heights[sth[-1]] >= heights[i]:
                sth.pop()
            sth.append(i)

```

```
Output: 4
Constraints:

Round 6 - Mid - Hard

p-

return max(h * (right[i] - left[i] - 2)) #
i, h in enumerate(heights)

#####

class Solution:
    def largestRectangleArea(self, heights:
    List[int]) -> int:
        if not heights:
            return 0

        heights.append(-1) # make for loop running
        stack = []
        ans = 0
        for i in range(0, len(heights)):
            while stack and heights[i] <
            heights[stack[-1]]:
                heights[stack[-1]]


```

```

Round 6 - Mid - Hard
#

right_i      # [0] [1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [19] [20] [21] [22] [23] [24] [25] [26] [27] [28] [29] [30] [31] [32] [33] [34] [35] [36] [37] [38] [39] [40] [41] [42] [43] [44] [45] [46] [47] [48] [49] [50] [51] [52] [53] [54] [55] [56] [57] [58] [59] [60] [61] [62] [63] [64] [65] [66] [67] [68] [69] [70] [71] [72] [73] [74] [75] [76] [77] [78] [79] [80] [81] [82] [83] [84] [85] [86] [87] [88] [89] [90] [91] [92] [93] [94] [95] [96] [97] [98] [99] [100] [101] [102] [103] [104] [105] [106] [107] [108] [109] [110] [111] [112] [113] [114] [115] [116] [117] [118] [119] [120] [121] [122] [123] [124] [125] [126] [127] [128] [129] [130] [131] [132] [133] [134] [135] [136] [137] [138] [139] [140] [141] [142] [143] [144] [145] [146] [147] [148] [149] [150] [151] [152] [153] [154] [155] [156] [157] [158] [159] [160] [161] [162] [163] [164] [165] [166] [167] [168] [169] [170] [171] [172] [173] [174] [175] [176] [177] [178] [179] [180] [181] [182] [183] [184] [185] [186] [187] [188] [189] [190] [191] [192] [193] [194] [195] [196] [197] [198] [199] [200] [201] [202] [203] [204] [205] [206] [207] [208] [209] [210] [211] [212] [213] [214] [215] [216] [217] [218] [219] [220] [221] [222] [223] [224] [225] [226] [227] [228] [229] [230] [231] [232] [233] [234] [235] [236] [237] [238] [239] [240] [241] [242] [243] [244] [245] [246] [247] [248] [249] [250] [251] [252] [253] [254] [255] [256] [257] [258] [259] [260] [261] [262] [263] [264] [265] [266] [267] [268] [269] [270] [271] [272] [273] [274] [275] [276] [277] [278] [279] [280] [281] [282] [283] [284] [285] [286] [287] [288] [289] [290] [291] [292] [293] [294] [295] [296] [297] [298] [299] [300] [301] [302] [303] [304] [305] [306] [307] [308] [309] [310] [311] [312] [313] [314] [315] [316] [317] [318] [319] [320] [321] [322] [323] [324] [325] [326] [327] [328] [329] [330] [331] [332] [333] [334] [335] [336] [337] [338] [339] [340] [341] [342] [343] [344] [345] [346] [347] [348] [349] [350] [351] [352] [353] [354] [355] [356] [357] [358] [359] [360] [361] [362] [363] [364] [365] [366] [367] [368] [369] [370] [371] [372] [373] [374] [375] [376] [377] [378] [379] [380] [381] [382] [383] [384] [385] [386] [387] [388] [389] [390] [391] [392] [393] [394] [395] [396] [397] [398] [399] [400] [401] [402] [403] [404] [405] [406] [407] [408] [409] [410] [411] [412] [413] [414] [415] [416] [417] [418] [419] [420] [421] [422] [423] [424] [425] [426] [427] [428] [429] [430] [431] [432] [433] [434] [435] [436] [437] [438] [439] [440] [441] [442] [443] [444] [445] [446] [447] [448] [449] [450] [451] [452] [453] [454] [455] [456] [457] [458] [459] [460] [461] [462] [463] [464] [465] [466] [467] [468] [469] [470] [471] [472] [473] [474] [475] [476] [477] [478] [479] [480] [481] [482] [483] [484] [485] [486] [487] [488] [489] [490] [491] [492] [493] [494] [495] [496] [497] [498] [499] [500] [501] [502] [503] [504] [505] [506] [507] [508] [509] [510] [511] [512] [513] [514] [515] [516] [517] [518] [519] [520] [521] [522] [523] [524] [525] [526] [527] [528] [529] [530] [531] [532] [533] [534] [535] [536] [537] [538] [539] [540] [541] [542] [543] [544] [545] [546] [547] [548] [549] [550] [551] [552] [553] [554] [555] [556] [557] [558] [559] [560] [561] [562] [563] [564] [565] [566] [567] [568] [569] [570] [571] [572] [573] [574] [575] [576] [577] [578] [579] [580] [581] [582] [583] [584] [585] [586] [587] [588] [589] [590] [591] [592] [593] [594] [595] [596] [597] [598] [599] [600] [601] [602] [603] [604] [605] [606] [607] [608] [609] [610] [611] [612] [613] [614] [615] [616] [617] [618] [619] [620] [621] [622] [623] [624] [625] [626] [627] [628] [629] [630] [631] [632] [633] [634] [635] [636] [637] [638] [639] [640] [641] [642] [643] [644] [645] [646] [647] [648] [649] [650] [651] [652] [653] [654] [655] [656] [657] [658] [659] [660] [661] [662] [663] [664] [665] [666] [667] [668] [669] [670] [671] [672] [673] [674] [675] [676] [677] [678] [679] [680] [681] [682] [683] [684] [685] [686] [687] [688] [689] [690] [691] [692] [693] [694] [695] [696] [697] [698] [699] [700] [701] [702] [703] [704] [705] [706] [707] [708] [709] [710] [711] [712] [713] [714] [715] [716] [717] [718] [719] [720] [721] [722] [723] [724] [725] [726] [727] [728] [729] [730] [731] [732] [733] [734] [735] [736] [737] [738] [739] [740] [741] [742] [743] [744] [745] [746] [747] [748] [749] [750] [751] [752] [753] [754] [755] [756] [757] [758] [759] [760] [761] [762] [763] [764] [765] [766] [767] [768] [769] [770] [771] [772] [773] [774] [775] [776] [777] [778] [779] [780] [781] [782] [783] [784] [785] [786] [787] [788] [789] [790] [791] [792] [793] [794] [795] [796] [797] [798] [799] [800] [801] [802] [803] [804] [805] [806] [807] [808] [809] [810] [811] [812] [813] [814] [815] [816] [817] [818] [819] [820] [821] [822] [823] [824] [825] [826] [827] [828] [829] [830] [831] [832] [833] [834] [835] [836] [837
```


Sheet 70/87 · LetterMastery Study-Order · 36-up Landscape · 6×6 · Round G · Mid · Hard · p.2518

```

j += 1

while i < len(left):
    self._addPoint(anns, left[i][0], left[i][2])
    i += 1

while j < len(right):
    self._addPoint(anns, right[j][0],
right[j][2])
    j += 1

return anns

def _addPoint(self, anns: list[list[int]], x:
int, y: int) -> None:
    if anns and anns[-1][0] == x:
        anns[-1][2] = y
    return
    if anns and anns[-1][0] == y:
        return
    anns.append([x, y])

```

• SimplyList

```
import heapq

def getTimeline(buildings):
    # List to store all events: (x, height, right)
    events = []

    # Create events for all buildings: left end (start) and right end (end)
    for left, right, height in buildings:
        # For the start point, height is negative for proper sorting (start events come before s events)
        events.append((left, -height, right))

        # For the end point, height is 0 to indicate removal of building from active heap
        events.append((right, 0, 0))

    # Sort events by a coordinate.
    events.sort()

    # Max heap to store active buildings (using negative heights)
    result = []

    # Initialize the heap with a dummy building
    height = 0
    heap = [(0, float('inf'))]
```

```

for x, seq_height, right in events:
    # Remove buildings from the heap that have
    # ended by current x.
    while heap and heap[0][1] <= x:
        heap.heappop(heap)

    # If event is a start event (seq_height, R),
    # push it into the heap.
    if seq_height == 0:
        heap.heappush(heap, (seq_height,
                               right))

    # Current highest building height is at the
    # top of the heap.
    current_height = -heap[0][0]

    # If current height is empty or current height
    # differs from last recorded height, add a new key
    # point.
    if not result or current_height !=
        result[-1][2]:
        result.append([x, current_height])

return result

```

```
# Example usage:
buildings =
[[2,9,10],[3,7,15],[5,12,12],[15,20,10],[19,24,8]]
print(getSkyline(buildings)) # Expected output: [[
2,10],[3,15],[7,12],[12,0],[15,10],[20,0],[24,0]]
```

#272 Closest Binary Search Tree

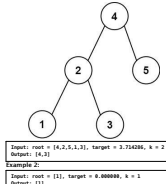
LeetCode - SimplyElastic - WalkCr - LeetCode
Two Pointers - Stack - Tree - DFS - BST - Heap
Likes: Premium 96

Time Complexity: $O(k + \log n)$ - $O(\log n)$ to initialize the two stacks and $O(k)$ to collect k elements. **Space Complexity:** $O(\log n)$ - due to the stack space in a balanced BST.

Problem:
Given the root of a binary search tree, a target value, and an integer k , return the k values in the BST that are closest to the target. You may return the answer in any order.

Hint:
You are guaranteed to have only one unique set of k values in the BST that are closest to the target.

Example 1:



Constraints:

- The number of nodes in the tree is n .
- $1 \leq k \leq m \leq 104$.
- $0 \leq \text{Node.val} \leq 109$
- $-109 \leq \text{target} \leq 109$

Follow up: Assume that the BST is balanced. Could you solve it in less than $O(n)$ runtime (where n = total nodes)?

Brute Force (Python)

```
# Brute Force Approach
class Solution:
    def closestValues(self, root: TreeNode, target: float, k: int):
        # Brute Force  $O(n \log n)$  - sort repeatedly
        instead of heap
        data = list(root)
        result = []
        for i in range(k):
            data.sort()
            result.append(data.pop(0))
        return result
```

Community Solutions (Python)

```
class Solution:
    def closestValues(
        self,
        root: TreeNode | None,
        target: float,
        k: int,
    ) -> List[int]:
        dq = collections.deque()

        def inorder(root: TreeNode | None) -> None:
            if not root:
                return

            inorder(root.left)
            dq.append(root.val)
            inorder(root.right)

        inorder(root)

        while len(dq) > k:
            if abs(dq[0] - target) > abs(dq[-1] - target):
                dq.popleft()
            else:
                dq.pop()

        return list(dq)

# LeetCode
```

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
class Solution:
    def insertIntoBST(self, root: TreeNode,
        target: float, k: int) -> List[int]:
        def dfs(root):
            if root is None:
                return
            dfs(root.left)
            if len(q) < k:
                q.append(root.val)
            else:
                if abs(root.val - target) >=
abs(q[0] - target):
                    return
                q.popLeft()
                q.append(root.val)
            dfs(root.right)

        q = deque()
        dfs(root)
        return list(q)

```

```
# Python solution for Classset Binary Search Tree
# Waleed II

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def constructMaximumBinaryTree(self, nums: List[int]) -> TreeNode:
        # Base Case
        if not nums:
            return None

        # Find the maximum value in the array
        max_val = max(nums)
        max_index = nums.index(max_val)

        # Create the root node
        root = TreeNode(max_val)

        # Recursively construct the left and right subtrees
        root.left = self.constructMaximumBinaryTree(nums[:max_index])
        root.right = self.constructMaximumBinaryTree(nums[max_index+1:])

        return root
```

```
# Helper function to initialize successors
stack
def initSuccessors(node):
    while node:
        # If node value is greater than
        target, push to succs and go left
        if node.val > target:
            succs.append(node)
            node = node.left
        else:
            node = node.right

# Helper function to get next predecessor
val = None
def getPredecessor():
    if not prede: return None
    node = prede.pop()
    val = node.val
    # Traverse the left subtree: push nodes
    from left child then rightmost branch
    node = node.left
    while node:
        prede.append(node)
```

```

        node = node.right
        return val

# Helper function to get next successor
value
def getNextSuccessor():
    if not succn: return None
    node = succn.pop()
    val = node.val
    # Traverse the right subtree: push
nodes from right child then leftmost branch
    node = node.right

while node:
    succn.append(node)
    node = node.left
    return val

preds, succs = [], []
# Initialize the stacks with correct
boundary
initPredecessor(root)
initSuccessor(root)

```

```

result = []
# If the closest value from predecessor and
# successor are same, skip one to avoid duplicates.
    if preds[-1].val == succs[-1].val:
        succs = succs[1:].val:
        getPredecessor()

# Merge k closest values
for _ in range(k):
    if not preds:
        # Only successors left
        result.append(getSuccessor())
    elif not succs:
        # Only predecessors left
        result.append(getPredecessor())
    else:
        # Compare which candidate is
        # closer to the target
        if abs(preds[-1].val - target) <=
abs(succs[-1].val - target):
            result.append(getPredecessor())
        else:
            result.append(getSuccessor())
return result

```

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def convertBstToDll(self, root: TreeNode) -> List[int]:
        target: float = k - int -> list[int]:
            def dfs(node):
                if node is None:
                    return
                dfs(node.left)
                if len(q) < k:
                    q.append(node.val)
                else:
                    if abs(node.val - target) >=
                        abs(q[-1] - target):
                        return
                    q.pop()
                    q.append(node.val)
                dfs(node.right)
            dfs(root)
            return q
```

```

q = deque()
dfs(root)
return list(q)

#####

from collections import deque

# iteration

class Solution:
    def climbStairs(self, root: TreeNode,
target: int, k: int) -> List[int]:
    stack = []
    q = deque()

    # Iterative in-order traversal
    while stack or root:
        while root:
            stack.append(root)
            root = root.left

```

```

root = stack.pop()

# Process current node
if len(q) < k:
    q.append(root.val)
else:
    if abs(root.val - target) <
abs(q[0] - target):
        q.popleft()
        q.append(root.val)

    else:
        break
# Early stop if the current
value is farther than the first value in queue

root = root.right

return list(q)

```

#295 Find Median from Data Stream

LeetCode · Simply Test · WalkCC · LeetCode
Two Pointers · Design · Sorting · Heap
Time: O(n log n)
Time Complexity:
Problem

The median is the middle value in an ordered integer list. If the size of the list is even, there is no middle value, and the median is the mean of the two middle values.

- For example, for $arr = [2, 3, 4]$, the median is 3.
- For example, for $arr = [2, 3]$, the median is $(2 + 3) / 2 = 2.5$.

Implement the MedianFinder class:

- MedianFinder() initializes the MedianFinder object.
- void addNum(int num) adds the integer num to the data stream to the data structure.
- double findMedian() returns the median of all elements so far. Answers within 10^{-5} of the actual answer will be accepted.

Example 1:

```

Input
["MedianFinder", "addNum", "addNum", "findMedian",
 "delete", "findMedian"]
[[1], [2], [1], [2], [1]]
Output
[mull, mull, mull, 1.5, mull, 2.0]

Explanation
MedianFinder medianFinder = new MedianFinder();
// ...

Constraints
• 105 <= num <= 105
• There will be at least one element in the data structure before calling findMedian.
• At most 5 * 104 calls will be made to addNum and findMedian.

Follow up:
• If all integer numbers from the stream are in the range [0, 100], how would you optimize your solution?
• If 99% of all integer numbers from the stream are in the range [0, 100], how would you optimize your solution?

Community Solutions (Python)
• WalkCC

```

```

class MedianFinder:
    def __init__(self):
        self._minheap = []
        self._maxheap = []

    def addNum(self, num: int) -> None:
        if not self._minheap or num <= -self._minheap[0]:
            heapq.heappush(self._minheap, -num)
        else:
            heapq.heappush(self._minheap, num)

    # Balance the two heaps s.t.
    # |minheap| == |maxheap| and |minheap| -
    # |maxheap| <= 1.
    def _balance(self):
        if len(self._minheap) < len(self._maxheap):
            heapq.heappush(self._minheap,
                           -heapq.heappop(self._maxheap))
        elif len(self._minheap) - len(self._maxheap) > 1:
            heapq.heappush(self._maxheap,
                           -heapq.heappop(self._minheap))

    def findMedian(self) -> float:
        if len(self._minheap) == len(self._minheap):
            return -(self._minheap[0] + self._maxheap[0]) / 2.0
        else:
            return -self._minheap[0]

```

• LeetCode

```

class MedianFinder:

    def __init__(self):
        self.nums = []
        self.mnq = []

    def addNum(self, num: int) -> None:
        heappush(self.nsq, num)
        -heappushpop(self.nsq, -num)
        if len(self.nsq) - len(self.mnq) > 1:
            heappush(self.mnq, -heappop(self.nsq))

    def findMedian(self) -> float:
        if len(self.nsq) == len(self.mnq):
            return (self.nsq[0] - self.mnq[0]) / 2
        return self.nsq[0]

# Your MedianFinder object will be instantiated and called as such:
# obj = MedianFinder()
# obj.addNum(num)
# param_2 = obj.findMedian()

```

• SimplyList

```
import heapq

class MedianFinder:
    def __init__(self):
        # max-heap for the lower half (store
        # negatives to simulate max-heap)
        self.lowerHalf = []
        # min-heap for the upper half
        self.upperHalf = []

    def addNum(self, num: int) -> None:
        # Push to max-heap (simulate by pushing
        # negative)
        heapq.heappush(self.lowerHalf, -num)
        # Ensure every element in lowerHalf is
        # always smaller in upperHalf
        # Move maximum of lowerHalf to upperHalf
        heapq.heappush(self.upperHalf,
            -heapq.heappop(self.lowerHalf))

        # Balance the heaps: if upperHalf exceeds
        # lowerHalf in size
        if len(self.upperHalf) >
            len(self.lowerHalf):
            heapq.heappush(self.lowerHalf,
                -heapq.heappop(self.upperHalf))
```

```
def findMedian(self) -> float:
    # If both heaps are balanced, the median is
    # average of both heaps' top elements
    if len(self.LowerHalf) ==
len(self.UpperHalf):
        return -(self.LowerHalf[0] +
self.UpperHalf[0]) / 2.0
    # Otherwise, median is the top element of
    # the Larger heap.
    return -self.LowerHalf[0]

# Example usage:
# medianFinder = MedianFinder()
# medianFinder.addNum(1)

# medianFinder.addNum(2)
# print(medianFinder.findMedian()) # Outputs 1.5
# medianFinder.addNum(3)
# print(medianFinder.findMedian()) # Outputs 2.0
```

```
from heapq import heappush, heappop

class MedianFinder:
    def __init__(self):
        # s is the smaller half of the list, max heap
        # (invert min-heap)
        self.sma = []
        # l is the larger half of the list, min heap
        self.sml = []

    def addNum(self, num: int) -> None:
        # trick for python does not have comparator like in Java
        # heappush is smaller does NOT have comparator like in Java
        heappush(self.sma, -num)
        heappush(self.sml, num)
        # heappop(self.sma) # note: not
        # self.sma[-1]

    if len(self.sma) > len(self.sml):
        heappop(self.sml, self.sml)
        heappop(self.sma)
    elif len(self.sml) > len(self.sma):
        heappop(self.sml, self.sml)
        heappop(self.sma)
```

```
def findMedian(self) -> float:
    if len(self.smallHalf) == len(self.largeHalf):
        return -(self.smallHalf[0] + self.largeHalf[0]) / 2.0
    else:
        return float(-self.smallHalf[0])

# Your MedianFinder object will be instantiated
called as such:
obj = MedianFinder()
obj.addNum(num)

# param_1 = obj.findMedian()

# Follow up
class MedianFinder:
    def __init__(self):
        self.counts = [0] * 101
        self.total = 0

    def addNum(self, num: int) -> None:
```

```
self.counts[num] += 1
self.total += 1

def findMedian(self) -> float:
    if self.total % 2 == 0:
        # even number of elements
        mid1 = self.total // 2
        mid1_2 = mid1 + 1
        count = 0
        i1 = 0
        for i in range(101):
            count = self.counts[i]
            if count == mid1 and i1 == 0:
                i1 = 1
            if count == mid1_2:
                i2 = i
                break
        return (i1 + i2) / 2
    else:
        # odd number of elements
```

```

        middle = self.total // 2 + 1
        count = 0
        for i in range(101):
            count += self.counts[i]
            if count == middle:
                return i

#####

class MedianFinder:
    def __init__(self):
        """
        initialize your data structure here.
        """
        self.h1 = []
        self.h2 = []

    def addNum(self, num: int) -> None:
        heappush(self.h1, num)

```

```

heapqpush(self.h2, -heapqpop(self.h2))
if len(self.h2) - len(self.h1) > 1:
    heapqpush(self.h1, -heapqpop(self.h2))

def findMedian(self) -> float:
    if len(self.h2) > len(self.h1):
        return -self.h2[0]
    return (self.h1[0] - self.h2[0]) / 2

# Your MedianFinder object will be instantiated
called as such:
# obj = MedianFinder()
# obj.addNum(num)
# param_2 = obj.findMedian()

```

#358 Rearrange String k Distance Apart

LeetCode · [SimpleLeetCode](#) · [Walk-EC](#) · [LeetCode](#)

Hash Table · String · Greedy · Sorting · Heap · Counting List · Prefixsum

Time Complexity: $O(n \log n)$ (where n is the length of s)
 C is the number of unique characters. Space Complexity: $O(n)$ due to the storage used for the heap and map queue.

Problem

Given a string s and an integer k , rearrange s such that the same characters are at least distance k from each other. If it is not possible to rearrange the string, return an empty string.

Example 1:

```
Input: s = "abcabc", k = 3
Output: "abcabc"
Explanation: The same letters are at least a distance of 3 from each other.
```

```
Input: s = "aabcd", k = 3
Output: ""
Explanation: It is not possible to rearrange the string.

Example 1:
Input: s = "aabbcd", k = 2
Output: "baadcb"
Explanation: The same letters are at least k distance of 2 from each other.

Constraints:
• 1 <= k <= length <= 2 * 10^5
• s consists of only lowercase English letters.
• 0 <= k <= length

Community Solutions (Python)

• WuJCC

class Solution:
    def rearrangeString(self, s: str, k: int) -> str:
        n = len(s)
        ans = []
        count = collections.Counter(s)
        # valid[i] -> the leftmost index i can appear
        valid = collections.Counter()
        def get_nextLetter(index: int) -> str:
            """returns the valid letter that has the most
            count."""
```

```

maxCount = 1
bestLetter = ''

for c in string.ascii_lowercase:
    if count[c] > 0 and count[c] > maxCount:
        index = valid[c]:
            bestLetter = c
            maxCount = count[c]

return bestLetter

for i in range(n):
    c = getBestLetter(i)
    if c == '':
        return ''
    ans.append(c)
    count[c] = 1
    valid[c] = i + k

return ''.join(ans)

```

```
class Solution:
    def rearrangeString(self, s: str, k: int) -> str:
        cnt = Counter(s)
        pq = [(-v, c) for c, v in cnt.items()]
        heapify(pq)
        ans = []
        while pq:
            v, c = heappop(pq)
            ans.append(c)
            q.append((v + 1, c))
            if len(q) >= k:
                e = q.popleft()
                if e[0]:
                    heappush(pq, e)
        return "" if len(ans) < len(s) else "".join(ans)
```

```

from collections import Counter, deque
import heapq

def rearrangeString(s: str, k: int) -> str:
    if k == 0:
        return s
    # No rearrangement needed if k is 0
    # Count the frequency of each character
    char_count = Counter(s)

    # Build a max-heap using negative frequencies
    max_heap = [(-freq, ch) for ch, freq in char_count.items()]
    heapq.heapify(max_heap)

    # Queue to keep track of characters waiting for k distance
    wait_queue = deque()
    result = []

    # Process the heap until it's empty
    while max_heap:

```

```

freq, ch = heapq.heappop(max_heap)
result.append(ch)

# Decrease frequency (remember freq is
negative)
freq += 1

# Add current char to the wait queue
wait_queue.append((freq, ch))

# If we've placed k characters, we can
re-insert the oldest one if needed
if len(wait_queue) < k:
    continue

freq, front, ch_front =
wait_queue.popleft()
if freq < 0:
    heapq.heappush(max_heap, (freq, front))

rearranged = ''.join(result)
return rearranged if len(rearranged) == len(s)
else:
    ''

```

```
from heapq import heappify

class Solution:
    def rearrangeCharacters(self, s: str, k: int) -> str:
        h = [(w, c) for c, w in Counter(s.items()).most_common(k)]
        w, c = deque()
        n = len(s)
        while h:
            w, c = heappop(h)

            w += 1
            c.append((w - 1, c)) if c[0] == w else c.append((w, c))

            if not 'a' <= c[0][1] <= 'z':
                w, c = heappop(h)
                w += 1
                c.append((w - 1, c)) if c[0] == w else c.append((w, c))

            if len(c) > k:
                # Queue this character in the next k-length.
                w, c = heappop(h)
                w += 1
                c.append((w - 1, c)) if c[0] == w else c.append((w, c))

            return "" if len(c) == len(s) else ""
```

```
#####

import collections
import heapq

class Solution:
    def rearrangeString(self, s: str, k: int) -> str:
        if k == 0:
            return s

        counts = collections.Counter(s)

        pq = []
        for ch, count in counts.items():
            heapq.heappush(pq, (-count, ch))

        sb = []
        while pq:
            top = []
```

```
d = min(k, len(s))
for i in range(d):
    if not pq:
        return ""
    count, ch = heapq.heappop(pq)
    sb.append(ch)
    if count < -1: # pushed in as
negated, so it's: if actual-positive-count > 1
        tmp.append((count+1, ch))
    s = s[1:]

for count, ch in tmp:
    heapq.heappush(pq, (count, ch))

return "".join(sb)
```

#499 The Maze III

LEADS: `Simplex4L`, `WallC`, `LC`, `LCe`
Array: `DFS`, `DFS`, `Graph`, `Matrix`, `Heap`, `Shortest Path`
Link: `Primem` `SD`
Time Complexity: $O(m \log(m))$ where m is the maze dimensions
Space Complexity: $O(m)$ where m is the maze dimensions
Problem

There is a maze in a maze with empty spaces (represented by 0) and walls (represented by 1). The ball can go through the empty spaces by rolling up, down, left or right, but it won't stop rolling until hitting a wall. When the ball stops, it could choose the next direction (must be different from last chosen direction). There is also a hole in this maze. The ball will drop into the hole if it rolls onto the hole.

Given the $m \times n$ maze, the ball's position ball and the hole hole, return the shortest distance to drop in the hole with the shortest distance possible. If there are no valid routes, return the lexicographically minimum one. If the ball can't drop in the hole, return -1.

Sheet 74/87 · LostMastery Study-Order · 36-up Landscape · 6x6 · Round 6 · Mid · Hard

<pre> queue = deque(beginIndex) length = 1 while queue: for k in range(1, len(queue)): top = queue.popleft() for dir in getDirList(beginIndex, distance): if dir not in distance: distance[beginIndex + distance[top] + 1] queue.append(dir) return distance def dfs(beginIndex, endIndex, wordList, path, res, distance): if beginIndex == endIndex: res.append(path + []) return for dir in getDir(beginIndex, endIndex, distance, getDir, -2) + 1 == distance[beginIndex]: path.append(dir) </pre>	<pre> distance = {} for dir, endIndex, wordList, path, res, distance): res = [] distance = {} for dir in getDir(beginIndex, endIndex, distance): if dir not in distance: distance[beginIndex + distance[dir] + 1] queue.append(dir) return distance def dfs(beginIndex, endIndex, wordList, path, res, distance): if beginIndex == endIndex: res.append(path + []) return for dir in getDir(beginIndex, endIndex, distance, getDir, -2) + 1 == distance[beginIndex]: path.append(dir) </pre>	#996 Number of Squared Arrays LeetCode - Simple - 90% AC Array - Math - Dynamic Programming - Backtracking Time Complexity: $O(n^2 \cdot 2^n)$ is the worst-case time complexity for this problem. Since we have n elements and checking pairwise sums, Space Complexity: $O(n)$ for recursion and $O(1)$ for the visited $\{$ Problem An array is squared if the sum of every pair of adjacent elements is a perfect square. Given n integers, return the number of combinations of numbers that are squared. Two permutations, perm1 and perm2 are different if there is some index i such that $perm1[i] \neq perm2[i]$. Example 1: Input: $[1, 2, 1, 1]$ Output: 3 Explanation: $[1, 2, 1, 1]$ and $[1, 1, 2, 1]$ are the valid permutations. Example 2:	<pre> Input: nums = [2,2,2] Output: 1 Constraints: 1 <= nums.length <= 10 0 <= nums[i] <= 100 Brute Force (Python) # Brute Force Approach def isSquared(self, nums): # Brute Force - exhaustive backtracking def backtrack(index, path): if index == len(nums): res.append(list(path)) return for i in range(index + 1, len(nums)): if nums[i] % 2 == 0: path.append(nums[i]) backtrack(i + 1, path) else: path.append(nums[i]) backtrack(i + 1, path) backtrack(0, []) return res </pre> Community Solutions (Python) • walrus	<pre> class Solution: def numSquares(self, n: int) -> int: dp = {} def dfs(n): if n == 0: return 0 if n in dp: return dp[n] min_val = float('inf') for i in range(1, int(n**0.5) + 1): min_val = min(min_val, 1 + dfs(n - i*i)) dp[n] = min_val return dfs(n) </pre>	<pre> if i < 0 and num[i] < num[i - 1] and not continue num[i] = True num[i + 1] = False num.sort() dfs(i + 1) return num </pre> #1000 Number of Squared Arrays LeetCode - Simple - 90% AC Array - Math - Dynamic Programming - Backtracking Time Complexity: $O(n^2 \cdot 2^n)$ is the worst-case time complexity for this problem. Since we have n elements and checking pairwise sums, Space Complexity: $O(n)$ for recursion and $O(1)$ for the visited $\{$ Problem An array is squared if the sum of every pair of adjacent elements is a perfect square. Given n integers, return the number of combinations of numbers that are squared. Two permutations, perm1 and perm2 are different if there is some index i such that $perm1[i] \neq perm2[i]$. Example 1: Input: $[1, 2, 1, 1]$ Output: 3 Explanation: $[1, 2, 1, 1]$ and $[1, 1, 2, 1]$ are the valid permutations. Example 2:
<pre> if i < 0 and num[i] < num[i - 1] and not continue num[i] = True num[i + 1] = False num.sort() dfs(i + 1) return num </pre>	<pre> visited = [False] * len(nums) def dfs(index, path): if index == len(nums): res.append(path + []) return for i in range(index + 1, len(nums)): if not visited[i]: path.append(nums[i]) dfs(i + 1, path) path.pop() visited[i] = False </pre>	<pre> dfs(i + 1, path) path.pop() visited[i] = False </pre>	<pre> if i < 0 and num[i] < num[i - 1] and not continue num[i] = True num[i + 1] = False num.sort() dfs(i + 1) return num </pre>	<pre> if i < 0 and num[i] < num[i - 1] and not continue num[i] = True num[i + 1] = False num.sort() dfs(i + 1) return num </pre>	<pre> if i < 0 and num[i] < num[i - 1] and not continue num[i] = True num[i + 1] = False num.sort() dfs(i + 1) return num </pre>
adjusting the next pointers. - Reconnect the reversed group with the previous part of the list and update pointers to move to the next group. The approach averages a dummy node for order to handle head modifications and uses a helper function to track the list ends. The in-place reversal ensures the extra memory usage is constant. #432 Odd Clone Data Structure LeetCode - Medium - 70% AC Key Insights • Use a doubly-linked list (DLL) where each node holds a unique count value and a list of keys with that count. • Maintain a hash table to quickly locate the count value corresponding to each key. • Insert new nodes in the DLL when a key's count is incremented or decremented and the associated count does not have the target count. • Remove nodes from the DLL when no keys remain in that count. • The head of the DLL holds the minimum count and the tail holds the maximum count. Space & Time Time Complexity: $O(n)$ average for insert, delete, get, and search. Space Complexity: $O(n)$ where n is the number of unique keys in the data structure.	visited = [False] * len(nums) def dfs(index, path): if index == len(nums): res.append(path + []) return for i in range(index + 1, len(nums)): if not visited[i]: path.append(nums[i]) dfs(i + 1, path) path.pop() visited[i] = False	<pre> dfs(i + 1, path) path.pop() visited[i] = False </pre>	<pre> if i < 0 and num[i] < num[i - 1] and not continue num[i] = True num[i + 1] = False num.sort() dfs(i + 1) return num </pre>	<pre> if i < 0 and num[i] < num[i - 1] and not continue num[i] = True num[i + 1] = False num.sort() dfs(i + 1) return num </pre>	<pre> if i < 0 and num[i] < num[i - 1] and not continue num[i] = True num[i + 1] = False num.sort() dfs(i + 1) return num </pre>
effectively finds the maximum remaining space possible. The value 0 is added at the end of the histogram to flush any remaining bars from the stack. #224 Basic Calculator LeetCode - Medium - 70% AC Key Insights • Use a stack to save previous results and signs when encountering parentheses. • Process the string left-to-right by accumulating multi-digit numbers and applying the current sign. • When encountering a ')', push the current result and sign onto the stack and reset them. • When encountering a '(', complete the current sub-expression first, pop the previous state to incorporate the completed value. • Handle whitespace by simply skipping them. Space & Time Time Complexity: $O(n)$ where n is the length of the input string, since we process each character once. Space Complexity: $O(n)$ in the worst-case scenario due to the use of a stack for nested parentheses. Solution The solution uses an iterative approach with a stack: - Initialize variables: result to hold the running total, sign to handle addition/subtraction, and index to iterate over the string. - As you iterate, build multi-digit numbers by checking for consecutive numerical characters. - Adjust the total by adding the product of the current number and its sign. When encountering a ')', push the current result and sign onto the stack, then reset result and sign for the new sub-expression. - When encountering a '(', complete the sub-expression by popping the previous result and sign, then combine. - Skip over spaces. - The final result is the completed value after the end of the string. Space Complexity: $O(n)$ where n is the number of elements in the stack. Time Complexity: $O(n)$ where n is the number of elements in the stack.	visited = [False] * len(nums) def dfs(index, path): if index == len(nums): res.append(path + []) return for i in range(index + 1, len(nums)): if not visited[i]: path.append(nums[i]) dfs(i + 1, path) path.pop() visited[i] = False	<pre> dfs(i + 1, path) path.pop() visited[i] = False </pre>	<pre> if i < 0 and num[i] < num[i - 1] and not continue num[i] = True num[i + 1] = False num.sort() dfs(i + 1) return num </pre>	<pre> if i < 0 and num[i] < num[i - 1] and not continue num[i] = True num[i + 1] = False num.sort() dfs(i + 1) return num </pre>	<pre> if i < 0 and num[i] < num[i - 1] and not continue num[i] = True num[i + 1] = False num.sort() dfs(i + 1) return num </pre>
When popping an element, remove i from the stack corresponding to the maximum frequency and update the frequency count if that stack becomes empty. Decrease the current element's frequency. #238 Merge k Sorted Lists LeetCode - Medium - 70% AC Key Insights • Use a min-heap (priority queue) to efficiently extract the smallest value among the current heads of the lists. • Push the head of each non-empty list into the heap, we can pop the smallest node, then push that node's next element into the heap. • A careful merge and compare approach can be used by repeatedly merging pairs of lists. • Alternatively, maintain a min-heap of pointers (or references) to key nodes. Space & Time Time Complexity: $O(N \log k)$ where N is the total number of nodes across all lists and k is the number of lists. Space Complexity: $O(k)$ due to the heap containing at most k nodes from each list. Solution Use a min-heap to always retrieve the smallest node among the current nodes in the linked lists. First, push the head of each non-empty list into the heap. Then, continuously extract the smallest node from the heap, append it to the result list, and push its next node into the heap. Repeat until the heap is empty.	visited = [False] * len(nums) def dfs(index, path): if index == len(nums): res.append(path + []) return for i in range(index + 1, len(nums)): if not visited[i]: path.append(nums[i]) dfs(i + 1, path) path.pop() visited[i] = False	<pre> dfs(i + 1, path) path.pop() visited[i] = False </pre>	<pre> if i < 0 and num[i] < num[i - 1] and not continue num[i] = True num[i + 1] = False num.sort() dfs(i + 1) return num </pre>	<pre> if i < 0 and num[i] < num[i - 1] and not continue num[i] = True num[i + 1] = False num.sort() dfs(i + 1) return num </pre>	<pre> if i < 0 and num[i] < num[i - 1] and not continue num[i] = True num[i + 1] = False num.sort() dfs(i + 1) return num </pre>
heap has more elements, the median is the top element of the heap. This method makes it efficient to dynamically update and retrieve the median as new numbers are inserted. #558 Rearrange String to Distance Apart LeetCode - Medium - 70% AC Key Insights • The frequency of each character dictates the feasibility and placement within the rearrangement. • A greedy approach sorts the most frequent available character to maximize the chance of successful placement. • A max-heap (priority queue) efficiently selects the highest frequency character time. • A waiting queue ensures that once a character is used, it can't be reused until k slots have passed. • Special case: When k is 0, no rearrangement is required because all orders satisfy the condition. Space & Time Time Complexity: $O(n)$ where n is the length of the input string. Space Complexity: $O(n)$ due to the storage used for the heap and wait queue. Solution Use a greedy algorithm with a max-heap to always select the character with the highest remaining frequency that is currently available. • Track the last index used for each character to avoid redundant work. Space & Time Time Complexity: $O(n \log m)$ where m is the number of unique characters. Space Complexity: $O(m)$ for the heap and wait queue. Solution Use a Dijkstra-like approach with a priority queue containing nodes of (frequency, char, last_index). The priority queue is ordered by frequency first and then by last_index. For each node, calculate the next possible position to place the character, ensuring it's at least k slots away from the last index.	visited = [False] * len(nums) def dfs(index, path): if index == len(nums): res.append(path + []) return for i in range(index + 1, len(nums)): if not visited[i]: path.append(nums[i]) dfs(i + 1, path) path.pop() visited[i] = False	<pre> dfs(i + 1, path) path.pop() visited[i] = False </pre>	<pre> if i < 0 and num[i] < num[i - 1] and not continue num[i] = True num[i + 1] = False num.sort() dfs(i + 1) return num </pre>	<pre> if i < 0 and num[i] < num[i - 1] and not continue num[i] = True num[i + 1] = False num.sort() dfs(i + 1) return num </pre>	<pre> if i < 0 and num[i] < num[i - 1] and not continue num[i] = True num[i + 1] = False num.sort() dfs(i + 1) return num </pre>
heap has more elements, the median is the top element of the heap. This method makes it efficient to dynamically update and retrieve the median as new numbers are inserted. #558 Rearrange String to Distance Apart LeetCode - Medium - 70% AC Key Insights • The frequency of each character dictates the feasibility and placement within the rearrangement. • A greedy approach sorts the most frequent available character to maximize the chance of successful placement. • A max-heap (priority queue) efficiently selects the highest frequency character time. • A waiting queue ensures that once a character is used, it can't be reused until k slots have passed. • Special case: When k is 0, no rearrangement is required because all orders satisfy the condition. Space & Time Time Complexity: $O(n)$ where n is the length of the input string. Space Complexity: $O(n)$ due to the storage used for the heap and wait queue. Solution Use a greedy algorithm with a max-heap to always select the character with the highest remaining frequency that is currently available. • Track the last index used for each character to avoid redundant work. Space & Time Time Complexity: $O(n \log m)$ where m is the number of unique characters. Space Complexity: $O(m)$ for the heap and wait queue. Solution Use a Dijkstra-like approach with a priority queue containing nodes of (frequency, char, last_index). The priority queue is ordered by frequency first and then by last_index. For each node, calculate the next possible position to place the character, ensuring it's at least k slots away from the last index.	visited = [False] * len(nums) def dfs(index, path): if index == len(nums): res.append(path + []) return for i in range(index + 1, len(nums)): if not visited[i]: path.append(nums[i]) dfs(i + 1, path) path.pop() visited[i] = False	<pre> dfs(i + 1, path) path.pop() visited[i] = False </pre>	<pre> if i < 0 and num[i] < num[i - 1] and not continue num[i] = True num[i + 1] = False num.sort() dfs(i + 1) return num </pre>	<pre> if i < 0 and num[i] < num[i - 1] and not continue num[i] = True num[i + 1] = False num.sort() dfs(i + 1) return num </pre>	<pre> if i < 0 and num[i] < num[i - 1] and not continue num[i] = True num[i + 1] = False num.sort() dfs(i + 1) return num </pre>

At each index, we: - Remove elements from the front of the deque if they are out of the current window (<code>i - index > k</code>). - Use the front of the deque (if non-empty) as the maximum <code>dp</code> value from the previous <code>k</code> indices. - Compare <code>dp[i]</code> and update the global maximum answer. - Remove from the back of the deque all indices with <code>dp</code> values less than the current <code>dp[i]</code> since they are not useful. - Insert the current index into the deque. #122 Word Search II [Hard] - Trie Key Insights - Use Depth-First Search (DFS) to explore possible words starting from each board cell. - Build a Trie (prefix tree) to store the words for DFS checks and efficient pruning. - Back visited cells and restore them after DFS to avoid revisiting in the current search path. - Prune search paths early if the current prefix is not part of any word in the Trie. Space & Time Time Complexity: $O(m * n * 4^L)$ in the worst case, where L is the maximum word length; however, using a Trie prunes unnecessary paths. Space Complexity: $O(N)$ for the Trie storage, where N is the total number of letters in all words, plus $O(n * m)$ for the board.	auxiliary space for recursion and board marking. Solution: We first insert all words into a Trie data structure to allow quick lookups of prefixes. Then, for each cell on the board, we perform DFS. During DFS, if the current path matches a Trie node that represents a complete word, we add that word to the result list and mark it as found to avoid duplicates. We continue exploring adjacent cells (up, down, left, right) while marking the cell as visited. After exploring, we restore the original state for further DFS calls. This combination of Trie and DFS with backtracking efficiently finds valid words on the board. #136 Palindrome Pairs [Hard] - Trie Key Insights - A palindrome is a string that reads the same forwards and backwards. - For any given word, if you split it into two parts, one part (either prefix or suffix) might be a palindrome. - If the other part reverse exists elsewhere in the array, then that reverse can be concatenated by a palindrome. - Carefully handle edge cases like empty strings, since an empty string is a palindrome by default. Space & Time Time Complexity: $O(n^2)$ in the worst case, where n is the number of words in the array.	Time Complexity: $O(N * L)$ in the worst case where N is the number of words and L is the maximum word length. We use the palindrome checking optimized this can be considered $O(n \log k)$ where k is the number of words in the array. Space Complexity: $O(N * L)$ due to the hashmap storing all words and their indices. Solution: We use a hashmap (dictionary) to store each word and its corresponding index. For every word, we iterate through every possible split position to divide the word into a prefix and a suffix. For each split: - If the prefix is a palindrome, we look for the reverse of the suffix in the hashmap. If found (and not the same word), then combining the reverse with the current word forms a palindrome. - Similarly, if the suffix is a palindrome, we look for the reverse of the prefix. We must be cautious with edge cases, especially when other part is empty, as the empty string is considered a palindrome. This method avoids checking every possible pair explicitly and leverages string reversal and palindrome checks to achieve efficiency. #158 Design In-Memory File System [Hard] - Trie Key Insights - Use a tree (trie) to represent directories and files.	Each node can be either a directory or a file. - Directories maintain a mapping of names to child nodes. - For <code>ls</code>, if the given path is a file, return only its name. If it's a directory, return the sorted list of names of its children. - When creating a new directory, we must ensure that all intermediate directories (if they do not already exist). - File nodes store content and support content appending. Space & Time Time Complexity: $O(\log k)$ where k is the number of entries in the directory (like in sorting) or $O(1)$ if reversing a long path. - Indexing: <code>addContentFolder</code>, <code>addContentFolderContent</code>, <code>dfs</code> where k is the depth of the file path. - Complexity: $O(N)$ where N is the total number of characters stored in directory names and the contents. The solution uses a tree-based approach where each node represents either a directory or a file. Each directory node stores its children in a hash table (or dictionary) mapping names to node objects. File nodes store their content as a string and carry a flag indicating if they are files. For the <code>ls</code> operation, check if the node is a	file - if <code>is</code>, return its name otherwise, list and sort the contents (if length is <code>> 1</code>). The <code>mkdir</code> operation traverses (or creates) nodes for each segment of the provided path. The <code>addContentFolder</code> method creates or creates the file node and appends the content. <code>addContentFolderContent</code> retrieves the content from the stored file node. #162 Design Search Autocomplete System [Hard] - Trie Key Insights - Use a Trie to efficiently store and retrieve sentences. - Traverse until the Trie is found the current prefix node. - Maintain a mapping at each Trie node to record sentences that follow the prefix (like in sorting). - Use sorting or a min-heap to select the top three sentences based on their frequency (that degree) and lexicographical order of frequencies list. - On receiving <code>W</code>, update the Trie with the current sentence and make the current search state. Space & Time Time Complexity: $O(\log k)$ per character input query, where k is the length of the current prefix and k is 3 (constant factor), thus ensuring efficient autocomplete.	Space Complexity: $O(N * L)$ where N is the number of unique sentences and L is the average sentence length, for maintaining the Trie. Solution: The solution involves around building a Trie where each node contains: - Counters for child nodes. - A hash map that records all sentences passing through the node along with their occurrence counts. For each character entered (except '<code>'</code>), the system: - Traverse until the Trie is found the current prefix node. - Retrieves stored sentences at that node and sorts them based on their frequency (that degree) and lexicographical order of frequencies list. - Returns the top three sentences from the sorted result. For the user inputs '<code>W</code>', the current sentence (framed as user input to <code>W</code>) is added or its frequency updated (if <code>W</code> is <code>'</code>), and the top three sentences from the Trie are returned. #170 Solitaire [Hard] - Backtracking Key Insights - Use backtracking to explore different placements for each empty cell. - Validate placements by ensuring that the chosen digit is not already present in the corresponding row, column, or 3x3 sub-box. - Employ recursion to fill the board incrementally and backtrack upon encountering invalid configurations. - Although the worst-case time complexity is exponential, the fixed board size (9x9) keeps the problem computationally manageable. Space & Time Time Complexity: In the worst-case scenario, the algorithm can explore up to $9! \times 9!$ possibilities, where n is the number of empty cells. However, since n is at most 12 and many branches are pruned early (due to the practical reasons) for less. Space Complexity: $O(n)$ due to the recursion stack, where n is the number of empty cells given the board's fixed size. This space is constant in practice. Solution: The solution uses a backtracking approach. For each empty cell on the board, we attempt to place a digit from '1' to '9'. Before placing a digit, we check if the placement is valid by ensuring that: - The digit is not already present in the same row. - The digit is not already present in the same column. - The digit is not already present in the corresponding 3x3 sub-box. If a valid digit is found, we place it on the board and recursively attempt to solve the rest of the board. If no valid digit can be placed, the algorithm backtracks by
---	---	---	--	---	--

Round 6 - Mid - Hard	a-2761	Round 6 - Mid - Hard	a-2762	Round 6 - Mid - Hard	a-2763	Round 6 - Mid - Hard	a-2764	Round 6 - Mid - Hard	a-2765	Round 6 - Mid - Hard	a-2766
----------------------	--------	----------------------	--------	----------------------	--------	----------------------	--------	----------------------	--------	----------------------	--------

reversing the last move and trying a different digit. This recursive search continues until the board is completely filled with a valid Sudoku configuration. #151 N-Queens [Hard] - Backtracking Key Insights - Use backtracking to explore potential positions row by row. - Use sets to track which columns and diagonals are under attack. - Diagonals can be tracked by <code>(row - col) + 1</code> and <code>(row + col) + 1</code>, ensuring that queens do not conflict along these diagonals. - Recursively build each valid board configuration and backtrack when a conflict arises. Space & Time Time Complexity: $O(N!)$ in the worst-case scenario, as the solution explores permutations for queen placements. Space Complexity: $O(N)$ for storing board configurations and $O(N)$ for recursion stack and conflict-tracking sets. Solution: We use a backtracking algorithm that places queens row by row. For each row, we try placing a queen in each column. Before placing, we check if the column, main	diagonal (row - col), and anti-diagonal (row + col) are free using dedicated sets. If a queen can be safely placed, we mark the column and diagonals as occupied and recurse to the next row. Once all rows are processed, the board configuration is a valid solution and is added to the result. After exhaustively trying all possible positions, the algorithm returns all valid solutions. #126 Word Ladder II [Hard] - Backtracking Key Insights - Use Breadth-First Search (BFS) to shortest level-by-level and capture only the shortest paths. - Build a parent mapping during BFS to record which words lead to each newly discovered word. - Once the shortest path is found via BFS, use backtracking (or DFS) from endword to reconstruct all transformation sequences. - Remove words as they are visited within a level to avoid unnecessary revisits and cycles. - The solution must handle multiple shortest paths. Space & Time Time Complexity: $O(N * L)$ where N is the number of words in the wordlist and L is the length of each word.	This accounts for checking all possible one-letter transformations when iterating in the backtracking. Space Complexity: $O(N * L)$ for storing the parent mapping and maintaining the BFS queue and other auxiliary data structures. Solution: - We solve the problem in two major phases: - Use BFS to explore from <code>beginWord</code> to <code>endWord</code>. - Once BFS reaches <code>endWord</code>, record all parents for each word encountered. This ensures we capture all possible predecessors that lead to a shortest transformation. - Once the BFS completes (once <code>endWord</code> is reached), use backtracking (DFS) beginning from <code>endWord</code> and moving backwards via the parent mapping to reconstruct all transformation sequences in reverse. Finally, reverse each sequence to present it from <code>beginWord</code> to <code>endWord</code>. This two-phase approach guarantees that only the shortest paths are reconstructed, as BFS ensures level-by-level exploration. #196 Number of Squarful Arrays [Hard] - Backtracking Key Insights - The problem requires ensuring adjacent pairs sum up to a perfect square.	During the array helps in efficiently handling transformations when iterating in the backtracking. Space Complexity: $O(N * L)$ for storing the parent mapping and maintaining the BFS queue and other auxiliary data structures. Solution: - We solve the problem in two major phases: - Use BFS to explore from <code>beginWord</code> to <code>endWord</code>. - Once BFS reaches <code>endWord</code>, record all parents for each word encountered. This ensures we capture all possible predecessors that lead to a shortest transformation. - Once the BFS completes (once <code>endWord</code> is reached), use backtracking (DFS) beginning from <code>endWord</code> and moving backwards via the parent mapping to reconstruct all transformation sequences in reverse. Finally, reverse each sequence to present it from <code>beginWord</code> to <code>endWord</code>. This two-phase approach guarantees that only the shortest paths are reconstructed, as BFS ensures level-by-level exploration. #196 Number of Squarful Arrays [Hard] - Backtracking Key Insights - The problem requires ensuring adjacent pairs sum up to a perfect square.	During the array helps in efficiently handling transformations when iterating in the backtracking. Space Complexity: $O(N * L)$ for storing the parent mapping and maintaining the BFS queue and other auxiliary data structures. Solution: - We solve the problem in two major phases: - Use BFS to explore from <code>beginWord</code> to <code>endWord</code>. - Once BFS reaches <code>endWord</code>, record all parents for each word encountered. This ensures we capture all possible predecessors that lead to a shortest transformation. - Once the BFS completes (once <code>endWord</code> is reached), use backtracking (DFS) beginning from <code>endWord</code> and moving backwards via the parent mapping to reconstruct all transformation sequences in reverse. Finally, reverse each sequence to present it from <code>beginWord</code> to <code>endWord</code>. This two-phase approach guarantees that only the shortest paths are reconstructed, as BFS ensures level-by-level exploration. #196 Number of Squarful Arrays [Hard] - Backtracking Key Insights - The problem requires ensuring adjacent pairs sum up to a perfect square.	During the array helps in efficiently handling transformations when iterating in the backtracking. Space Complexity: $O(N * L)$ for storing the parent mapping and maintaining the BFS queue and other auxiliary data structures. Solution: - We solve the problem in two major phases: - Use BFS to explore from <code>beginWord</code> to <code>endWord</code>. - Once BFS reaches <code>endWord</code>, record all parents for each word encountered. This ensures we capture all possible predecessors that lead to a shortest transformation. - Once the BFS completes (once <code>endWord</code> is reached), use backtracking (DFS) beginning from <code>endWord</code> and moving backwards via the parent mapping to reconstruct all transformation sequences in reverse. Finally, reverse each sequence to present it from <code>beginWord</code> to <code>endWord</code>. This two-phase approach guarantees that only the shortest paths are reconstructed, as BFS ensures level-by-level exploration. #196 Number of Squarful Arrays [Hard] - Backtracking Key Insights - The problem requires ensuring adjacent pairs sum up to a perfect square.
---	--	---	---	---	---

Round 6 - Mid - Hard	a-2767	Round 6 - Mid - Hard	a-2768	Round 6 - Mid - Hard	a-2769	Round 6 - Mid - Hard	a-2770	Round 6 - Mid - Hard	a-2771	Round 6 - Mid - Hard	a-2772
----------------------	--------	----------------------	--------	----------------------	--------	----------------------	--------	----------------------	--------	----------------------	--------

Dynamic Programming #70 Climbing Stairs [Easy] LeetCode - Simple: LeetCode - LeetCode Math - Dynamic Programming - Memorization Link: Climbing Stairs Time Complexity: $O(n)$ - requires one pass through the price array. Space Complexity: $O(1)$ - only a few extra variables are used. Problem: You are climbing a staircase. It takes <code>n</code> steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top? Example 1: Input: <code>n = 2</code> Output: <code>2</code> Explanation: There are two ways to climb to the top. 1. 1 step + 1 step 2. 2 steps Example 2: Input: <code>n = 3</code> Output: <code>3</code> Explanation: There are three ways to climb to the top. 1. 1 step + 1 step + 1 step 2. 1 step + 2 steps 3. 2 steps	Input: <code>n = 3</code> Output: <code>3</code> Explanation: There are three ways to climb to the top. 1. 1 step + 1 step + 1 step 2. 1 step + 2 steps 3. 2 steps + 1 step Constraints: <code>1 <= n <= 45</code> Brute Force (Python) <pre>def climbStairs(n): if n == 1: return 1 if n == 2: return 2 return climbStairs(n-1) + climbStairs(n-2)</pre> Community Solutions (Python) - WuLCx	Input: <code>n = 3</code> Output: <code>3</code> Explanation: There are three ways to climb to the top. 1. 1 step + 1 step + 1 step 2. 1 step + 2 steps 3. 2 steps + 1 step Constraints: <code>1 <= n <= 45</code> Brute Force (Python) <pre>def climbStairs(n): if n == 1: return 1 if n == 2: return 2 return climbStairs(n-1) + climbStairs(n-2)</pre> Community Solutions (Python) - WuLCx	<pre>class Solution: def numWays(self, n: int) -> int: dp = [0] * (n + 1) dp[1] = 1 dp[2] = 2 for i in range(3, n + 1): dp[i] = dp[i-1] + dp[i-2] return dp[n]</pre> Community Solutions (Python) WuLCx	<pre>def numWays(self, n: int) -> int: dp = [0] * (n + 1) dp[1] = 1 dp[2] = 2 for i in range(3, n + 1): dp[i] = dp[i-1] + dp[i-2] return dp[n]</pre> Community Solutions (Python) WuLCx	<pre>def numWays(self, n: int) -> int: dp = [0] * (n + 1) dp[1] = 1 dp[2] = 2 for i in range(3, n + 1): dp[i] = dp[i-1] + dp[i-2] return dp[n]</pre> Community Solutions (Python) WuLCx	<pre>def numWays(self, n: int) -> int: dp = [0] * (n + 1) dp[1] = 1 dp[2] = 2 for i in range(3, n + 1): dp[i] = dp[i-1] + dp[i-2] return dp[n]</pre> Community Solutions (Python) WuLCx
--	--	--	---	---	---	---

Dynamic Programming #121 Best Time to Buy and Sell Stock [Easy] LeetCode - Simple: LeetCode - LeetCode Array - Dynamic Programming Link: Climbing Stairs Time Complexity: $O(n)$ - requires one pass through the price array. Space Complexity: $O(1)$ - only a few extra variables are used. Problem: You are given an array <code>prices</code> where <code>prices[i]</code> is the price of a given stock on the <code>i</code>th day. You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock. Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0. Example 1: Input: <code>prices = [7,1,5,3,6,4]</code> Output: <code>5</code> Explanation: Buy on day 2 (<code>price = 1</code>) and sell on day 5 (<code>price = 6</code>), profit = <code>6 - 1 = 5</code>. Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.	Input: <code>prices = [7,1,5,3,6,4]</code> Output: <code>5</code> Explanation: Buy on day 2 (<code>price = 1</code>) and sell on day 5 (<code>price = 6</code>), profit = <code>6 - 1 = 5</code>. Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.	<pre>class Solution: def maxProfit(self, prices: List[int]) -> int: min_price = float('inf') max_profit = 0 for price in prices: min_price = min(min_price, price) max_profit = max(max_profit, price - min_price) return max_profit</pre> Community Solutions (Python) WuLCx	<pre>class Solution: def maxProfit(self, prices: List[int]) -> int: min_price = float('inf') max_profit = 0 for price in prices: min_price = min(min_price, price) max_profit = max(max_profit, price - min_price) return max_profit</pre> Community Solutions (Python) WuLCx	<pre>class Solution: def maxProfit(self, prices: List[int]) -> int: min_price = float('inf') max_profit = 0 for price in prices: min_price = min(min_price, price) max_profit = max(max_profit, price - min_price) return max_profit</pre> Community Solutions (Python) WuLCx	<pre>class Solution: def maxProfit(self, prices: List[int]) -> int: min_price = float('inf') max_profit = 0 for price in prices: min_price = min(min_price, price) max_profit = max(max_profit, price - min_price) return max_profit</pre> Community Solutions (Python) WuLCx	<pre>class Solution: def maxProfit(self, prices: List[int]) -> int: min_price = float('inf') max_profit = 0 for price in prices: min_price = min(min_price, price) max_profit = max(max_profit, price - min_price) return max_profit</pre> Community Solutions (Python) WuLCx	<pre>class Solution: def maxProfit(self, prices: List[int]) -> int: min_price = float('inf') max_profit = 0 for price in prices: min_price = min(min_price, price) max_profit = max(max_profit, price - min_price) return max_profit</pre> Community Solutions (Python) WuLCx
--	---	---	---	---	---	---	---

Bit Manipulation #67 Add Binary [Easy] LeetCode - Simple: LeetCode - LeetCode Math - String - Bit Manipulation - Simulation Link: Climbing Stairs Time Complexity: $O(m + n)$, where m and n are the lengths of the two binary strings. Space Complexity: $O(1)$, only for storing the resulting binary string. Problem: Given two binary strings <code>a</code> and <code>b</code>, return their sum as a binary string. Example 1: Input: <code>a = "11", b = "1"</code> Output: <code>"100"</code> Example 2: Input: <code>a = "1010", b = "1011"</code> Output: <code>"10101"</code> Constraints: <code>1 <= a.length, b.length <= 1000</code> <code>a</code> and <code>b</code> consist only of '0' or '1' characters. - Each string does not contain leading zeros except for the zero itself.	Input: <code>a = "11", b = "1"</code> Output: <code>"100"</code> Example 2: Input: <code>a = "1010", b = "1011"</code> Output: <code>"10101"</code> Constraints: <code>1 <= a.length, b.length <= 1000</code> <code>a</code> and <code>b</code> consist only of '0' or '1' characters. - Each string does not contain leading zeros except for the zero itself.	<pre>class Solution: def addBinary(self, a: str, b: str) -> str: # Function to add two binary strings def addBinary(a: str, b: str) -> str: carry = 0 result = [] for i in range(len(a) + len(b)): digit_a = int(a[i] if i < len(a) else 0) digit_b = int(b[i] if i < len(b) else 0) sum = digit_a + digit_b + carry result.append(str(sum % 2)) carry = sum // 2 return ''.join(result[::-1])</pre> Community Solutions (Python) WuLCx	<pre>class Solution: def addBinary(self, a: str, b: str) -> str: # Function to add two binary strings def addBinary(a: str, b: str) -> str: carry = 0 result = [] for i in range(len(a) + len(b)): digit_a = int(a[i] if i < len(a) else 0) digit_b = int(b[i] if i < len(b) else 0) sum = digit_a + digit_b + carry result.append(str(sum % 2)) carry = sum // 2 return ''.join(result[::-1])</pre> Community Solutions (Python) WuLCx	<pre>class Solution: def addBinary(self, a: str, b: str) -> str: # Function to add two binary strings def addBinary(a: str, b: str) -> str: carry = 0 result = [] for i in range(len(a) + len(b)): digit_a = int(a[i] if i < len(a) else 0) digit_b = int(b[i] if i < len(b) else 0) sum = digit_a + digit_b + carry result.append(str(sum % 2)) carry = sum // 2 return ''.join(result[::-1])</pre> Community Solutions (Python) WuLCx	<pre>class Solution: def addBinary(self, a: str, b: str) -> str: # Function to add two binary strings def addBinary(a: str, b: str) -> str: carry = 0 result = [] for i in range(len(a) + len(b)): digit_a = int(a[i] if i < len(a) else 0) digit_b = int(b[i] if i < len(b) else 0) sum = digit_a + digit_b + carry result.append(str(sum % 2)) carry = sum // 2 return ''.join(result[::-1])</pre> Community Solutions (Python) WuLCx	<pre>class Solution: def addBinary(self, a: str, b: str) -> str: # Function to add two binary strings def addBinary(a: str, b: str) -> str: carry = 0 result = [] for i in range(len(a) + len(b)): digit_a = int(a[i] if i < len(a) else 0) digit_b = int(b[i] if i < len(b) else 0) sum = digit_a + digit_b + carry result.append(str(sum % 2)) carry = sum // 2 return ''.join(result[::-1])</pre> Community Solutions (Python) WuLCx
---	---	--	--	--	--	--

Bit Manipulation #136 Single Number [Easy] LeetCode - Simple: LeetCode - LeetCode Array - Bit Manipulation Link: Climbing Stairs Time Complexity: $O(n)$. We only traverse the array once. Space Complexity: $O(1)$. Only a single extra variable is used regardless of input size. Problem: Given a non-empty array of integers <code>nums</code>, every element appears twice except for one. Find that single one. You must implement a solution with a linear runtime complexity and use only constant extra space. Example 1: Input: <code>nums = [2,2,1]</code> Output: <code>1</code> Example 2: Input: <code>nums = [4,1,4,1,2]</code> Output: <code>2</code> Example 3: Input: <code>nums = [1]</code> Output: <code>1</code>	Input: <code>nums = [2,2,1]</code> Output: <code>1</code> Example 2: Input: <code>nums = [4,1,4,1,2]</code> Output: <code>2</code> Example 3: Input: <code>nums = [1]</code> Output: <code>1</code>	<pre>class Solution: def singleNumber(self, nums: List[int]) -> int: return functools.reduce(lambda x, y: x ^ y, nums, 0)</pre> Community Solutions (Python) WuLCx	<pre>class Solution: def singleNumber(self, nums: List[int]) -> int: return functools.reduce(lambda x, y: x ^ y, nums, 0)</pre> Community Solutions (Python) WuLCx	<pre>class Solution: def singleNumber(self, nums: List[int]) -> int: return functools.reduce(lambda x, y: x ^ y, nums, 0)</pre> Community Solutions (Python) WuLCx	<pre>class Solution: def singleNumber(self, nums: List[int]) -> int: return functools.reduce(lambda x, y: x ^ y, nums, 0)</pre> Community Solutions (Python) WuLCx	<pre>class Solution: def singleNumber(self, nums: List[int]) -> int: return functools.reduce(lambda x, y: x ^ y, nums, 0)</pre> Community Solutions (Python) WuLCx	<pre>class Solution: def singleNumber(self, nums: List[int]) -> int: return functools.reduce(lambda x, y: x ^ y, nums, 0)</pre> Community Solutions (Python) WuLCx
--	---	--	--	--	--	--	--

Sheet 77/83 · LoetMastery Study-Order · 36-up Landscape · 6x6 Round 7 · Low · Easy

<pre>class Solution: def canPartition(self, nums: List[int]) -> bool: s = sum(nums) if s % 2 != 0: return False n = len(nums) dp = [[False] * (s + 1)] dp[0][0] = True for i in range(1, n + 1): for j in range(s + 1): # for target is target, s - i, -1, -1 including = False # dp[target - i] # dp[target] = dp[target] or # return True</pre> class Solution: # recursive, but over time limit in O3 <pre>def canPartition(self, nums: List[int]) -> bool: total_sum = sum(nums) if total_sum % 2 != 0: return False target_sum = total_sum // 2</pre> Round 0 - Low - Medium p.2881	<pre>def dfs(idx, curr_sum): if curr_sum == target_sum: return True if curr_sum > target_sum or idx == len(nums): return False return dfs(idx + 1, curr_sum) or dfs(idx + 1, curr_sum + nums[idx])</pre> class Solution: # recursive, but over time limit in O3 <pre>def canPartition(self, nums: List[int]) -> bool: total_sum = sum(nums) if total_sum % 2 != 0: return False target_sum = total_sum // 2</pre> Round 0 - Low - Medium p.2882	<pre>class Solution: def canPartition(self, nums: List[int]) -> bool: n, mod = divmod(sum(nums), 2) if mod: return False for i in range(1, n + 1): f[i] = f[i] or f[i - i] return f[n]</pre> Round 0 - Low - Medium p.2883	#435 Non-overlapping Intervals MED LeetCode - Simplify: WalCC - LeetCode Array - Dynamic Programming - Greedy - Sorting Time Complexity: O(n log n) due to sorting the intervals. Space Complexity: O(n) for storing the intervals. Note that intervals are already sorted by their start time for O(n) if sorting in-place. Problem: Given an array of intervals where intervals[i] = [start_i, end_i], return the minimum number of intervals you need to remove to make the rest of the intervals non-overlapping. Example 1: Input: intervals = [[1,2],[2,3],[3,4]] Output: 2 Explanation: [2,3] can be removed and the rest of the intervals are non-overlapping. Example 2: Round 0 - Low - Medium p.2884	<pre>Input: intervals = [[1,2],[1,2],[1,2]] Output: 2 Explanation: You need to remove two [1,2] to make the rest of the intervals non-overlapping. Example 3: Input: intervals = [[1,2],[2,3],[3,4]] Output: 0 Explanation: No intervals need to be removed as they are already non-overlapping. Constraints: 1 <= intervals.length <= 1000 intervals[i].length == 2 -50 <= start_i <= end_i <= 50</pre> Brute Force Approach <pre>class Solution: def eraseOverlapIntervals(self, intervals: List[List[int]]) -> int: intervals.sort(key=lambda x: x[1]) count = 1 for i in range(1, len(intervals)): if intervals[i][0] >= intervals[i-1][1]: count += 1 return count</pre> class Solution: <pre>def eraseOverlapIntervals(self, intervals: List[List[int]]) -> int: intervals.sort(key=lambda x: x[1]) count = 1 for i in range(1, len(intervals)): if intervals[i][0] >= intervals[i-1][1]: count += 1 return count</pre> Round 0 - Low - Medium p.2885	Community Solutions (Python) <pre>def eraseOverlapIntervals(self, intervals: List[List[int]]) -> int: intervals.sort(key=lambda x: x[1]) count = 1 for i in range(1, len(intervals)): if intervals[i][0] >= intervals[i-1][1]: count += 1 return count</pre> Round 0 - Low - Medium p.2886		
<pre>class Solution: def longestPalindrome(self, s: str) -> int: n = len(s) dp = [[0] * n] for i in range(n): dp[i][i] = 1 for i in range(n - 1, -1, -1): for j in range(i + 1, n): if s[i] == s[j]: dp[i][j] = dp[i + 1][j - 1] + 2 else: dp[i][j] = max(dp[i + 1][j], dp[i][j - 1]) return dp[0][n - 1]</pre> Python Implementation for Non-overlapping Substrings <pre>class Solution: def longestPalindrome(self, s: str) -> int: n = len(s) dp = [[0] * n] for i in range(n): dp[i][i] = 1 for i in range(n - 1, -1, -1): for j in range(i + 1, n): if s[i] == s[j]: dp[i][j] = dp[i + 1][j - 1] + 2 else: dp[i][j] = max(dp[i + 1][j], dp[i][j - 1]) return dp[0][n - 1]</pre> Round 0 - Low - Medium p.2887	<pre>class Solution: def longestPalindrome(self, s: str) -> int: n = len(s) dp = [[0] * n] for i in range(n): dp[i][i] = 1 for i in range(n - 1, -1, -1): for j in range(i + 1, n): if s[i] == s[j]: dp[i][j] = dp[i + 1][j - 1] + 2 else: dp[i][j] = max(dp[i + 1][j], dp[i][j - 1]) return dp[0][n - 1]</pre> Round 0 - Low - Medium p.2888	Dynamic Programming #516 Longest Palindromic Subsequence MED LeetCode - Simplify: WalCC - LeetCode String - Dynamic Programming Time Complexity: O(n^2) where n is the length of the string. Space Complexity: O(n^2) due to the usage of a 2D DP table. Problem: Given a string s, find the longest palindromic subsequence's length in s. Example 1: Input: s = "bbbab" Output: 4 Explanation: The possible longest palindromic subsequence is "bbbb". Example 2: Round 0 - Low - Medium p.2889	Dynamic Programming <pre>Input: s = "bbbab" Output: 4 Explanation: The possible longest palindromic subsequence is "bbbb". Example 2: Input: s = "cbbd" Output: 2 Explanation: The possible longest palindromic subsequence is "bb".</pre> Community Solutions (Python) <pre>class Solution: def longestPalindromicSubsequence(self, s: str) -> int: n = len(s) dp = [[0] * n] for i in range(n): dp[i][i] = 1 for i in range(n - 1, -1, -1): for j in range(i + 1, n): if s[i] == s[j]: dp[i][j] = dp[i + 1][j - 1] + 2 else: dp[i][j] = max(dp[i + 1][j], dp[i][j - 1]) return dp[0][n - 1]</pre> Round 0 - Low - Medium p.2890	Dynamic Programming #576 Out of Boundary Paths MED LeetCode - Simplify: WalCC - LeetCode Dynamic Programming Time Complexity: O(m * n * m) since we compute the number of ways to reach each cell in the grid (possibly out of the grid crossing the grid boundary). Space Complexity: O(m * n) for the memoization table. Problem: There is an m x n grid with a ball. The ball is initially at the position (startRow, startColumn). You are allowed to move the ball to one of the four adjacent cells in the grid (possibly out of the grid crossing the grid boundary). You can apply at most maxMove moves to the ball. Example 1: Input: m = 2, n = 2, maxMove = 3, startRow = 0, startColumn = 0 Output: 12 Explanation: There are 12 ways to move the ball out of the grid boundary. Since the answer can be very large, return it modulo 10^9 + 7. Round 0 - Low - Medium p.2891	class Solution: <pre>def findPaths(self, m: int, n: int, maxMove: int, startRow: int, startColumn: int) -> int: @cache def dfs(x: int, y: int, moves: int) -> int: if x < 0 or x > m - 1 or y < 0 or y > n - 1: return 1 if moves == 0: return 0 res = 0 for dx, dy in [(1, 0), (-1, 0), (0, 1), (0, -1)]: res += dfs(x + dx, y + dy, moves - 1) return res</pre> Example 1: Input: m = 2, n = 2, maxMove = 3, startRow = 0, startColumn = 0 Output: 12 Explanation: There are 12 ways to move the ball out of the grid boundary. Since the answer can be very large, return it modulo 10^9 + 7. Round 0 - Low - Medium p.2892	<pre>class Solution: def findPaths(self, m: int, n: int, maxMove: int, startRow: int, startColumn: int) -> int: @cache def dfs(x: int, y: int, moves: int) -> int: if x < 0 or x > m - 1 or y < 0 or y > n - 1: return 1 if moves == 0: return 0 res = 0 for dx, dy in [(1, 0), (-1, 0), (0, 1), (0, -1)]: res += dfs(x + dx, y + dy, moves - 1) return res</pre> Round 0 - Low - Medium p.2893	<pre>class Solution: def findPaths(self, m: int, n: int, maxMove: int, startRow: int, startColumn: int) -> int: @cache def dfs(x: int, y: int, moves: int) -> int: if x < 0 or x > m - 1 or y < 0 or y > n - 1: return 1 if moves == 0: return 0 res = 0 for dx, dy in [(1, 0), (-1, 0), (0, 1), (0, -1)]: res += dfs(x + dx, y + dy, moves - 1) return res</pre> Round 0 - Low - Medium p.2894
Brute Force Approach <pre>def findPaths(self, m: int, n: int, maxMove: int, startRow: int, startColumn: int) -> int: @cache def dfs(x: int, y: int, moves: int) -> int: if x < 0 or x > m - 1 or y < 0 or y > n - 1: return 1 if moves == 0: return 0 res = 0 for dx, dy in [(1, 0), (-1, 0), (0, 1), (0, -1)]: res += dfs(x + dx, y + dy, moves - 1) return res</pre> Community Solutions (Python) <pre>class Solution: def findPaths(self, m: int, n: int, maxMove: int, startRow: int, startColumn: int) -> int: @cache def dfs(x: int, y: int, moves: int) -> int: if x < 0 or x > m - 1 or y < 0 or y > n - 1: return 1 if moves == 0: return 0 res = 0 for dx, dy in [(1, 0), (-1, 0), (0, 1), (0, -1)]: res += dfs(x + dx, y + dy, moves - 1) return res</pre> Round 0 - Low - Medium p.2895	Dynamic Programming #1143 Longest Common Subsequence MED LeetCode - Simplify: WalCC - LeetCode String - Dynamic Programming Time Complexity: O(m * n) where m and n are the lengths of text1 and text2. Space Complexity: O(m * n) for maintaining the 2D DP table. Space can be optimized to O(min(m, n)) if needed. Problem: Given two strings text1 and text2, return the length of their longest common subsequence. If there is no common subsequence, return 0. Example 1: Input: text1 = "abcde", text2 = "ace" Output: 3 Explanation: The longest common subsequence is "ace". Example 2: Input: text1 = "abc", text2 = "abc" Output: 3 Explanation: The longest common subsequence is "abc". Example 3: Input: text1 = "abc", text2 = "def" Output: 0 Explanation: There is no common subsequence, so the result is 0. Constraints: 1 <= text1.length, text2.length <= 1000 text1 and text2 consist of only lowercase English characters. Community Solutions (Python) <pre>class Solution: def longestCommonSubsequence(self, text1: str, text2: str) -> int: m, n = len(text1), len(text2) dp = [[0] * (n + 1)] for i in range(1, m + 1): for j in range(1, n + 1): if text1[i - 1] == text2[j - 1]: dp[i][j] = dp[i - 1][j - 1] + 1 else: dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]) return dp[m][n]</pre> Round 0 - Low - Medium p.2896	<pre>class Solution: def longestCommonSubsequence(self, text1: str, text2: str) -> int: m, n = len(text1), len(text2) dp = [[0] * (n + 1)] for i in range(1, m + 1): for j in range(1, n + 1): if text1[i - 1] == text2[j - 1]: dp[i][j] = dp[i - 1][j - 1] + 1 else: dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]) return dp[m][n]</pre> Round 0 - Low - Medium p.2897	<pre>class Solution: def longestCommonSubsequence(self, text1: str, text2: str) -> int: m, n = len(text1), len(text2) dp = [[0] * (n + 1)] for i in range(1, m + 1): for j in range(1, n + 1): if text1[i - 1] == text2[j - 1]: dp[i][j] = dp[i - 1][j - 1] + 1 else: dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]) return dp[m][n]</pre> Round 0 - Low - Medium p.2898	<pre>class Solution: def longestCommonSubsequence(self, text1: str, text2: str) -> int: m, n = len(text1), len(text2) dp = [[0] * (n + 1)] for i in range(1, m + 1): for j in range(1, n + 1): if text1[i - 1] == text2[j - 1]: dp[i][j] = dp[i - 1][j - 1] + 1 else: dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]) return dp[m][n]</pre> Round 0 - Low - Medium p.2899	<pre>class Solution: def longestCommonSubsequence(self, text1: str, text2: str) -> int: m, n = len(text1), len(text2) dp = [[0] * (n + 1)] for i in range(1, m + 1): for j in range(1, n + 1): if text1[i - 1] == text2[j - 1]: dp[i][j] = dp[i - 1][j - 1] + 1 else: dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]) return dp[m][n]</pre> Round 0 - Low - Medium p.2900		
<pre>class Solution: def longestCommonSubsequence(self, text1: str, text2: str) -> int: m, n = len(text1), len(text2) dp = [[0] * (n + 1)] for i in range(1, m + 1): for j in range(1, n + 1): if text1[i - 1] == text2[j - 1]: dp[i][j] = dp[i - 1][j - 1] + 1 else: dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]) return dp[m][n]</pre> Round 0 - Low - Medium p.2901	Bit Manipulation #78 Subsets MED LeetCode - Simplify: WalCC - LeetCode Array - Backtracking - Bit Manipulation Time Complexity: O(2^n * n) where n is the number of elements in the array. Space Complexity: O(n) for the recursion stack. Problem: Given an integer array nums of unique elements, return all possible subsets (the power set). The solution set must not contain duplicate subsets. Return the solution in any order. Example 1: Input: nums = [1,2,3] Output: [[], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3]] Example 2: Input: nums = [0] Output: [[], [0]] Constraints: 1 <= nums.length <= 10 -10 <= nums[i] <= 10 Round 0 - Low - Medium - 4 questions p.2902	<pre>class Solution: def subsets(self, nums: List[int]) -> List[List[int]]: res = [] def dfs(start: int, path: List[int]) -> None: res.append(path[:]) for i in range(start, len(nums)): path.append(nums[i]) dfs(i + 1, path) path.pop() dfs(0, []) return res</pre> Round 0 - Low - Medium p.2903	<pre>class Solution: def subsets(self, nums: List[int]) -> List[List[int]]: res = [] def dfs(start: int, path: List[int]) -> None: res.append(path[:]) for i in range(start, len(nums)): path.append(nums[i]) dfs(i + 1, path) path.pop() dfs(0, []) return res</pre> Round 0 - Low - Medium p.2904	<pre>class Solution: def subsets(self, nums: List[int]) -> List[List[int]]: res = [] def dfs(start: int, path: List[int]) -> None: res.append(path[:]) for i in range(start, len(nums)): path.append(nums[i]) dfs(i + 1, path) path.pop() dfs(0, []) return res</pre> Round 0 - Low - Medium p.2905	<pre>class Solution: def subsets(self, nums: List[int]) -> List[List[int]]: res = [] def dfs(start: int, path: List[int]) -> None: res.append(path[:]) for i in range(start, len(nums)): path.append(nums[i]) dfs(i + 1, path) path.pop() dfs(0, []) return res</pre> Round 0 - Low - Medium p.2906		

Sheet 83/87 - LoebMastery Study-Order - 36-up Landscape - 6x6 - Round 8 - Low - Medium p.2986

We use a dynamic programming approach by iterating through each house in the array. At each house, we consider two scenarios: either we rob the current house and add its money to the total from two houses ago, or we skip it (leaving the optimal solution up to the previous house). By using two variables to store these intermediate results, we achieve an optimal space complexity of $O(1)$. #256 Paint House (Medium) - Dynamic Programming Key Insights - The choice for painting each house depends on the previous house's color choices. - The dynamic programming to accumulate the cost while ensuring adjacent houses do not have the same color. - The state of each house can be represented by the minimum cost when the house is painted a certain color, computed using the minimal costs of the previous house from the other two colors. - The problem only requires updating the cost matrix in place for optimal space usage. Space & Time Time Complexity: $O(n)$, where n is the number of houses.	Space Complexity: $O(1)$ extra space if the input matrix is modified in place. Solution We use a dynamic programming approach with iterative in-place updating of the cost matrix. For each house starting from the second one, we update the cost for each color by adding the minimal cost of painting the previous house with either of the other two colors. This ensures we never use the same color for adjacent houses while accumulating the minimal total cost. The final answer is the minimum value for the last house. #309 Best Time to Buy and Sell Stock with Cooldown (Medium) - Dynamic Programming Key Insights - Use Dynamic Programming to solve the problem. - Maintain three states to represent: $dp[i][0]$: the state where you are ready to buy (not holding a stock). $dp[i][1]$: the state where you are holding a stock. $dp[i][2]$: the cooldown state encountered right after selling a stock. - Transition into all can come from staying in all or coming out from the cooldown state $dp[i-2]$.	length length, the solution is the maximum sequence length by either excluding the left or the right character. The approach builds the solution from smaller substrings (length 1) to the entire string. Space & Time Time Complexity: $O(n^2)$ where n is the length of the string. Space Complexity: $O(n^2)$ due to the usage of a 2D DP table. Solution We solve this problem using a bottom-up dynamic programming approach. We create a 2D table $dp[i][j]$ such that $dp[i][j]$ stores the longest palindromic substring length for the substring $arr[i..j]$. For any single character, $dp[i][i] = 1$. For substrings of length greater than 1, we check the characters at positions i and j and if they match, then $dp[i][j] = dp[i+1][j-1] + 2$. If they do not match, we take the maximum of $dp[i][i+1]$ and $dp[i+1][j]$. Finally, the $dp[i][i-1]$ holds the result of the best substring. #750 Word Search (Medium) - Dynamic Programming Key Insights Use Dynamic Programming (DP) as the state is defined by (current row, current column, moves remaining).	node since every node's value (except the root's) is added once as a child. Finally, we iterate over the nodes to find the node with the computed root value and return it. This approach meets the requirement of constant space complexity and linear time. #7 Reverse Integer (Medium) - Math Key Insights - The problem requires reversing the digits while preserving the sign. - Match out for integer overflow due to the reversed number exceeding the 32-bit signed integer limits. - The approach involves extracting the last digit of the number and inserting it into the reversed number. - Overflow checks are done during each iteration before actually appending the digit. Space & Time Time Complexity: $O(n)$, where n is the number of digits. Space Complexity: $O(1)$, only constant extra space is accumulated. Solution The solution uses a while loop to extract the last digit from the original number using modulo operation ($\% 10$) and then removes this digit using integer division ($/ 10$).	#1377 Combination Sum IV (Medium) - Dynamic Programming Key Insights - Transitions are made from a cell to its four adjacent cells. - If the cell moves out of bounds, it contributes to a valid path. - States can be memoized to avoid recomputation and optimize the recursive solution. Space & Time Time Complexity: $O(n \times m)$, where n is the number of rows and m is the number of columns. Space Complexity: $O(n \times m)$ for the memoization table. Solution We use a recursive dynamic programming (or memoization) approach. Define a function $dfs(i, j)$ that returns the number of ways to move out of bounds starting from cell (i, j) with movement matrix $dirs$. For each cell, we check if it's out of bounds, return 1 (base case). If not, we move in four directions ($-1, 0, 0, 1$) and are still inside, then $dfs(i+1, j)$, $dfs(i-1, j)$, $dfs(i, j+1)$, and $dfs(i, j-1)$ holds the result of the best direction and sum paths. Since results in a memoization table to avoid recomputation, then return the computed value modulo $10^9 + 7$. #1413 Longest Common Subsequence (Medium) - Dynamic Programming Key Insights - If an old color cell becomes zero, remove it from the current digits. - Add the new color to the ball and increment its frequency. - After each query, the number of distinct colors is the number of keys in the color-frequency map. Space & Time Time Complexity: $O(n)$ average for q queries, since each query does only $O(1)$ extra space hash map usage. Space Complexity: $O(n)$, only constant extra space is accumulated. Solution We solve the problem using two hash maps. The first map stores the current set of balls, and the second map stores the frequency of each active color. For every color, if the ball exists in the original frequency map, we decrement the old color's count and remove that color from the frequency map. If the new color exists, we update its ball count and its new color and increment the new color's frequency. After processing the query, the number of distinct colors is simply the number of keys remaining in the frequency map. #4267 Debounce (Medium) - JavaScript Key Insights - Each call to the debounced function resets the timer.	#470 Subsets (Medium) - Bit Manipulation Key Insights - The problem is equivalent to generating the power set of the given set. - A recursive backtracking approach can be used to explore both including and excluding each element. - Iterative bit manipulation approach is also available. - The use of sorted order and conditionally skipping elements is the trick to handle duplicates. Space & Time Time Complexity: $O(2^n \times n)$, where n is the number of elements and creating each subset may take up to $O(n)$ time. Space Complexity: $O(2^n \times n)$ for storing all subsets. The recursion stack has a maximum depth of $O(n)$. Solution This solution uses a dynamic programming approach with a 2D array dp to record the lengths of common subsequences for substrings of $text1$ and $text2$ up to dimensions $(i+1)$ and $(j+1)$ with zeros, where i and j are the lengths of text1 and text2. We iterate over both strings, comparing characters. If they match, we set $dp[i+1][j+1] = dp[i][j] + 1$, otherwise we choose the maximum from $dp[i][j+1]$ and $dp[i+1][j]$. The final answer is found at $dp[m][n]$. #500 Longest Valid Parentheses (O(n)) (Medium) - Math Key Insights - For a negative, number x to $1/x$ and to $-x$, initialize the result as -1. Loop while n is greater than 0 if the current n is odd, multiply the result by x. Square the base x. Divide n by 2 using integer division. - For a non-zero, swap the edge case when n is even with the last element in the array, then double the last element. - This avoids expensive shifting of elements. - greediness can be implemented by generating a stack and using it to store the array. Space & Time Time Complexity: $O(n)$ for the iteration solution, or $O(n \log n)$ for the implemented solution. Solution We use representation by squaring an algorithm that reduces the number of multiplications by repeatedly squaring the base and halving the exponent. Here's how	#719 Subsets II (Medium) - Bit Manipulation Key Insights - If x is negative, number x to $1/x$ and to $-x$, initialize the result as -1. Loop while n is greater than 0 if the current n is odd, multiply the result by x. Square the base x. Divide n by 2 using integer division. - For a non-zero, swap the edge case when n is even with the last element in the array, then double the last element. - This avoids expensive shifting of elements. - greediness can be implemented by generating a stack and using it to store the array. Space & Time Time Complexity: $O(n)$ for the iteration solution, or $O(n \log n)$ for the implemented solution. Solution We use representation by squaring an algorithm that reduces the number of multiplications by repeatedly squaring the base and halving the exponent. Here's how	#90 Subsets II (Medium) - Bit Manipulation Key Insights - Sort the array first so that duplicate elements are adjacent. - Use backtracking to generate all possible subsets. - Skip duplicate elements in the recursion to avoid duplicates. - The use of sorted order and conditionally skipping elements is the trick to handle duplicates. Space & Time Time Complexity: $O(2^n \times n)$ since each subset creation may take $O(n)$ time. Space Complexity: $O(2^n \times n)$ for storing the subsets. Solution The solution uses a backtracking technique that builds subsets one element at a time. The input array is sorted to bring duplicates together. During recursion, if the current character is the same as the previous one and it has not been used in the current recursion path, it is skipped at the next level. This ensures that each subset is unique. The algorithm stores each valid subset by copying the current subset state into the result list.
Space Complexity: $O(n \log n)$ - extra space for storing the computed digits. Solution To solve the problem, we simulate the conversion process similar to how you would convert an integer to a positive base. However, since the base here is -2, special handling of negative remainders is needed in each iteration, we: - Divide the current number by -2. - Calculate the remainder of the remainder is negative, adjust it by adding 2, and increment the quotient to compensate. - Append the computed remainder to a list (which represents the binary digits). - Continue until the number reduces to zero. Finally, reverse the list to form the correct binary representation from the most significant to the least significant digit. #2160 Find the Number of Distinct Colors Among All Balls (Medium) - Math Key Insights - Use a hash map to store the current color of each ball and how many balls currently use each color. - Use another hash map to store how many balls currently use each color. - When a ball is recolored, decrease the count of its old color first.	#4267 Debounce (Medium) - JavaScript Key Insights Each call to the debounced function resets the timer.	#470 Subsets (Medium) - Bit Manipulation Key Insights - The problem is equivalent to generating the power set of the given set. - A recursive backtracking approach can be used to explore both including and excluding each element. - Iterative bit manipulation approach is also available. - The use of sorted order and conditionally skipping elements is the trick to handle duplicates. Space & Time Time Complexity: $O(2^n \times n)$, where n is the number of elements and creating each subset may take up to $O(n)$ time. Space Complexity: $O(2^n \times n)$ for storing all subsets. The recursion stack has a maximum depth of $O(n)$. Solution This solution uses a dynamic programming approach with a 2D array dp to record the lengths of common subsequences for substrings of $text1$ and $text2$ up to dimensions $(i+1)$ and $(j+1)$ with zeros, where i and j are the lengths of text1 and text2. We iterate over both strings, comparing characters. If they match, we set $dp[i+1][j+1] = dp[i][j] + 1$, otherwise we choose the maximum from $dp[i][j+1]$ and $dp[i+1][j]$. The final answer is found at $dp[m][n]$. #500 Longest Valid Parentheses (O(n)) (Medium) - Math Key Insights - For a negative, number x to $1/x$ and to $-x$, initialize the result as -1. Loop while n is greater than 0 if the current n is odd, multiply the result by x. Square the base x. Divide n by 2 using integer division. - For a non-zero, swap the edge case when n is even with the last element in the array, then double the last element. - This avoids expensive shifting of elements. - greediness can be implemented by generating a stack and using it to store the array. Space & Time Time Complexity: $O(n)$ for the iteration solution, or $O(n \log n)$ for the implemented solution. Solution We use representation by squaring an algorithm that reduces the number of multiplications by repeatedly squaring the base and halving the exponent. Here's how	#719 Subsets II (Medium) - Bit Manipulation Key Insights - If x is negative, number x to $1/x$ and to $-x$, initialize the result as -1. Loop while n is greater than 0 if the current n is odd, multiply the result by x. Square the base x. Divide n by 2 using integer division. - For a non-zero, swap the edge case when n is even with the last element in the array, then double the last element. - This avoids expensive shifting of elements. - greediness can be implemented by generating a stack and using it to store the array. Space & Time Time Complexity: $O(n)$ for the iteration solution, or $O(n \log n)$ for the implemented solution. Solution We use representation by squaring an algorithm that reduces the number of multiplications by repeatedly squaring the base and halving the exponent. Here's how	#90 Subsets II (Medium) - Bit Manipulation Key Insights - Sort the array first so that duplicate elements are adjacent. - Use backtracking to generate all possible subsets. - Skip duplicate elements in the recursion to avoid duplicates. - The use of sorted order and conditionally skipping elements is the trick to handle duplicates. Space & Time Time Complexity: $O(2^n \times n)$ since each subset creation may take $O(n)$ time. Space Complexity: $O(2^n \times n)$ for storing the subsets. Solution The solution uses a backtracking technique that builds subsets one element at a time. The input array is sorted to bring duplicates together. During recursion, if the current character is the same as the previous one and it has not been used in the current recursion path, it is skipped at the next level. This ensures that each subset is unique. The algorithm stores each valid subset by copying the current subset state into the result list.			
Dynamic Programming #10 Regular Expression Matching (Hard) LeetCode - Single Edit - WalkCC - LeetCode Strs - Dynamic Programming - Recursion Unit-Theory-Design Time Complexity: $O(n \times m)$ where n is <code>length(s)</code> and m is <code>length(p)</code>, due to exploring subproblems. Space Complexity: $O(n)$ for storing the memoization table in the dynamic programming or $O(1)$ if using tabulation. Given an input string <code>s</code> and a pattern <code>p</code>, implement regular expression matching with support for <code>'.'</code> and <code>'*'</code>, where: <code>'.'</code> Matches any single character. <code>'*'</code> Matches zero or more of the preceding element. - Return a boolean indicating whether the matching covers the entire input string (not partial). Example 1: <pre>Input: s = "aa", p = "a*" Output: true Explanation: "a*" does not match the entire string "aa".</pre> Example 2:	<pre>Input: s = "aa", p = "a*" Output: true Explanation: "a*" does not match the entire string "aa".</pre> Example 2:	<pre>class Solution: def isMatch(self, s: str, p: str) -> bool: m, n = len(s), len(p) dp = [[False] * (n + 1) for _ in range(m + 1)] dp[0][0] = True for i in range(1, m + 1): for j in range(1, n + 1): if p[j-1] == s[i-1] or p[j-1] == '.': dp[i][j] = dp[i-1][j-1] elif p[j-1] == '*': if p[j-2] == s[i-1] or p[j-2] == '.': dp[i][j] = dp[i-1][j] or dp[i][j-2] else: dp[i][j] = dp[i][j-2] return dp[m][n]</pre> Key Insights - The solution involves a recursive function to match the string <code>s</code> against the pattern <code>p</code>. - The base case is when both strings are empty, return <code>True</code>. - The recursive step involves checking the current character in <code>s</code> and <code>p</code> and deciding whether to match it or skip it (if <code>*</code> is present). - The solution uses memoization to store the results of subproblems to avoid redundant calculations.	<pre>class Solution: def isMatch(self, s: str, p: str) -> bool: m, n = len(s), len(p) dp = [[False] * (n + 1) for _ in range(m + 1)] dp[0][0] = True for i in range(1, m + 1): for j in range(1, n + 1): if p[j-1] == s[i-1] or p[j-1] == '.': dp[i][j] = dp[i-1][j-1] elif p[j-1] == '*': if p[j-2] == s[i-1] or p[j-2] == '.': dp[i][j] = dp[i-1][j] or dp[i][j-2] else: dp[i][j] = dp[i][j-2] return dp[m][n]</pre> Key Insights - The solution involves a recursive function to match the string <code>s</code> against the pattern <code>p</code>. - The base case is when both strings are empty, return <code>True</code>. - The recursive step involves checking the current character in <code>s</code> and <code>p</code> and deciding whether to match it or skip it (if <code>*</code> is present). - The solution uses memoization to store the results of subproblems to avoid redundant calculations.	<pre>class Solution: def isMatch(self, s: str, p: str) -> bool: m, n = len(s), len(p) dp = [[False] * (n + 1) for _ in range(m + 1)] dp[0][0] = True for i in range(1, m + 1): for j in range(1, n + 1): if p[j-1] == s[i-1] or p[j-1] == '.': dp[i][j] = dp[i-1][j-1] elif p[j-1] == '*': if p[j-2] == s[i-1] or p[j-2] == '.': dp[i][j] = dp[i-1][j] or dp[i][j-2] else: dp[i][j] = dp[i][j-2] return dp[m][n]</pre> Key Insights - The solution involves a recursive function to match the string <code>s</code> against the pattern <code>p</code>. - The base case is when both strings are empty, return <code>True</code>. - The recursive step involves checking the current character in <code>s</code> and <code>p</code> and deciding whether to match it or skip it (if <code>*</code> is present). - The solution uses memoization to store the results of subproblems to avoid redundant calculations.	<pre>class Solution: def isMatch(self, s: str, p: str) -> bool: m, n = len(s), len(p) dp = [[False] * (n + 1) for _ in range(m + 1)] dp[0][0] = True for i in range(1, m + 1): for j in range(1, n + 1): if p[j-1] == s[i-1] or p[j-1] == '.': dp[i][j] = dp[i-1][j-1] elif p[j-1] == '*': if p[j-2] == s[i-1] or p[j-2] == '.': dp[i][j] = dp[i-1][j] or dp[i][j-2] else: dp[i][j] = dp[i][j-2] return dp[m][n]</pre> Key Insights - The solution involves a recursive function to match the string <code>s</code> against the pattern <code>p</code>. - The base case is when both strings are empty, return <code>True</code>. - The recursive step involves checking the current character in <code>s</code> and <code>p</code> and deciding whether to match it or skip it (if <code>*</code> is present). - The solution uses memoization to store the results of subproblems to avoid redundant calculations.		

Dynamic Programming

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round 9 - Low - Hard - 6 questions

Round 9

Low Priority - Hard

7 questions - 2 patterns

Dynamic Programming

Round

